Iron – libreria C++ di base

Versione 0.1

Indice generale

Introduzione....2
Piattaforme....2
Licenza....2
Linux - Compilazione ed installazione....3
Win32/Win64 – Compilazione ed installazione....3
Gestione degli errori....3
Sicurezza....5
Metodi non sicuri....6
Vincoli....7
Macro Pubbliche....7
Spazi dei nomi....10
Testata fwd.h....10
Concetti....11
Classi non sicure....18
- allocator....18
- errors::base....20
- errors::base_with_info....22
- errors::posix....23
- errors::os....24
- errors::exception....25
- raw_storage....26
- raw_wrapper....29
- unicode_conversions....31
- unicode....34

Classi sicure....37
- adapter_box....37
- array....39
- bitset....43
- box....45
- box_deleter....49
- btree_table....51
- cfg....58
- chr....61
- chr::sequence....67
- clone....69
- collector....71
- comparables::equal_to....76
- comparables::less_to....77
- enum_set....78
- error_handler....80
- hasher....83
- hash_table....84

integrals....92
integrals bounded....98
list....102
memory::condition_variable....112
memory::lock_guard....114
memory::mutex....115
memory::spin_lock....116
option....117
ref_wrapper....121
result....123
sequence....126
shared_box....132
str....137
strview....149
vector....155
Version....161

Introduzione.

`Iron` e' una libreria `c++` sperimentale che garantisce sicurezza e facilità d'uso.
La sicurezza non puo essere garantita al 100% senza intervenire sul linguaggio, ma comunque una buona approssimazione è ottenibile a patto che l'utente si attinga ad usare metodi sicuri nel modo corretto.
La garanzia sull' uso è garantito a tempo di compilazione tramite la clausole `require` e/o `static_assert` di `c++`, a tempo di esecuzione grazie all'uso di una istanza di classe `security_checker` interno alla classe desiderata e la definizione di metodi sicuri nell'uso.
Per questioni prestazionali una istanza `security_checker` è attiva solo in modalità `debug`.
Altra caratteristica è quella di evitare le copie potenzialmente costose disabilitando il costruttore/assegnamento di copia per quelle classi che lo richiedono.
In questo caso se occorre una copia il programmatore deve usare il metodo `clone()` o `try_clone()`.
Un' altrà comodità sono i metodi valutabili a tempo di compilazione.
Quasi tutti i metodi di una classe sono valutabili a tempo di compilazione,le eccezioni sono poche.
Inoltre la libreria non usa la gestione delle eccezioni a causa di varie problematiche che comporta.

Piattaforme.

La libreria è stata testata sotto `linux` con i compilatori `g++` da versione `12` a versione `15`, `clang++` versione `19` e sotto `win32/win64` con `visual c++ 2022 (visual studio 2022)`.
E' richiesta l'attivazione della modalità `c++23`.

Licenza.

La libreria è sotto licenza `MIT`.
Vedi `LICENSE`.
Nessuna linea di codice della libreria è stata scritta da una IA o copiata da altro codice ,anche open source,ma è frutto completo dell'autore.

Linux - Compilazione ed installazione.

Estrarre nella `directory <iron_dir>`

```
export EXCLUDE_COMPILERS=<list>
```
opzionale – imposta i compilatori da escludere

```
cd <iron dir>
make
```
compila e crea le librerie dinamiche e statiche per i vari compilatori `c++` trovati e non esclusi

```
make test [m=<test_name>]
```
esegue i test di verifica

```
make install [PREFIX=<dir>]
```
esegue l'installazione in `PREFIX` che ha come valore predefinito `$HOME/iron`

```
make clean
```
pulizia

Win32/Win64 – Compilazione ed installazione.

Estrarre in D:\iron_REPO

```
cd iron_REPO
build.cmd
```

costruisce le librerie dinamiche per win32 e win64 (le librerie statiche per ora non sono incluse) e gli eseguibili dei test di regressione.

```
test.cmd
```

esegue gli eseguibili dei test di regressione.

```
install.cmd <dir>
```

costruisce le libreria dinamiche ed esegue l'installazione in <dir>

Per questo ambiente e' previsto l'uso dei moduli `c++` - impostare la macro globale `USE_MODULES` a tempo di compilazione.

Gestione degli errori.

Come anticipato la libreria non usa il metodo `c++` classico delle eccezioni per gestire gli errori.
Purtroppo la gestione delle eccezioni porta con sé queste problematiche:

1) I `debugger` qualche volta hanno dei problemi ad intercettare il punto in cui è stata sollevata l'eccezione

2) Nella libreria standard la classe base `std::exception` e le classi derivate da essa non riportano nessuna informazione di dove l'eccezione è stata sollevata.

3) Se un'eccezione viene sollevata prima che la precedente sia stata catturata, `il runtime c++` chiama immediatamente `std::terminate()`, causando il crash istantaneo dell'applicazione.
Trovare il problema può essere complicato ed il codice d'uscita al sistema operativo non è determinato anche se comunque dovrebbe essere diverso da zero.

4) Decidere se una funzione solleva o non solleva eccezioni potrebbe essere complicato.
Normalmente si usa un analizzatore del codice per i casi più complessi ma non sempre sono precisi.

5) Se una funzione nella sua definizione ha l'operatore `noexcept(true)` ed invece direttamente od indirettamente viene sollevata una eccezione viene chiamato il solito `std::terminate()` comportando le stesse problematiche del punto 3.

6) In fase di manutenzione del codice una funzione potrebbe cambiare il modo di trattare gli errori.
Potrebbe passare da una funzione che non solleva eccezioni ad una che può o viceversa
In questo caso occorre rivedere tutte le chiamate dirette ed indirette alla funzione.

7)
Per una funzione che può sollevare eccezioni il compilatore può aggiungere codice extra per gestirle anche se in realtà quella funzione non solleva eccezioni.
Questo perché il compilatore guarda l'operatore `noexcept` e non sempre puo decidere se quello dichiarato dal progettista sia vero o no.

Per via di queste problematiche in fase di progetto si è deciso di non utilizzare questo meccanismo ma un' altra soluzione.
Sono state progettate due classi apposite per gestire gli errori: `error_handler` e `result`.
La classe `error_handler` ha il compito di memorizzare un errore compreso uno pseudo nessun errore.
Se un errore in una istanza di questa classe è memorizzato il suo metodo `operator bool()` ritorna falso.
Gli errori effettivi sono memorizzati internamente come istanza di una classe derivata da `unsafe::errors::base`

Esistono 3 tipi di errore corrispondenti ad altre classi

1) `posix`: errori dello standard `POSIX`, normalmente e quando è possibile questa è la modalità preferita.

2) `os`: un errore rappresentato da un codice d'errore del sistema operativo di tipo `std::int32_t`. In `UNIX/Linux` se il numero dell'errore è convertibile in `POSIX` viene istanziata una classe `posix`.

3) `exception`: la memorizzazione di una istanza `std::exception_ptr` usata per convertire una eccezione in `error_handler`

E' possibile aggiungere altre classi di errore implementando una classe che deriva da `unsafe::errors::base` o `unsafe::errors::base_with_info`

Per segnalare un errore una classe sicura espone metodi che ritornano un `error_handler` con un nome avente come prefisso `try_`.
Esempio di questi nomi sono `try_resize` della classe `vector` la cui firma è

```
[[nodiscard]] constexpr error_handler try_resize(size_t)noexcept
```

Invece nel caso in cui si voglia ritornare una determinata informazione oppure un errore si usa la classe `result` con il solito prefisso `try_` nel nome.
Ad esempio `try_clone` sempre della classe `vector` ha come firma

```
[[nodiscard]] constexpr result<vector,error_handler> try_clone(size_t n=1)const noexcept.
```

Anche `result` come `error_handler` ha un metodo `operator bool()` che ritorna vero per nessun errore.
Notare che `result` può memorizzare in questo caso una istanza `vector` o una istanza `error_handler` ed in generale può essere usata anche per altri scopi oltre alla classica memorizzazione di un errore.
Per ogni metodo il cui nome inizia con `try_` esiste un corrispondente metodo con nome senza prefisso `try_`.
Ad esempio seguendo i due esempi presentati prima abbiamo

```
constexpr vector& resize(size_t)noexcept
[[nodiscard]] constexpr vector clone(size_t n=1)const noexcept
```

In questi casi se un errore si verifica viene chiamata la macro `PANIC()` la quale emette un messaggio di errore su `stderr` comprendendo il punto di chiamata ed il programma termina restituendo al sistema operativo un codice diverso da zero.
In modalità `release` questo codice è `101`, in modalità `debug` invece viene chiamata `std::terminate()` per cui il codice d'uscita dipende dalla sua implementazione ma comunque non dovrebbe essere mai zero.
In modalità `debug` è stato scelto di usare `std::terminate()` perché i `debugger` in generale hanno la proprietà di fermare l'esecuzione alla chiamata di questa funzione.
Quando si usa una chiamata ad un metodo con prefisso `try_` ?
Se l'errore è recuperabile oppure occorre trasmetterlo al chiamante ha senso usarlo.
Altrimenti è inutile, meglio usare il corrispondente metodo senza prefisso `try_`.
E' possibile avere la stampa dello `stacktrace` della chiamata a `PANIC()` invocando l'eseguibile con la variabile d'ambiente `IRON_STACKTRACE` impostata ad un valore diverso da zero.
Ad esempio in ambiente `linux`:

```
IRON_STACKTRACE=1 ./a.out
```

Utile specialmente per analizzare un errore in ambiente di produzione.
In altri casi si è preferito usare un altro metodo per segnalare un comportamento non corretto di una funzione senza usare un gestore degli errori.

Ad esempio sempre in `vector` il metodo `nth` ha come firma

```
[[nodiscard]] option<ref_t> nth(size_t idx)noexcept
```

Se il valore di `idx` non è corretto viene restituita una istanza di `option` vuota .
Nel caso venga tentato di estrarne il valore viene chiamata la solita macro `PANIC()`
Un altro caso simile, sempre di vector, è

```
[[nodiscard]] ref_t at(size_t idx)noexcept
```

Se il valore di `idx` non è corretto viene chiamata `PANIC()`

Sicurezza.

La sicurezza è garantita da un gestore di sicurezza, una istanza della classe `security_checker,` ma prima di proseguire occorre introdurre una terminologia.
Con classe principale si identifica una classe che possiede i proprio dati.
Con classe referente si identifica una classe che possiede dei riferimenti ai dati di una classe principale.
Esempi di classi principali sono `vector` e `list` mentre esempi di classi referenti sono `sequence,string_view` e `ref_wrapper`
Quando si usa un metodo non statico di una istanza di una classe con incapsulata una istanza del gestore di sicurezza per ogni chiamata vengono eseguiti dei controlli di verifica e in caso di fallimento viene chiamata la macro `PANIC()` con un opportuno messaggio di errore.
Per questioni prestazionali e dimensioni del codice il gestore di sicurezza è eseguito solo in modalità `debug`.
Nessun codice avente a che fare con il gestore di sicurezza viene compilato in modalità `release`.
Un utente puo implementare una propria classe con incapsulato un gestore di sicurezza usando apposite macro.
Le problematiche di sicurezza gestite sono sostanzialmente 3.

1) Uso di un temporaneo

Ad esempio:

```
iron::vector<int> v1,v2;
v1 = std::move(v2);
//now v2 is marked temporary
v2.clear();   //PANIC - use a temporary object
v2 = v1.clone();
// OK - now v2 is not a temporary
v2.clear();  //OK
```

2) Una istanza di una classe referente tenta di usare dei dati di una istanza della propria classe principale non più in vita

Ad esempio:

```
iron::vector<int> v1; //v1 is a master class instance
auto seq = v1.seq(); //seq is a reference class instance of v1
{
        auto v2 = iron::collector::make_vector<int>(1,2,3); //make a vector
                                                             // with elements
                                                            //1,2,3
        seq = v2.seq(); // now seq is a reference class instance of v2
} //now v2 is destroyed and deallocated
auto r=*seq; //PANIC - the master object is not alive
```

3) Una istanza di una classe principale tenta di modificare la propria struttura dati interna che potrebbe comportare una invalidazione dei riferimenti ai dati di una o piu istanze di almeno una classe referente

Ad esempio:

```
iron::vector<int> v1;
auto seq=v1.seq();
v1.push_back(1);  //OK - seq is empty, is not a reference to v1
seq = v1.seq();
//now seq is not empty and it's a v1 reference
//use seq
v1.push_back(2);  //PANIC - master object has reference active
```

Per rendere valido questo esempio:

```
iron::vector<int> v1;
auto seq=v1.seq();
v1.push_back(1);  //OK - seq is empty, is not a reference
seq = v1.seq();
//now seq is not empty
//use seq
seq.reset();
//now seq is empty
v1.push_back(2);  //OK - seq is empty,is not a reference
```

Oppure:

```
iron::vector<int> v1;
v1.push_back(1);
{
        auto seq = v1.seq();
        //now seq is not empty
        //use seq
}  //seq is destroyed and deallocated
v1.push_back(2);  //OK - not reference actives
```

Metodi non sicuri

Alcune classi sicure della libreria hanno dei metodi ritenuti non sicuri nell'uso
I metodi non sicuri hanno nel nome `unsafe` o `unchecked`.
I metodi `unchecked` sono metodi la cui sicurezza sull'uso degli argomenti di ingresso, impliciti o espliciti non è garantita.
Ad esempio i metodi che in ingresso hanno una coppia di iteratori non sono sicuri perché l'uso di un iteratore non è sicuro.
I metodi `unsafe` sono metodi che ritornano un tipo non sicuro.
Ad esempio un metodo che ritorna un puntatore od una referenza non è ritenuto sicuro (l'unica eccezione è nella classe `ref_wrapper`).
Questi metodi esistono perchè potrebbero essere utili in codice di basso livello ma il loro uso sta completamente nella responsabilità del programmatore.
Ad esempio la solita classe `vector` ha un metodo `value_t& unchecked_nth(size_t idx)noexcept.`
In caso il valore di `idx` non sia valido il valore ritornato dalla chiamata al metodo è indeterminato.
I metodi non sicuri comunque eseguono dei controlli di validità ma solo in modalità `debug`.

Vincoli.

In generale una classe utente che deve interagire con una classe della libreria deve sottostare ad alcuni vincoli.
Questi vincoli non sono limitativi ma,anzi, sono fondamentali per una buona progettazione della classe utente stessa e sono imposti attraverso dichiarazioni `static_assert`.
Le classi della libreria potrebbero introdurre ulteriori vincoli o di meno ma in generale i vincoli imposti sono:

1) La classe utente deve essere definita, una semplice dichiarazione non è sufficiente.

2) Il tipo di un contenitore non deve essere immutabile,non deve essere una referenza e non deve essere un `builtin array`.

3) Una classe utente `T` deve avere

```
T(T&&)noexcept
T& operator=(T&&)noexcept
swap(T&)noexcept
~T()noexcept
```

ovvero metodi che non devono sollevare eccezioni.

Nessun problema invece per `T(const T&)` e `T& operator=(const T&)`,questi metodi possono sollevare eccezioni.
Una classe utente può implementare i metodi per la copia `try_clone()` e `clone()` in modo da soddisfare il concetto `cloneable;` in questo caso nelle operazioni di copia verranno date precedenza a questi metodi.
Sta al progettista della classe garantire la congruità tra una operazione di clone ed un eventuale metodo `T(const T&)` o `T& operator=(const T&)`.
Altri vincoli sono imposti a tempo di compilazione ai singoli metodi usando la clausola `requires`.

Macro Pubbliche.

`IRON_LIKELY`
Espande in `[[likely]]`

`IRON_UNLIKELY`
Espande in `[[unlikely]]`

`IRON_EBCDIC`
Definita se la codifica di un char è `EBCDIC`
Nota, l'implementazione con `EBCDIC` non è completa

`IRON_NO_UNIQUE_ADDRESS`
Espande in `[[msvc::no_unique_address]]` per `visual c++`,`[[no_unique_address]]` per altri

`TODO()`
In modalità `debug` scrive su `stderr` `"TODO in (<file>,<line>)"` e prosegue
In modalità `release` non fa niente

`PANIC(<text>)`
Scrive su `stderr` un messaggio `<text>(<file>,<line>)`
Se la variabile d'ambiente `IRON_STACKTRACE` è impostata ad un valore diverso da zero scrive su `stderr` uno `stacktrace` delle chiamate.
Infine in modalità `release` chiama `exit(101)` mentre, in modalità `debug`, chiama `std::terminate()`.
Attenzione: purtroppo le chiamate ad `exit()` e `std::terminate()` non eseguono la chiamata dei distruttori sugli oggetti allocati sullo `stack` ed eventuali `thread` attivi vengono brutalmente terminati.
Questo problema è comunque relativo visto che `PANIC()` in un programma corretto non viene mai chiamata ma, comunque, attenzione ad eventuali risorse allocate come `lock, file, socket`,ecc..
Nel caso, l'utente puo impostare un `handler` sia su `std::terminate()` che su `exit()`

`PANIC_IF(<bool expr>,<text>)`
Se `<bool expr>` è vera chiama `PANIC(<text>)`

`PANIC_UNLESS(<bool expr>,<text>)`
Se `<bool expr>` è falsa chiama `PANIC(<text>)`

`ASSERT(<bool expr>)`
Equivalente a `PANIC_UNLESS(<bool expr>," <expr>: assert failed in (<file>,<line>)" )`

`DEBUG_ASSERT(<bool expr>)`
Come `ASSERT(<bool expr>)` ma attiva solo in modalità `debug`

`ASSERT_EQ(<expr1>,<expr2>)`
Equivalente a `ASSERT(<expr1> == <expr2>)`

`DEBUG_ASSERT_EQ(<expr1>,<expr2>)`
Come `ASSERT_EQ(<expr1>,<expr2>)` ma attiva solo in modalità `debug`

`ASSERT_NE(<expr1>,<expr2>)`
Equivalente a `ASSERT(!(<expr1> == <expr2>))`

`DEBUG_ASSERT_NE(<expr1>,<expr2>)`
Come `ASSERT_NE(<expr1>,<expr2>)` ma attiva solo in modalità `debug`

`ERROR_HANDLER(<error expr>)`
Costruisce un oggetto `error_handler`

`RE(<expr>)`
Se `<expr>` e' falsa ritorna `<expr>` convertita in una istanza della classe `error_handler`

`RR(<expr>)`
Se `<expr>` e' falsa ritorna `<expr>` convertita in una istanza della classe `result`

`RP(<result>)`
Estrae da `<result>` la parte `T` se impostata altrimenti chiama `PANIC()` con uno specifico messaggio di errore

`RVP(<expr1>,<expr2>)`
Se `<expr1>` e' vero ritorna `<expr2>` altrimenti chiama `PANIC()` con uno specifico messaggio di errore

`USE_MODULES`
Definire questa macro a livello globale se si vuole usare i moduli `c++`
Per ora i moduli sono stati testati solo nell'ambiente `win32`

`IRON_MODULE_EXPORT`
Se la macro `USE_MODULES` è definita viene espansa nella parola chiave `export`

`IRON_DLL_EXPORT`
Se la macro `IRON_EXPORTS` è definita viene espansa nell'attributo di esportazione altrimenti nell'attributo di importazione.
L'attributo di importazione/esportazione è dipendente dal compilatore.
Questa macro serve per esportare od importare funzioni da una libreria.

`IRON_SECURITY_CHECKER`
Questa macro è impostata a uno in modalità `debug`, viceversa a zero.
Se questa macro è impostata a uno le seguenti macro sono attivate chiamando un `PANIC()` in caso di fallimento con un opportuno messaggio di errore.
Questi test vengono eseguiti su una istanza di una classe che contiene una istanza della classe `security_checker` altrimenti vengono ignorati.

`IRON_SECURITY_CHECK_TEMPORARY(<obj>)`
Fallisce per `<obj>` marcato come temporaneo

`IRON_SECURITY_CHECK_TEMPORARY_THIS()`
Equivalente a `IRON_SECURITY_CHECK_TEMPORARY(*this)`

`IRON_SECURITY_MARK_TEMPORARY(<obj>)`
Marca `<obj>` come temporaneo

`IRON_SECURITY_CHECK_REF_ACTIVES(<obj>)`
Fallisce se `<obj>` ha delle referenze attive

`IRON_SECURITY_CHECK_REF_ACTIVES_THIS()`
Equivalente a `IRON_SECURITY_CHECK_REF_ACTIVES(*this)`

`IRON_SECURITY_CHECK_MASTER_LIVE(<obj>)`
Fallisce se l'instanza della classe principale di `<obj>` è stata distrutta

`IRON_SECURITY_CHECK_MASTER_LIVE_THIS()`
Equivalente a `IRON_SECURITY_CHECK_MASTER_LIVE(*this)`

`IRON_SECURITY_CHECK_TEMPORARY_ARGS(<args>)`
Equivalente a `IRON_SECURITY_CHECK_TEMPORARY(<arg>)` per ogni valore di `<arg>`

`IRON_SECURITY_DCL()`
espande nella dichiarazione di una istanza della classe `security_checker`

`IRON_SECURITY_SWAP(<obj>)`
esegue uno scambio tra l'istanza `security_checker` corrente e quella che appartiene ad `<obj>`

`IRON_SECURITY_METHOD()`
espande nella dichiarazione del metodo `checker()` che ritorna una referenza immutabile dell'istanza `security_checker` dichiarato nella classe corrente

`IRON_SECURITY_ASSIGN(<obj>)`
assegna l'istanza `security_checker` di `<obj>` all'istanza `security_checker` della classe corrente

`IRON_SECURITY_MAKE_REF(<obj>)`
marca l'istanza `security_checker` corrente come referente di `<obj>`

`IRON_SECURITY_MAKE_REF_TO(<obj_ref>,<obj>)`
marca l'istanza `security_checker` di `<obj_ref>` come referente di `<obj>`

`IRON_SECURITY_REQ()`
Espande in `requires(iron::concepts::has_security_checker<_MASTER>)`

`IRON_SECURITY_RESET_THIS()`
: marca l'istanza `security_checker` corrente come non referente

`IRON_SECURITY_RESET(<obj>)`
: marca l'istanza `security_checker` di `<obj>` come non referente

Spazi dei nomi.

La libreria usa i seguenti spazi dei nomi.
`iron` – lo spazio dei nomi principale in cui sono definiti i contenitori e le classi accessorie
`iron::concepts` – lo spazio dei nomi in cui sono definiti i concetti aggiuntivi a quelli della libreria standard
`iron::memory` – lo spazio dei nomi in cui sono definite classi che hanno a che fare con la gestione della memoria
`iron::unsafe` – lo spazio dei nomi in cui risiedono classi il cui uso non è sicuro
`iron::unsafe::iterators` – lo spazio dei nomi in cui risiedono iteratori vari,l'uso non è sicuro.
`iron::unsafe::errors` – lo spazio dei nomi in cui sono definiti i gestori degli errori, l'uso non è sicuro.

Ulteriori spazi di nomi possono essere aggiunti in futuro

Testata fwd.h

Questa testata contiene le definizioni di tutte le classi e funzioni pubbliche della libreria.
Queste definizioni sono spiegate in dettaglio per ogni classe.

Concetti.

La libreria oltre ad includere ed usare i concetti della libreria standard definisce questi nuovi concetti nello spazio dei nomi `iron::concepts`. La testata da includere è `iron/concepts.h`

`binary_data<T>`
: Vero se `T` è `unsigned char`

`basic_charset<T>`
: Vero se `T` è `un char`

`utf8_charset<T>`
: Vero se `T` è un `utf8`

`utf16_charset<T>`
: Vero se `T` è un `utf16`

`utf32_charset<T>`
: Vero se `T` è un `utf32`

`varying_charset<T>`
: Vero se `T` è un `utf8` o `utf16`

`fixed_charset<T>`
: Vero se `T` è un `utf32` o `basic_charset`

`charset<T>`
: Vero se `T` è un `fixed_charset<T>` o un `varying_charset<T>`

`charset_forward_iterator<T>`
: Vero se `std::forward iterator<T>` è soddisfatto e `T::operator*()` ritorna un `charset` o un riferimento ad un `charset`

`numeric_integral<T>`
: Vero per `std::integral<T>` escludendo `charset<T>`, i puntatori ed il tipo booleano

`std_pair<T>`
: Vero se `T` è la classe `std::pair`

`proxy_pair<T>`
: Vero se `T` è la classe `proxy_pair`

`cloneable<T>`
: Vero se `T` ha i metodi `T clone()const noexcept` e `result<T,error_handler> try_clone()const noexcept`

`std_tuple<T>`
: Vero se `T` è la classe `std::tuple`

`bool_predicate<F,T>`
: Vero se `F` è una funzione `bool(const T&)noexcept`

`bit_set<T>`
: Vero se `T` è la classe `bit_set`

`base_integral<T>`
: Vero se `T` è la classe `base_integral`

`base_integral_bounded<T>`
: Vero se `T` è la classe `base_integral_bounded`

`has_security_checker<T>`
: Vero se `T` è una classe con una istanza della classe `security_checker`

`sequence<T>`
: Vero se `T` e' la classe `sequence`

`charset_sequence<T>`
: Vero se `sequence<T>` è soddisfatto e `T::operator*()` ritorna un `charset` o un referimento ad un `charset`

`fwd_sequence_maker<T>`
: Vero se `T` è una classe con metodi che ritornano una sequenza il cui iteratore è `forward`

`bd_sequence_maker<T>`
: Vero se `T` è una classe con metodi che ritornano una sequenza il cui iteratore è `bidirectional`

`raw_sequence_maker<T>`
: Vero se `T` è una classe con metodi che ritornano una sequenza il cui tipo è grezzo. Questo implica avere metodi `T::raw_seq()noexcept` e `T::craw_seq()noexcept`

`nothrow_equality_comparable<T>`
: Vero `std::equality_comparabile<T>` è soddisfatto senza sollevare eccezioni

`nothrow_equality_comparable_with<T1,T2>`
: Vero se `std::equality_comparable_with<T1,T2>` è soddisfatto senza sollevare eccezioni

`less_comparable<T>`
: Vero se `T` è confrontabile per minoranza

`nothrow_less_comparable<T>`
: Vero se `less_comparable<T>` è soddisfatto senza sollevare eccezioni

`less_comparable_with<T1,T2>`
: Vero se `T1` è confrontabile per minoranza con `T2`

`nothrow_less_comparable_with<T1,T2>`
: Vero se `less_comparable_with<T1,T2>` è soddisfatto senza sollevare eccezioni

`constructible_at<T,ARGS...>`
: Vero se `T` è costruibile da `ARGS`…

`nothrow_constructible_at<T,ARGS...>`
: Vero se `constructible<T,ARGS...>` è soddisfatto senza sollevare eccezioni

`std_allocator<T>`
: Vero se `T` è un allocatore che segue i dettami della libreria standard

`allocator<T>`
: Vero se `T` è un allocatore che segue i dettami della libreria

`iron_allocator<T>`
: Vero se `T` è la classe `unsafe::allocator`

`reverse_iterator<T>`
: Vero se `T` è un `reverse_iterator` della libreria standard

`box_deleter<T>`
: Vero se `T` è una classe `deleter` per la classe `box` e `shared_box`

`lock<T>`
: Vero se `T` ha i metodi `lock()noexcept` ed `unlock()noexcept`

`nothrow_convertible_to_integral<T>`
: Vero se `T` è convertibile in un `integral` senza sollevare eccezioni

`hashable<H,T>`
: Vero se `H` ha un metodo `std::size_t operator()(const T&)noexcept`

`comparable<C,T>`
: Vero se `C` ha un metodo `bool operator()(const T&,const T&)noexcept`

`condition_variable<T>`
: Vero se `T` è la classe `condition_variable` della libreria

`error_handler<T>`
: Vero se `T` è la classe `error_handler`

`vector<T>`
: Vero se `T` è la classe `vector` della libreria

`array<T>`
: Vero se `T` è la classe `array` della libreria

`vector_iterator<T>`
`array_iterator<T>`
: Vero se `T` è la classe `contiguos_iterator`

`fwd_list<T>`
: Vero se `T` è la classe `fwd_list` della libreria

`list<T>`
: Vero se `T` è la classe `list` della libreria

`base_list<T>`
: Vero per `fwd_list<T>` o `list<T>`

`list_iterator<T>`
: Vero se `T` è la classe `list_iterator`

`hash_map<T>`
: Vero se `T` è la classe `hash_map`

`multi_hash_map<T>`
: Vero se `T` è la classe `multi_hash_map`

`base_hash_map<T>`
: Vero per `hash_map<T>` o `multi_hash_map<T>`

`hash_set<T>`
: Vero se `T` è la classe `hash_set`

`multi_hash_set<T>`
: Vero se `T` è la classe `multi_hash_set`

`base_hash_set<T>`
: Vero per `hash_set<T>` o `multi_hash_set<T>`

`hash_table<T>`
: Vero per `base_hash_set<T>` o `base_hash_map<T>`

`btree_map<T>`
: Vero se `T` è la classe `btree_map`

`multi_btree_map<T>`
: Vero se `T` è la classe `multi_btree_map`

`base_btree_map<T>`
: Vero per `btree_map<T>` o `multi_btree_map<T>`

`btree_set<T>`
: Vero se `T` è la classe `btree_set`

`multi_btree_set<T>`
: Vero se `T` è la classe `multi_btree_set`

`base_btree_set<T>`
: Vero per `btree_set<T>` o `multi_btree_set<T>`

`btree_table<T>`
: Vero per `base_btree_set<T>` o `base_btree_map<T>`

`str<T>`
: Vero se `T` è la classe `base_str`

`safe_str<T>`
: Vero per `str<T>` con il metodo `safe()` che ritorna vero

`unsafe_str<T>`
: Vero per `str<T>` con il metodo `safe()` che ritorna falso

`str_view<T>`
: Vero se `T` è la classe `base_str_view`

`safe_str_view<T>`
: Vero per `str_view<T>` con il metodo `safe()` che ritorna vero

`unsafe_str_view<T>`
: Vero per `str_view<T>` con il metodo `safe()` che ritorna falso

`raw_str<T>`
: Vero se `T` è la classe `raw_str`

`box<T>`
: Vero se `T` è la classe `box`

`rc<T>`
: Vero se `T` è la classe `rc`

`arc<T>`
: Vero se `T` è la classe `arc`

`shared_box<T>`
: Vero per `rc<T>` o `arc<T>`

`rcw<T>`
: Vero se `T` è la classe `rcw`

`arcw<T>`
: Vero se `T` è la classe `arcw`

`weak_box<T>`
: Vero per `rcw<T>` o `arcw<T>`

`raw_vector<T>`
: Vero se `T` è la classe `raw_vector`

`list_node<T>`
: Vero se `T` è la classe `list_node`

`raw_list<T>`
: Vero se `T` è la classe `raw_list`

`adapter_box<T>`
: Vero se T è la classe `adapter_box`

`raw_hash_table<T>`
: Vero se `T` è la classe `raw_hash_table`

`btree_node<T>`
: Vero se `T` è la classe `node` per `btree`

`cfg<T>`
: Vero se `T` è la classe `cfg`

`chr<T>`
: Vero se `T` è la classe `chr`

`chr_sequence<T>`
: Vero se `T` è la classe `sequence` di `chr`

`mutex<T>`
: Vero se `T` è la classe `mutex` della libreria

`recursive_mutex<T>`
: Vero se `T` è la classe `recursive_mutex`

`base_mutex<T>`
: Vero per `mutex<T>` o `recursive_mutex<T>`

`spin_lock<T>`
: Vero se `T` è la classe `spin_lock`

`recursive_spin_lock<T>`
: Vero se `T` è la classe `recursive_spin_lock`

`base_spin_lock<T>`
: Vero per `spin_lock<T>` o `recursive_spin_lock<T>`

`option<T>`
: Vero se `T` è la classe `option`

`result<T>`
: Vero se `T` è la classe `result`

`lock_guard<T>`
: Vero se `T` è la classe `lock_guard` della libreria

`ref_wrapper<T>`
: Vero se `T` è la classe `ref_wrapper`

`security_checker<T>`
: Vero se `T` è la classe `security_checker`

`unicode_iterator<T>`
: Vero se `T` è la classe `unicode_iterator`

Classi non sicure.

Nello spazio dei nomi `iron::unsafe` sono definite classi il cui uso può essere utile ma contemporaneamente il loro uso non e' considerato sicuro per cui è sconsigliato l'uso se non per codice di basso livello.
Non vengono qui descritte tutte le classi non sicure ma solo quelle ritenute utili.
Queste classi non incorporano e non usano un gestore di sicurezza.

`allocator`

Uso:

allocare/deallocare elementi consecutivi nella memoria dinamica

Testata:

```
iron/unsafe/allocator.h
```

Sinossi:

```
template <typename T> class base_allocator final
```

Alias esterni:

```
template <typename T> using allocator = base_allocator<T>
```

Alias:

```
using value_t = T
using value_type = value_t
using pointer = value_type*
using size_t = std::size_t
using size_type = size_t
using diff_t = std::difference_type
using difference_type = diff_t
```

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
```

Costruttori:

```
constexpr base_allocator()noexcept

template<typename  U>
constexpr base_allocator(const base_allocator<U>& v)noexcept

constexpr base_allocator(const base_allocator&)noexcept
constexpr base_allocator(base_allocator&&)noexcept
```

Distruttore:

```
constexpr ~base_allocator()
```

Metodi:

```
constexpr base_allocator& operator=(const base_allocator&)noexcept
constexpr base_allocator& operator=(base_allocator&&)noexcept
```

```
template <bool init=false>
[[nodiscard]] constexpr pointer allocate(size_t n)noexcept
```

Alloca nella memoria dinamica `n` elementi consecutivi di tipo `value_t`
Se `n` è zero o in caso di memoria insufficiente ritorna un puntatore nullo.
In modalità non a tempo di compilazione se `init` è vero la memoria allocata viene impostata ai valori `0xFD`.

```
template <bool reset=false>
constexpr void deallocate(pointer p, size_t n)noexcept
```

Dealloca dalla memoria dinamica `n` elementi consecutivi di tipo `value_t`
In modalità non a tempo di compilazione se `reset` è vero, prima di deallocare la memoria viene impostata ai valori `0xDF`.
Questo viene fatto per impedire attacchi di sicurezza su dati sensibili.
Purtroppo è rischioso impostare `reset` a valore vero come predefinito in quanto non è possibile garantire che il valore puntato da `p` sia effettivamente di tipo `value_t`.

```
[[nodiscard]] constexpr bool operator==(const base_allocator&)const noexcept
```

Operatore di confronto, ritorna sempre vero

errors::base

Uso:

la classe base per memorizzare un errore

Testata:

```
iron/unsafe/errors.h
```

Sinossi:

```
class base
```

Alias interni:

```
using line_number_t = std::uint_least32_t
using size_t=std::size_t
```

Costanti statiche:

```
constexpr size_t max_code_sz = 20
constexpr size_t max_path_sz = 256
```

Distruttore:

```
virtual ~base()=default
```

Costruttori:

```
base(const base&) = delete
```

Metodi:

```
base& operator=(const base&)=delete
```

```
[[nodiscard]] const char* code()const noexcept
```

Ritorna il codice d'errore in formato stringa
Chiama `virtual_code()`

```
[[nodiscard]] const char8_t* description()const noexcept
```

Ritorna la descrizione dell' errore
Chiama `virtual_description()`

```
[[nodiscard]] base* const clone()const noexcept
```

Ritorna una copia dell' errore
Puo ritornare `NULL` in caso di fallimento
Chiama `virtual_clone()`

```
[[nodiscard]] explicit operator bool()const noexcept
```

Ritorna vero se l'errore non è impostato
Chiama `virtual_is()`

`[[nodiscard]] const char* file()const noexcept`

Ritorna il nome del file troncato eventualmente a `max_path_sz`
Chiama `virtual_file()`

`[[nodiscard]] line_number_t line()const noexcept`

Ritorna il numero di linea
Chiama `virtual_line()`

`[[nodiscard]] const char* func()const noexcept`

Ritorna il nome della funzione troncato eventualmente a `max_path_sz`
Chiama `virtual_func()`

Costruttori protetti:

```
base()noexcept = default
base(base&&)noexcept
```

Metodi protetti:

```
base& operator=(base&&)noexcept
void swap(base&)noexcept
```

`[[nodiscard]] virtual const char* virtual_code()const noexcept`

Ritorna “”

`[[nodiscard]] virtual const char8_t* virtual_description()const noexcept`

Ritorna `u8`“”

`[[nodiscard]] virtual base* const virtual_clone()const noexcept`

Ritorna `NULL`
Nelle classi che ereditano ritorna un puntatore dell’istanza clonata in memoria dinamica usando `unsafe::allocator`
E’ responsabilità del chiamante distruggere e liberare la memoria dopo l’uso.
Può ritornare `NULL` in caso di memoria insufficiente

`[[nodiscard]] virtual bool virtual_is()const noexcept=0`

Ritorna vero se l’errore non è impostato

`[[nodiscard]] virtual const char* virtual_file()const noexcept`

Ritorna “”

`[[nodiscard]] virtual line_number_t virtual_line()const noexcept`

Ritorna zero

`[[nodiscard]] virtual const char* virtual_func()const noexcept`

Ritorna “”

errors::base_with_info

Uso:

memorizzare il punto in cui si è verificato un errore

Testata:

`iron/unsafe/errors.h`

Sinossi:

```
class base_with_info : public base
```

Distruttore:

```
~base_with_info()=default
```

Costruttori:

```
base_with_info(const base_with_info&) = delete
```

Costruttori protetti:

```
base_with_info()noexcept = default

base_with_info(base_with_info&& _v)noexcept


base_with_info(
        const char* file_name
        , line_number_t line
        , const char* func_name
)noexcept
```

I valori di `file_name` e/o `func_name` possono essere `NULL`
Se la lunghezza compreso `\0` finale supera `max_path_sz` la stringa viene troncata

Metodi protetti:

```
base_with_info& operator=(base_with_info&&)noexcept
void swap(base_with_info&)noexcept
[[nodiscard]] const char* virtual_file()const noexcept override
[[nodiscard]] line_number_t virtual_line()const noexcept override
[[nodiscard]] const char* virtual_func()const noexcept override
```

Metodi:

```
base_with_info& operator=(const base_with_info&) = delete
```

errors::posix

Uso:

memorizzare un errore `POSIX`

Testata:

```
iron/unsafe/errors.h
```

Sinossi:

```
class posix final : public  base_with_info
```

Alias interni:

```
using code_t = std::errc
```

Costanti statiche:

```
constexpr code_t code_ok = code_t(0)
```

Metodi statici:

```
[[nodiscard]] constexpr bool make_code(
        code_t& out
        ,std::underlying_type_t<code_t> c
)noexcept
```

Converte `c` in codice `POSIX`
Ritorna falso se `c` è zero o non è un valore `POSIX`.

```
[[nodiscard]] constexpr std::underlying_type_t<code_t>
numeric_code(code_t code)noexcept
```

Converte `code` in forma numerica

Distruttore:

```
~posix()=default
```

Costruttori:

```
posix()noexcept = default
posix(
        code_t code
        ,const char* file_name
        ,line_number_t line
        ,const char* func_name
)noexcept
```

```
posix(posix&&)noexcept
posix(const posix&) = delete
```

Metodi:

```
posix& operator=(posix&&)noexcept
posix& operator=(const posix&)=delete
void swap(posix&)noexcept
```

```
[[nodiscard]] std::underlying_type_t<code_t> numeric_code()const noexcept
```

Ritorna il codice numerico corrente

errors::os

Uso:

memorizzare un errore del sistema operativo

Testata:

```
iron/unsafe/errors.h
```

Sinossi:

```
class os final : public  base_with_info
```

Alias interni:

```
using code_t = std::int32_t
```

Costanti statiche:

```
constexpr code_t code_ok = {}
```

Distruttore:

```
~os()=default
```

Costruttori:

```
os()noexcept = default
os(
        code_t code
        ,const char* file_name
        ,line_number_t line
        ,const char* func_name
)noexcept


os(os&&)noexcept
os(const os&) = delete
```

Metodi:

```
os& operator=(os&&)noexcept
os& operator=(const os&)=delete
void swap(os&)noexcept

[[nodiscard]] code_t code()const noexcept
```

Ritorna il codice d’errore corrente

errors::exception

Uso:

memorizzare una eccezione

Testata:

`iron/unsafe/errors.h`

Sinossi:

```
class exception final : public  base_with_info
```

Alias interni:

```
using exception_ptr_t = std::exception_ptr
```

Distruttore:

`~exception()`

Costruttori:

```
exception()noexcept = default
exception(
        exception_ptr_t&&
        ,const char* file_name
        ,line_number_t line
        ,const char* func_name
)noexcept


exception(exception&&)noexcept
exception(const exception&) = delete
```

Metodi:

```
exception& operator=(exception&&)noexcept
exception& operator=(const exception&)=delete
void swap(exception&)noexcept
```

```
[[nodiscard]] const exception_ptr_t& current_exception()const noexcept
```

Ritorna l'eccezione
L'eccezione deve essere impostata altrimenti viene chiamata `PANIC()`

```
void rethrow_exception()const
```

Risolleva l'eccezione
L'eccezione deve essere impostata altrimenti viene chiamata `PANIC()`

```
[[nodiscard]] posix::code_t to_posix()const noexcept
```

Converte l'eccezione in errore `POSIX`
Se la conversione non avviene ritorna `posix::code_ok`

```
[[nodiscard]] os::code_t to_os()const noexcept
```

Converte l'eccezione in errore `os`
Se la conversione non avviene ritorna `os::code_t{}`

raw_storage

Uso:

per operazioni per lo più sulla memoria grezza

Testata:

`iron/unsafe/raw_storage.h`

Sinossi:

```
template <typename T> class raw_storage final
```

Alias interni:

```
using value_t = _T
using size_t = std::size_t
using ptrdiff_t = std::ptrdiff_t
using difference_type = ptrdiff_t
using error_handler_t = error_handler
```

Vincoli:

```
sizeof(value_t) <
static_cast<std::size_t>(std::numeric_limits<std::ptrdiff_t>::max())
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
std::is_nothrow_destructible_v<value_t>
```

Metodi statici:

```
[[nodiscard]] constexpr size_t max_size()noexcept
```

Ritorna il massimo valore per allocare elementi consecutivi di tipo `value_t`

```
[[nodiscard]] constexpr error_handler_t
default_extension_policy(
                              size_t& new_capacity
                              ,size_t current_capacity
                              ,size_t additional
                        )noexcept
```

Imposta in `new_capacity` la nuova capacità dati la capacità corrente ed il numero di elementi da aggiungere
In caso di memoria insufficiente una istanza `error_handler_t` viene impostata con l'errore `POSIX not_enough_memory`

```
template <typename A>
[[nodiscard]] constexpr error_handler_t allocate(value_t*& p, size_t n,A a)noexcept
requires concepts::allocator<A>
```

Alloca `n` elementi consecutivi di tipo `value_t` dalla memoria dinamica e ritorna in `p` l'indirizzo al primo elemento.
`p` deve essere nullo prima della chiamata.
Se `n` è zero viene fatto un ritorno di una istanza `error_handler_t` senza errore impostato e `p` rimane nullo.
In caso di memoria insufficiente una istanza `error_handler_t` viene impostata con l'errore `POSIX not_enough_memory` e `p` rimane nullo.
Non e' necessario che `A::value_t` sia `value_t` in quanto viene fatta una riallocazione al tipo `value_t`

```
template <bool reset=false,typename A>
constexpr void deallocate(value_t* p, const size_t n,A a)noexcept
requires (concepts::allocator<A>)
```

Dealloca `n` elementi consecutivi di tipo `value_t` in `p` dalla memoria dinamica
`p` non deve essere nullo se `n > 0`.
Non e' necessario che `A::value_t` sia `value_t` in quanto viene fatta una riallocazione al tipo `value_t`.
Per `reset` riferirsi a `A::deallocate()`

```
template <typename A>
[[nodiscard]] constexpr error_handler_t reallocate(
                                        value_t*& p
                                        , size_t& current_capacity
                                        , const size_t n
                                        , size_t new_capacity
                                        ,A a
                                    )noexcept
requires (
               concepts::allocator<A> &&
               std::is_nothrow_move_constructible_v<value_t>
       )
```

Rialloca in `p` `new_capacity` elementi consecutivi allocati nella memoria dinamica da una dimensione di `current_capacity` con `new_capacity > current_capacity`.
`n` indica il numero di elementi effettivi allocati
`p` non deve essere nullo se `current_capacity > 0`.
In caso di memoria insufficiente una istanza `error_handler_t` viene impostata con l'errore `POSIX not_enough_memory`
In `current_capacity` viene ritornata la nuova capacità
Non e' necessario che `A::value_t` sia `value_t` in quanto viene fatta una riallocazione al tipo `value_t`
L'operazione è transazionale

```
template <typename A>
[[nodiscard]] constexpr error_handler_t shrink_to_fit(
                                        value_t*& p
                                        , size_t& current_capacity
                                        , const size_t n
                                        ,A a
                                        )noexcept
requires (
       concepts::allocator<A> &&
       std::is_nothrow_move_constructible_v<value_t>
       )
```

Tronca la capacità di `p` a `n` elementi effettivi con `current_capacity > n`
`p` non deve essere nullo se `current_capacity > 0`.
Ritorna in `current_capacity` la nuova capacita ed in `p` il nuovo `array`
Se `n` è zero `p` viene deallocato ed impostato a nullo e `current_capacity` a zero
In caso di memoria insufficiente una istanza `error_handler_t` viene impostata con l'errore `posix not_enough_memory`
Non e' necessario che `A::value_t` sia `value_t` in quanto viene fatta una riallocazione al tipo `value_t`
L'operazione è transazionale

```
constexpr value_t* uninitialized_move_n(
                                    value_t* to
                                    , size_t n
                                    , value_t* from
                                    )noexcept
requires std::is_nothrow_move_constructible_v<value_t>
```

Costruisce `n` elementi `rvalue` da `from` a `to` di memoria non inizializzata
Ritorna `to + n`

```
constexpr value_t* uninitialized_move_and_destroy_n(
                                                    value_t* to
                                                    , const size_t n
                                                    ,value_t* from
                                                )noexcept
requires std::is_nothrow_move_constructible_v<value_t>
```

Come `uninitialized_move_n` con in più che gli elementi di `from` vengono distrutti

```
[[nodiscard]] constexpr error_handler_t uninitialized_fill(
                                                            value_t* to
                                                            , size_t n
                                                            )noexcept
requires(std::is_default_constructible_v<value_t>)
```

Inizializza con il valore predefinito di `value_t n` elementi di `to` di memoria non inizializzata
L' operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t uninitialized_fill(
                                                    value_t* to
                                                    , size_t n
                                                    , value_t&& from
                                                )noexcept
requires (
        concepts::copyable_or_cloneable<value_t> &&
        std::is_nothrow_move_constructible_v<value_t>
)
```

Inizializza con il valore `rvalue` di `from n` elementi di `to` di memoria non inizializzata
L' operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t uninitialized_copy_n(
                                                    value_t* to
                                                    , size_t n
                                                    , const value_t* from
                                                    )noexcept
requires (
               concepts::copyable_or_cloneable<value_t>&&
               std::is_nothrow_move_constructible_v<value_t>
)
```

Costruisce copiando `n` valori di `from` in `to` di memoria non inizializzata
L' operazione è transazionale

```
template <typename... ARGS>
[[nodiscard]] constexpr error_handler_t construct_at(
                                                value_t* p
                                                , ARGS&&... args
                                                )noexcept
requires (concepts::constructible_at<value_t, ARGS&&...>)
```

Costruisce `p` da `args`

```
constexpr void destroy_n(value_t* p, size_t n)noexcept
```

Distrugge, ma non dealloca, `n` elementi di `p`
`p` non deve essere nullo se `n > 0`

```
constexpr void destroy_at(value_t* p)noexcept
```

Distrugge,ma non dealloca,`*p`
`p` non deve essere nullo.

```
constexpr void swap_n(value_t* v, size_t n,value_t* u)noexcept
requires (std::is_nothrow_swappable_v<value_t>)
```

Scambia `n` elementi di `v` con `u`

```
[[nodiscard]] constexpr error_handler_t assign_fill(
                                            value_t* d
                                            , size_t n
                                            ,value_t&& v
                                        )noexcept
requires(
        std::is_nothrow_move_assignable_v<value_t> &&
        concepts::copyable_or_cloneable<value_t>
)
```

Assegna `n` elementi di `d` con `rvalue v`
L' operazione non è transazionale

```
[[nodiscard]] constexpr error_handler_t assign_n(
                                            value_t* d
                                            ,size_t n
                                            ,const value_t* s
                                        )noexcept
requires(
        std::is_nothrow_move_assignable_v<value_t> &&
        concepts::copyable_or_cloneable<value_t>
)
```

Assegna `n` elementi di `d` con `s`
L' operazione non è transazionale

```
template <typename U>
[[nodiscard]] constexpr error_handler_t assign(
                                            value_t& v
                                            , U&& e
                                        )noexcept
requires(
        concepts::constructible_at<value_t, U&&> &&
        std::is_nothrow_move_assignable_v<std::decay_t<U>>
)
```

Converte `e` in `value_t` e lo assegna a `v`
L' operazione è transazionale se la costruzione di `value_t` da `e` è transazionale

Costruttori:

```
raw_storage()=delete
```

raw_wrapper

Uso:

mappa un tipo su memoria non inizializzata

Testata:

```
iron/unsafe/raw_wrapper.h
```

Sinossi:

```
template <typename T> class raw_wrapper final
```

Alias interni:

```
using value_t = T
```

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
std::is_nothrow_destructible_v<value_t>
```

Costruttori:

```
constexpr raw_wrapper()noexcept
```

Costruisce un oggetto `raw_wrapper`, l'oggetto interno rimane memoria grezza

```
raw_wrapper(const raw_wrapper&) = delete
```

```
constexpr raw_wrapper(raw_wrapper&& v)noexcept
requires(std::is_nothrow_move_constructible_v<value_t>)
```

Costruisce un oggetto `raw_wrapper` da `v rvalue`
L'oggetto interno di `v` deve essere già costruito

Distruttore:

```
constexpr ~raw_wrapper()
```

L' oggetto interno deve già essere stato distrutto
Usare i metodi `destroy()` o `release_and_destroy()` prima

Metodi:

```
raw_wrapper& operator=(const raw_wrapper&) = delete
```

```
constexpr raw_wrapper& operator=(raw_wrapper&& v)noexcept
```

I due oggetti interni di `*this` e di `v` devono essere già costruiti

```
#ifndef NDEBUG
        [[nodiscard]] constexpr bool constructed()const noexcept
#endif
```

Ritorna vero se l'oggetto interno è costruito

```
constexpr void destroy()noexcept
```

Distrugge l'oggetto interno,esso deve essere già costruito

```
[[nodiscard]] constexpr value_t release()noexcept
requires(std::is_nothrow_move_constructible_v<value_t>)
```

Rilascia l’oggetto interno,esso deve essere già costruito
Dopo il rilascio l’oggetto interno non viene distrutto

```
[[nodiscard]] constexpr value_t release_and_destroy()noexcept
requires(std::is_nothrow_move_constructible_v<value_t>)
```

Rilascia l’oggetto interno,esso deve essere già costruito
Dopo il rilascio l’oggetto interno viene distrutto

```
[[nodiscard]] constexpr const value_t& value()const noexcept
[[nodiscard]] constexpr const value_t& cvalue()const noexcept
[[nodiscard]] constexpr value_t& value()noexcept
```

Ritorna una referenza dell’ oggetto interno,esso deve essere già costruito

```
template <typename... ARGS>
constexpr void construct(ARGS&&... args)noexcept
requires (concepts::nothrow_constructible_at<value_t, ARGS&&...>)
```

Costruisce l’oggetto interno,esso non deve essere già costruito

unicode_conversions

Uso:

manipolazione unicode

Testata:

`iron/unsafe/unicode_conversions.h`

Sinossi:

```
class unicode_conversions final
```

Alias interni:

```
using u8_t = std::uint8_t
using u16_t = std::uint16_t
using u32_t = std::uint32_t
using size_t = std::size_t
```

Costanti:

```
constexpr char32_t default_replacement_chr = 0xFFFD
```

Il carattere predefinito per rimpiazzare un `utf32` non valido

```
constexpr u32_t max=0x0010fffe
```

Il valore massimo di un `utf32`

Metodi statici:

```
template <typename T>
[[nodiscard]] constexpr bool is_basic_charset(T v)noexcept
requires (concepts::charset<T>)
```

Ritorna vero se `v` contiene un carattere del `basic charset`

```
template <typename T>
[[nodiscard]] constexpr bool is_utf32(T v)noexcept
requires(concepts::utf32_charset<T>)
```

Ritorna vero se `v` contiene un `utf32` valido

```
template <typename T>
[[nodiscard]] constexpr size_t len(const T* p)noexcept
requires (concepts::charset<T>)
```

Calcola la lunghezza di una stringa usando `\0` come terminatore

```
template <typename T>
[[nodiscard]] constexpr size_t size_utf8(T v)noexcept
requires (
        concepts::basic_charset<T>
        || concepts::utf8_charset<T>
        || concepts::utf32_charset<T>
)
```

Ritorna il numero di `char8_t` occupati da `v` nella codifica `utf8`
Il valore ritornato è nell' intervallo `0..4` con zero per indicare il valore di `v` non valido

```
template <typename T>
[[nodiscard]] constexpr size_t size_utf16(T v)noexcept
requires(concepts::utf16_charset<T> || concepts::utf32_charset<T>)
```

Ritorna il numero di `char16_t` occupati in `utf16`
Il valore ritornato è nell' intervallo `0..2` con zero per indicare il valore di `v` non valido

```
template <typename U8,typename U32, size_t N>
[[nodiscard]] constexpr size_t utf32_to_utf8(U8(&buffer)[N],U32 u)noexcept
requires (
        concepts::utf8_charset<U8>
        && concepts::utf32_charset<U32>
        && N > 4
        )
```

Converte `u` in `utf8` con terminarore `\0`, ritorna il numero di caratteri `utf8` memorizzati in `buffer`
Il valore ritornato è nell' intervallo `0..4` con zero per indicare il valore di `u` non valido

```
template <typename U16,typename U32, size_t N>
[[nodiscard]] constexpr size_t utf32_to_utf16(U16(&buffer)[N],U32 u)noexcept
requires (
        concepts::utf16_charset<U16>
        && concepts::utf32_charset<U32>
        && N > 2
        )
```

Converte `u` in `utf16` con terminarore `\0`, ritorna il numero di caratteri `utf16` memorizzati in `buffer`
Il valore ritornato è nell' intervallo `0..2` con zero per indicare il valore di `u` non valido

```
template <typename U32,typename ITER>
[[nodiscard]] constexpr size_t utf8_to_utf32(
                                            U32& cp
                                            ,ITER& b
                                            ,const ITER& e
                                            )noexcept
requires (
        concepts::utf32_charset<U32>
        && concepts::charset_forward_iterator<ITER>
        && concepts::utf8_charset<decltype(*b)>
        )
```

Converte la sequenza `utf8 [b,e[` in `utf32`
`b` viene fermato oltre alla fine logica della sequenza
Il valore ritornato è uno per valore valido o zero per valore non valido

```
template <typename U32,typename ITER>
[[nodiscard]] constexpr size_t utf16_to_utf32(
                                            U32& c
                                            ,ITER& b
                                            ,const ITER& e
                                            )noexcept
requires (
                concepts::utf32_charset<U32>
                && concepts::charset_forward_iterator<ITER>
                && concepts::utf16_charset<decltype(*b)>
        )
```

Converte la sequenza `utf16 [b,e[` in `utf32`.
`b` viene fermato oltre la fine logica della sequenza
Il valore ritornato è uno per valore valido o zero per valore non valido

```
template <typename U1,typename U2>
[[nodiscard]] constexpr size_t utf32_to_utf32(U1& d, U2 s)noexcept
requires (
               concepts::utf32_charset<U1>
               && concepts::utf32_charset<U2>
       )
```

Copia `s` in `d`
In caso `s` contenga un `utf32` non valido ritorna zero ma la copia è avvenuta ugualmente altrimenti ritorna uno

```
template <bool CHECK, typename T, typename ITER>
[[nodiscard]] constexpr bool copy(
                                     T* d
                                     , size_t& size
                                     , ITER& b
                                     , const ITER& e
                              )noexcept
requires(
       concepts::charset<T> &&
       concepts::charset_forward_iterator<ITER>
)
```

Copia la sequenza `[b,e[` in `d` eventualmente effettuando una conversione.
`size` contiene il numero di `T` dell' `array d`.
Se `CHECK` è impostato vengono fatti tutti i controlli per verificare la validità del contenuto della sequenza.
In `size` viene ritornato il valore di `T` necessari per la copia e nel caso `size` contenga in ingresso un valore insufficiente per la copia i caratteri in più non vengono copiati.
In caso di valori non validi con `CHECK` impostato viene ritornato falso altrimenti vero

Costruttori:

```
unicode_conversions()=delete
```

unicode

Uso:

Mappatura dei caratteri `utf32` e sua classificazione
Usare questa classe solo per codice di basso livello, per avere le stesse informazioni si puo usare la classe `chr`.

Testata:

```
iron/unsafe/unicode.h
```

Sinossi:

```
class unicode final
```

Alias interni:

```
using u32_t = std::uint32_t
using u8_t = std::uint8_t
typedef enum class attribute : u32_t {
        numeric
        , alphabetic
        , lowercase
        , uppercase
        , whitespace
        , alphanumeric
        , control
        , ascii
        , ascii_digit
        , ascii_hexdigit
        , ascii_punctuation
        , ascii_graphic
        , not_unicode
        , first = numeric
        , last = not_unicode
} attr_t;
using version_t = std::tuple<u8_t, u8_t, u8_t>
using iter_t = unsafe::iterators::contiguous_iterator<const u32_t>
using seq_t = std::pair<iter_t, iter_t>
```

Metodi statici:

```
[[nodiscard]] constexpr u32_t min()noexcept
```

Ritorna il valore minimo valido di un `utf32` – il valore è zero

```
[[nodiscard]] static constexpr u32_t max()noexcept
```

Ritorna il valore massimo valido di un `utf32` – il valore è `0x0010fffe`

```
[[nodiscard]] constexpr version_t unicode_version()noexcept
```

Ritorna la versione di `unicode`, attualmente è 14.0.0.

```
[[nodiscard]] bool is(u32_t v, attr_t a)noexcept
```

Verifica se il valore di `v` appartiene ad `a`

```
[[nodiscard]] constexpr bool is_ascii(u32_t v)noexcept
```

Verifica se `v` appartiene alla codifica `ASCII`

```
[[nodiscard]] constexpr bool is_ascii_digit(u32_t v)noexcept
```

Verifica se `v` è una cifra nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_hexdigit(u32_t v)noexcept`

Verifica se `v` è una cifra esadecimale nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_punctuation(u32_t v)noexcept`

Verifica se `v` è un carattere di punteggiatura nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_graphic(u32_t v)noexcept`

Verifica se `v` è un carattere che ha una rappresentazione grafica
nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_whitespace(u32_t v)noexcept`

Verifica se `v` è un carattere di spaziatura nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_lowercase(u32_t v)noexcept`

Verifica se `v` è una lettera minuscola nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_uppercase(u32_t v)noexcept`

Verifica se `v` è una lettera maiuscola nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_alphabetic(u32_t v)noexcept`

Verifica se `v` è una lettera maiuscola o minuscola nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_alphanumeric(u32_t v)noexcept`

Verifica se `v` è una lettera maiuscola o minuscola o una cifra nella codifica `ASCII`

`[[nodiscard]] constexpr bool is_ascii_control(u32_t v)noexcept`

Verifica se `v` è un carattere di controllo nella codifica `ASCII`

`[[nodiscard]] seq_t uppers(u32_t v)noexcept`

Ritorna una sequenza di lettere maiuscole corrispondenti a `v` in `utf32`
La sequenza può essere vuota se `v` non ha corrispondenze maiuscole

`[[nodiscard]] seq_t lowers(u32_t v)noexcept`

Ritorna una sequenza di lettere minuscole corrispondenti a `v` in `utf32`
La sequenza può essere vuota se `v` non ha corrispondenze minuscole

Costruttori:

`unicode()=delete`

Classi sicure.

adapter_box

Uso:

una classe che serve per scambiare dati in modo efficiente tra contenitori

Testata:

```
iron/adapter_box.h
```

Sinossi:

```
template <typename CONTAINER> class adapter_box final
```

Alias interni:

```
using container_t = CONTAINER
using container_type = container_t
using value_t = typename container_t::value_t
using value_type = value_t
using allocator_t = typename container_t::allocator_t
using allocator_type = allocator_t
using error_handler_t = typename container_t::error_handler_t
using error_handler_type  = error_handler_t
```

Vincoli:

```
concepts::raw_vector<container_t>
```

Per ora `adapter_box` accetta solo un `raw_vector`

Costruttori:

```
[[nodiscard]] static constexpr adapter_box unchecked_from(container_t&& v)noexcept
```

Costruisce da `rvalue v`
La costruzione non è sicura perchè `container_t` non è sicuro

```
constexpr adapter_box()noexcept=default
adapter_box(const adapter_box&)=delete
constexpr adapter_box(adapter_box&&)noexcept
```

Distruttore:

```
constexpr ~adapter_box()=default
```

Metodi:

```
adapter_box& operator=(const adapter_box&)=delete
constexpr adapter_box& operator=(adapter_box&&)noexcept

constexpr adapter_box& swap(adapter_box& _v)noexcept
requires(std::is_nothrow_swappable_v<container_t>)
```

```
[[nodiscard]] constexpr size_t capacity()const noexcept
requires(concepts::raw_vector<container_t>)
```

Ritorna il numero di elementi preallocati

```
[[nodiscard]] constexpr size_t size()const noexcept
```

Ritorna il numero di elementi effettivi

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se non ci sono elementi effettivi

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se ci sono elementi effettivi

```
[[nodiscard]] constexpr container_t unsafe_release()noexcept
requires(std::is_nothrow_move_constructible_v<container_t>)
```

Rilascia l'istanza interna.
L'operazione non è sicura in quanto `container_t` non è sicuro
Dopo questa operazione `empty()` ritorna vero.

```
[[nodiscard]] constexpr result<adapter_box, error_handler_t>
try_clone()const noexcept
requires(concepts::cloneable<container_t>)
```

```
[[nodiscard]] constexpr adapter_box clone()const noexcept
requires(concepts::cloneable<container_t>)
```

Operazione di copia

```
[[nodiscard]] constexpr allocator_t unsafe_allocator()const noexcept
```

Ritorna l'allocatore
L'operazione non è sicura in quanto `allocator_t` non è sicuro

array

Uso:

astrazione di `N` elementi contigui, con `N` immutabile

Testata:

```
iron/array.h
```

Sinossi:

```
template <typename T, std::size_t N> class array final
```

Alias interni:

```
using value_t = T
using value_type = value_t
using size_t = std::size_t
using size_type = size_t
using diff_t = std::ptrdiff_t
using difference_type = diff_t
using ptr_t = value_t*
using pointer = ptr_t
using reference = value_t&
using ref_t = ref_wrapper<value_t>
using cref_t = ref_wrapper<const value_t>
using iterator = <private definition>
using const_iterator = <private definition>
using reverse_iterator = <private definition>
using const_reverse_iterator = <private definition>
using seq_t = <private definition>
using seq_type = seq_t
using cseq_t = <private definition>
using cseq_type = cseq_t
using rseq_t = <private definition>
using rseq_type = rseq_t
using crseq_t = <private definition>
using crseq_type = crseq_t
using error_handler_t = ierror_handler
using error_handler_type = error_handler_t
using adapter_box_t = <private definition>
```

Vincoli:

```
N <= max_size()
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
std::is_nothrow_move_constructible_v<value_t>
std::is_nothrow_swappable_v<value_t>
std::is_nothrow_destructible_v<value_t>
std::is_nothrow_default_constructible_v<value_t>
```

Metodi statici:

```
[[nodiscard]] constexpr size_t max_size()noexcept
```

Ritorna il massimo valore di `N` per allocare elementi consecutivi di tipo `value_t`

```
[[nodiscard]] consteval size_t size()noexcept
```

Ritorna `N`

```
[[nodiscard]] consteval bool empty()noexcept
```

Ritorna vero se `N` è zero

```
[[nodiscard]] consteval explicit operator bool()const noexcept
```

Ritorna vero se `N` non è zero

Costruttori:

```
[[nodiscard]] static constexpr array unchecked_from(value_t(&&)[N])noexcept
requires(N > 0)
```

Costruisce da un `rvalue builtin array`
Il metodo non è sicuro perché un `builtin array` non è sicuro

```
template <typename A>
[[nodiscard]] static constexpr result<array, error_handler_t>
try_from(adapter_box_t<A>&& box)noexcept
requires(concepts::allocator<A>)
```

```
template <typename A>
[[nodiscard]] static constexpr array from(adapter_box_t<A>&& box)noexcept
requires(concepts::allocator<A>)
```

Costruisce da un `rvalue adapter_box_t`
Il numero di elementi effettivi in `box` deve corrispondere a `N`

```
constexpr array()noexcept
array(const array&)=delete
constexpr array(array&&)noexcept
```

Distruttore:

```
constexpr ~array()=default
```

Metodi:

```
array& operator=(const array&) = delete
constexpr array& operator=(array&&)noexcept
constexpr void swap(array&)noexcept
```

```
template <size_t M=1>
[[nodiscard]] constexpr result<array<value_t,M * N>, error_handler_t>
try_clone()const noexcept
requires (
        (N == 0 || max_size() / N >= M)
        && concepts::copyable_or_cloneable<value_t>
)
```

```
template <size_t M = 1>
[[nodiscard]] constexpr array<value_t, M * N> clone()const noexcept
requires (
        (N == 0 || max_size() / N >= M)
        && concepts::copyable_or_cloneable<value_t>
)
```

Costruisce una copia moltiplicata per `M`

```
[[nodiscard]] constexpr option<ref_t> nth(size_t idx)noexcept
[[nodiscard]] constexpr option<cref_t> nth(size_t idx)const noexcept
[[nodiscard]] constexpr option<cref_t> cnth(size_t idx)const noexcept
```

Ritorna l’elemento all’indice `idx`

```
[[nodiscard]] constexpr ref_t operator[](size_t idx)noexcept
requires(_N > 0)

[[nodiscard]] constexpr ref_t at(size_t idx)noexcept
requires(_N > 0)

[[nodiscard]] constexpr cref_t operator[](size_t idx)const noexcept
requires(_N > 0)

[[nodiscard]] constexpr cref_t at(size_t idx)const noexcept
requires(_N > 0)
```

Ritorna l'elemento alla posizione `idx`, il primo elemento ha posizione zero
In caso di indice fuori dai limiti viene chiamata `PANIC()`

```
[[nodiscard]] constexpr value_t& unchecked_nth(size_t idx)noexcept
requires(N > 0)

[[nodiscard]] constexpr const value_t& unchecked_nth(size_t idx)const noexcept
requires(N > 0)

[[nodiscard]] constexpr const value_t& unchecked_cnth(size_t idx)const noexcept
requires(N > 0)
```

Ritorna l'elemento all'indice `idx`
Il metodo non è sicuro perché il valore di `idx` potrebbe non essere valido e perché ritorna una referenza

```
[[nodiscard]] constexpr ref_t front()noexcept
requires(N > 0)

[[nodiscard]] constexpr cref_t front()const noexcept
requires(N > 0)

[[nodiscard]] constexpr cref_t cfront()const noexcept
requires(N > 0)
```

Ritorna l'elemento all'indice zero

```
[[nodiscard]] constexpr value_t& unchecked_front()noexcept
requires(N > 0)

[[nodiscard]] constexpr const value_t& unchecked_front()const noexcept
requires(N > 0)

[[nodiscard]] constexpr const value_t& unchecked_cfront()const noexcept
requires(N > 0)
```

Ritorna l'elemento all'indice zero
Il metodo non è sicuro perché ritorna una referenza

```
[[nodiscard]] constexpr ref_t back()noexcept
requires(N > 0)

[[nodiscard]] constexpr cref_t back()const noexcept
requires(N > 0)

[[nodiscard]] constexpr cref_t cback()const noexcept
requires(N > 0)
```

Ritorna l'ultimo elemento

```
[[nodiscard]] constexpr value_t& unchecked_back()noexcept
requires(N > 0)
```

```
[[nodiscard]] constexpr const value_t& unchecked_back()const noexcept
requires(N > 0)
```

```
[[nodiscard]] constexpr const value_t& unchecked_cback()const noexcept
requires(N > 0)
```

Ritorna l'ultimo elemento
Il metodo non è sicuro perché ritorna una referenza

```
[[nodiscard]] constexpr error_handler_t try_fill(value_t&& v)noexcept
requires(concepts::copyable_or_cloneable<value_t>)
```

```
constexpr array& fill(value_t&& v)noexcept
requires(concepts::copyable_or_cloneable<value_t>)
```

Riempie con il valore `rvalue v`
Non ci devono essere referenti attivi
L'operazione non è transazionale

```
constexpr array& fill(const value_t& v)noexcept
requires(std::is_nothrow_copy_constructible_v<value_t>)
```

Riempie con il valore di `v`
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr seq_t seq()noexcept
[[nodiscard]] constexpr cseq_t cseq()const noexcept
```

Ritorna una sequenza sugli elementi

```
[[nodiscard]] constexpr rseq_t rseq()noexcept
[[nodiscard]] constexpr crseq_t crseq()const noexcept
```

Ritorna una sequenza inversa sugli elementi

```
template <typename A=unsafe::allocator<value_t>>
[[nodiscard]] constexpr result<adapter_box_t<A>, error_handler_t>
try_release()noexcept
requires(concepts::allocator<A>)
```

```
template <typename A=unsafe::allocator<value_t>>
[[nodiscard]] constexpr adapter_box_t<A> release()noexcept
requires(concepts::allocator<A>)
```

Esporta gli elementi
Non ci devono essere referenti attivi
Dopo l'esportazione gli elementi del contenitore vengono impostati al valore predefinito
L'operazione è transazionale

```
constexpr array& clear()noexcept
```

Imposta gli elementi al valore predefinito
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr bool operator==(const array<value_t,N>&)const noexcept
requires(concepts::nothrow_equality_comparable<value_t>)
```

Ritorna vero per uguaglianza elemento per elemento

bitset

Uso:

Un contenitore di `N` bit con `N` immutabile

Testata:

`iron/bitset.h`

Sinossi:

```
template <std::size_t N> class bit_set final
```

Alias interni:

```
using size_t=std::size_t
```

Vincoli:

```
N > 0
```

Metodi statici:

```
constexpr size_t size()noexcept
```

Ritorna `N`

```
constexpr size_t capacity()noexcept
```

Ritorna il numero di byte occupati internamente

Costruttori:

```
constexpr bit_set()noexcept
constexpr bit_set(const bit_set&)noexcept
constexpr bit_set(bit_set&&)noexcept
```

Costruttori esterni:

```
template <std::size_t SZ, typename... ARGS>
[[nodiscard]] constexpr bit_set<SZ> make_bit_set(const ARGS&… args)noexcept
requires (SZ > 0 && (concepts::nothrow_convertible_to_integral<ARGS>&& ...))
```

Ritorna un `bit_set` con i valori impostati a vero per gli indici `args`

Distruttore:

```
constexpr ~bit_set() = default
```

Metodi:

```
constexpr bit_set& operator=(const bit_set&)noexcept
constexpr bit_set& operator=(bit_set&&)noexcept
constexpr bit_set& swap(bit_set&)noexcept
```

```
constexpr bit_set& set(size_t n)noexcept
```

Imposta a vero il bit a indice `n`, il primo bit ha indice zero
Se `n >= N` viene chiamata `PANIC()`

`constexpr bit_set& reset(size_t n)noexcept`

Imposta a falso il bit a indice `n`, il primo bit ha indice zero
Se `n >= N` viene chiamata `PANIC()`

`constexpr bit_set& set_all()noexcept`

Imposta a vero tutti i bit

`constexpr bit_set& reset_all()noexcept`

Imposta a falso tutti i bit

`[[nodiscard]] constexpr bool test(size_t n)const noexcept`

Ritorna vero se il bit a indice `n` è impostato
Se `n >= N` viene chiamata `PANIC()`

`[[nodiscard]] constexpr bool test_or(std::initializer_list<size_t> v)const noexcept`

Se almeno l'indice di un bit in `v` è impostato ritorna vero
Se viene incontrato un indice di bit `>= N` viene chiamata `PANIC()`

`[[nodiscard]] constexpr bool test_and(std::initializer_list<size_t> v)const noexcept`

Se almeno l'indice di un bit in `v` non è impostato ritorna falso
Se viene incontrato un indice di bit `>= N` viene chiamata `PANIC()`

`[[nodiscard]] constexpr bool all()const noexcept`

Ritorna vero se tutti i bit sono impostati

`[[nodiscard]] constexpr bool none()const noexcept`

Ritorna vero se tutti i bit non sono impostati

`[[nodiscard]] constexpr explicit operator bool()const noexcept`

Ritorna vero se almeno un bit è impostato

box

Uso:

Mappare un tipo nella memoria dinamica

Testata:

`iron/box.h`

Sinossi:

```
template <typename T,typename D> class base_box final
```

Alias esterni:

```
template <typename T, typename D = box_deleter<T,unsafe::allocator<T>>>
using box = base_box<T, D>
```

Alias interni:

```
using value_t=T
using value_type = value_t
using deleter_t = D
using deleter_type = deleter_t
using ptr_t = value_t*const
using pointer = ptr_t
using cptr_t = const value_t* const
using const_pointer = cptr_t
using size_t = std::size_t
using size_type = size_t
using ref_t = typename std::add_lvalue_reference<value_t>::type
using reference = ref_t
using cref_t = typename std::add_lvalue_reference<const value_t>::type
using const_reference = cref_t
using error_handler_t = error_handler
using error_handler_type = error_handler_t
template <typename T1>
using rebind_t = base_box<T1,typename deleter_t:: template rebind_t<T1>>
template <typename T1>
using rebind_type=rebind_t<T1>
```

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
!std::is_same_v<value_t*,void*>
concepts::box_deleter<deleter_t>
std::is_same_v<value_t, typename deleter_t::value_t>
```

Costruttori:

```
[[nodiscard]] static constexpr base_box unchecked_from(value_t* p)noexcept
```

Costruisce da `p`
Non è sicuro perche un puntatore non è sicuro

```
template <typename... ARGS >
[[nodiscard]] static constexpr result<base_box, error_handler_t>
try_from(ARGS&&... args)noexcept
requires (concepts::constructible_at<value_t,ARGS&&...>)
```

```
template <typename... ARGS >
[[nodiscard]] static constexpr base_box from(ARGS&&... args)noexcept
requires (concepts::constructible_at<value_t, ARGS&&...>)
```

Costruisce da `args`

```
constexpr base_box()noexcept = default
constexpr explicit base_box(std::nullptr_t)noexcept
base_box(const base_box&) = delete
constexpr base_box(base_box&&)noexcept
```

Costruttori esterni:

```
template <typename T>
[[nodiscard]] constexpr box<T> make_empty_box()noexcept
```

```
template <typename T, typename... ARGS>
[[nodiscard]] constexpr result<box<T>,error_handler>
try_make_box(ARGS&&...)noexcept
```

```
template <typename T, typename... ARGS>
[[nodiscard]] constexpr box<T> make_box(ARGS&&...)noexcept
```

Distruttore:

```
constexpr ~base_box()
requires(sizeof(value_t) > 0 && std::is_nothrow_destructible_v<value_t>)
```

Metodi:

```
base_box& operator=(const base_box&) = delete
constexpr base_box& operator=(base_box&&)noexcept
constexpr base_box& swap(base_box&)noexcept
```

```
constexpr base_box& operator=(std::nullptr_t)noexcept
requires(sizeof(value_t) > 0 && std::is_nothrow_destructible_v<value_t>)
```

```
constexpr base_box& reset()noexcept
requires(sizeof(value_t) > 0 && std::is_nothrow_destructible_v<value_t>)
```

Distrugge l’oggetto interno
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se il puntatore interno è nullo

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se il puntatore interno non è nullo

```
[[nodiscard]] constexpr ptr_t unsafe_release()noexcept
```

Rilascia il puntatore dell’oggetto interno
Il metodo non è sicuro perche un puntatore non è sicuro
Dopo questa operazione `empty()` ritorna vero

```
[[nodiscard]] constexpr cptr_t unsafe_data()const noexcept
[[nodiscard]] constexpr cptr_t unsafe_cdata()const noexcept
[[nodiscard]] constexpr ptr_t unsafe_data()noexcept
```

Ritorna il puntatore dell’oggetto interno
Il metodo non è sicuro perchè un puntatore non è sicuro

```
[[nodiscard]] constexpr option<ref_wrapper<value_t>> value()noexcept

[[nodiscard]] constexpr option<ref_wrapper<const value_t>> value()const noexcept

[[nodiscard]] constexpr option<ref_wrapper<const value_t>> cvalue()const noexcept
```

Ritorna il valore dell'istanza interna

```
[[nodiscard]] constexpr ref_wrapper<value_t> operator*()noexcept

[[nodiscard]] constexpr ref_wrapper<const value_t> operator*()const noexcept
```

Ritorna il valore dell'istanza interna
Se non è impostata viene chiamata `PANIC()`

```
[[nodiscard]] constexpr value_t* operator->()noexcept
requires(sizeof(value_t) > 0 && std::is_class_v<value_t>)

[[nodiscard]] constexpr const value_t* operator->()const noexcept
requires(sizeof(value_t) > 0 && std::is_class_v<value_t>)
```

Ritorna il puntatore all'istanza interna
Se non è impostata viene chiamata `PANIC()`
Da usare solo con `value_t` classe o struttura per accedere ad un valore od un metodo

```
[[nodiscard]] constexpr ref_t unchecked_value()noexcept

[[nodiscard]] constexpr cref_t unchecked_value()const noexcept

[[nodiscard]] constexpr cref_t unchecked_cvalue()const noexcept
```

Ritorna un riferimento al valore dell'istanza interna
L'operazione non è sicura perché il contenitore potrebbe essere vuoto e perché ritorna una referenza

```
template <typename U>
[[nodiscard]] constexpr auto upcast()noexcept
requires (
               sizeof(value_t) > 0
               && (std::is_same_v<U,value_t> || std::is_base_of_v<U, value_t>)
               && std::has_virtual_destructor_v<U>
        )
```

Ritorna una istanza con tipo `U`, una classe base o uguale a `value_t`
Dopo questa operazione `empty()` ritorna vero.

```
template <typename U>
[[nodiscard]] constexpr auto downcast()noexcept
requires (
               sizeof(value_t) > 0
               && (std::is_same_v<U, value_t> || std::is_base_of_v<value_t,U>)
               && std::has_virtual_destructor_v<value_t>
        )
```

Ritorna una istanza con tipo `U`, una classe derivata o uguale a `value_t`
Dopo questa operazione l'istanza corrente sarà vuota se ha avuto
successo altrimenti viene ritornata una istanza vuota e l'istanza corrente rimane invariata.

```
template <typename U>
[[nodiscard]] constexpr auto cast()noexcept
requires (
        sizeof(value_t) > 0
        && (
                std::is_same_v<U, value_t>
                || std::is_base_of_v<value_t, U>
                || std::is_base_of_v<U, value_t>
        )
        && std::has_virtual_destructor_v<value_t>
)
```

Ritorna una istanza con tipo `U`, una classe base,derivata o uguale a `value_t`
Dopo questa operazione l'istanza corrente sarà vuota se ha avuto successo altrimenti viene ritornata una istanza vuota e l'istanza corrente rimane invariata.

```
[[nodiscard]] constexpr result<value_t,error_handler_t>
try_release_value()noexcept
requires (
                sizeof(value_t) > 0
                && std::is_nothrow_move_constructible_v<value_t>
                && std::is_nothrow_destructible_v<value_t>
        )

[[nodiscard]] constexpr value_t release_value()noexcept
requires (
                sizeof(value_t) > 0
                && std::is_nothrow_move_constructible_v<value_t>
                && std::is_nothrow_destructible_v<value_t>
        )
```

Rilascia il valore interno,non ci devono essere referenti attivi
Dopo questa operazione `empty()` ritorna vero
Se l'istanza corrente è vuota viene impostato l'errore `POSIX result_out_of_range`

```
[[nodiscard]] constexpr value_t release_value_or_default()noexcept
requires (
                sizeof(value_t) > 0
                && std::is_nothrow_move_constructible_v<value_t>
                && std::is_nothrow_destructible_v<value_t>
                && std::is_nothrow_default_constructible_v<value_t>
)
```

Rilascia il valore interno se è impostato altrimenti ritorna il valore preimpostato di `value_t`.
Non ci devono essere referenti attivi
Dopo questa operazione `empty()` ritorna vero

```
[[nodiscard]] constexpr result<base_box, error_handler_t> try_clone()const
noexcept
requires (concepts::copyable_or_cloneable<value_t>)

[[nodiscard]] constexpr base_box clone()const noexcept
requires (concepts::copyable_or_cloneable<value_t>)
```

Ritorna una copia.

box_deleter

Uso:

Il deallocatore predefinito per `box` e `shared_box`

Testata:

```
iron/box.h
```

Sinossi:

```
template <typename T,typename A> class box_deleter final
```

Alias interni:

```
using value_t = T
using value_type = value_t
using allocator_t = A
using allocator_type = allocator_t
template <typename T1>
using rebind_t = box_deleter<T1,<private>>
template <typename T1>
using rebind_type = rebind_t<T1>
```

Vincoli:

```
!std::is_array_v<value_t>
!std::is_const_v<value_t>
 concepts::allocator<allocator_t>
```

Costruttori:

```
constexpr box_deleter()noexcept = default
constexpr box_deleter(const box_deleter&)noexcept
constexpr box_deleter(box_deleter&&)noexcept
```

Distruttore:

```
constexpr ~box_deleter()=default
```

Metodi:

```
constexpr box_deleter& operator=(const box_deleter&)noexcept
constexpr box_deleter& operator=(box_deleter&&)noexcept
```

```
[[nodiscard]] constexpr allocator_t unsafe_allocator()const noexcept
```

Ritorna l'allocatore
Il metodo non è sicuro perche `allocator_t` non è sicuro

```
constexpr void operator()(value_t* p)noexcept
requires (sizeof(value_t) > 0 && std::is_nothrow_destructible_v<value_t>)
```

Dealloca p

```
template <typename U>
[[nodiscard]] static rebind_t<U> upcast()noexcept
        requires (
                (std::is_same_v<U,value_t> || std::is_base_of_v<U, value_t>)
                && std::has_virtual_destructor_v<U>
        )


template <typename U>
[[nodiscard]] static rebind_t<U> downcast()noexcept
        requires (
                (std::is_same_v<U,value_t> || std::is_base_of_v<value_t,U>)
                && std::has_virtual_destructor_v<U>
        )
```

Ritorna il `deleter` per `U`, un tipo `upcast` o `downcast` di `value_t`

btree_table

Uso:

Implementazione di un `btree set/map`

Testata:

`iron/btree_table.h`

Sinossi:

```
template <
        typename KEY
        ,typename MAPPED
        ,typename LT
        ,std::size_t ORDER
        ,typename A
        ,bool MULTI
        ,bool HAS_MAPPED
>
class btree_table final
```

Costanti esterne:

```
constexpr std::size_t btree_default_order = 3
```

Alias esterni:

```
template <
        typename KEY
        , typename MAPPED
        , typename LT= comparables::less_to<KEY>
        , typename A=unsafe::allocator<std::pair<KEY,MAPPED>>
        , std::size_t ORDER = btree_default_order
>
using btree_map = btree_table<KEY,MAPPED,LT,ORDER,A,false,true>

template <
        typename KEY
        ,typename LT = comparables::less_to<KEY>
        ,typename A =unsafe::allocator<KEY>
        ,std::size_t ORDER = btree_default_order
>
using btree_set = btree_table<KEY,<private>,LT,ORDER,A,false,false>

template <
        typename KEY
        ,typename MAPPED
        ,typename LT = comparables::less_to<KEY>
        ,typename A =unsafe::allocator<std::pair<KEY, MAPPED>>
        ,std::size_t ORDER = btree_default_order
>
using multi_btree_map = btree_table<KEY, MAPPED,LT,ORDER,A,true,true>

template <
        typename KEY
        ,typename LT = comparables::less_to<KEY>
        ,typename A =unsafe::allocator<KEY>
        ,std::size_t ORDER = btree_default_order
>
using multi_btree_set = btree_table<KEY,<private>,LT,ORDER,A,true,false>
```

Alias interni:

```
using key_t = KEY
using key_type = key_t
using mapped_t = MAPPED
using mapped_type = mapped_t
using allocator_t = A
using allocator_type = allocator_t
using key_less_t = LT
using key_less_type = key_less_t
using size_t = std::size_t
```

```
using size_type = size_t
using value_t = <private>
using value_type = value_t
using exported_value_t = <private>
using exported_value_type = exported_value_t
using ptr_t = value_t*
using pointer = ptr_t
using cptr_t = const value_t*
using const_pointer = cptr_t
using diff_t = std::ptrdiff_t
using difference_type = diff_t
using reference = value_t&
using const_reference = const value_t&
using ref_t = ref_wrapper<value_t>
using cref_t = ref_wrapper<const value_t>
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using adapter_box_t = adapter_box<<private>>
using iterator = <private>
using const_iterator = <private>
using reverse_iterator = <private>
using const_reverse_iterator = <private>
using seq_t = sequence<iterator>
using seq_type = seq_t
using cseq_t = sequence<const_iterator>
using cseq_type = cseq_t
using rseq_t = sequence<reverse_iterator>
using rseq_type = rseq_t
using crseq_t = sequence<const_reverse_iterator>
using crseq_type = crseq_t
using tree_node_t = <private>
```

Costanti interne:

```
static constexpr size_t order = ORDER
static constexpr bool is_multi = MULTI
static constexpr bool has_mapped = HAS_MAPPED
```

Metodi statici:

```
[[nodiscard]] static constexpr size_t max_size()noexcept
```

Ritorna il massimo numero di elementi memorizzabile

Vincoli:

```
sizeof(key_t) > 0
order > 1
!std::is_reference_v<key_t>
!std::is_array_v<key_t>
!std::is_const_v<key_t>
std::is_nothrow_destructible_v<key_t>
std::is_nothrow_move_constructible_v<key_t>
std::is_nothrow_swappable_v<key_t>
!has_mapped || !std::is_reference_v<mapped_t>
!has_mapped || !std::is_array_v<mapped_t>
!has_mapped || !std::is_const_v<mapped_t>
!has_mapped || std::is_nothrow_destructible_v<mapped_t>
!has_mapped || std::is_nothrow_move_constructible_v<mapped_t>
!has_mapped || std::is_nothrow_swappable_v<mapped_t>
concepts::allocator<allocator_t>
concepts::comparable<key_less_t,key_t>
```

Costruttori:

```
[[nodiscard]] static constexpr result<btree_table, error_handler_t>
try_from(adapter_box_t&&)noexcept
```

```
[[nodiscard]] static constexpr btree_table from(adapter_box_t&&)noexcept
```

Costruisce da `rvalue adapter_box_t`

```
[[nodiscard]] static constexpr btree_table unchecked_from(allocator_t a)noexcept
```

Costruisce una istanza vuota usando `a`
Il metodo non è sicuro perche `allocator_t` non è sicuro

```
constexpr btree_table()noexcept
btree_table(const btree_table&) = delete
constexpr btree_table(btree_table&&)noexcept
```

Distruttore:

```
constexpr ~btree_table()=default
```

Metodi:

```
btree_table& operator=(const btree_table&) = delete
constexpr btree_table& operator=(btree_table&&)noexcept
constexpr btree_table& swap(btree_table&)noexcept
```

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se il contenitore non ha elementi

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se il contenitore ha elementi

```
[[nodiscard]] constexpr size_t size()const noexcept
```

Ritorna il numero di elementi

```
[[nodiscard]] constexpr result<btree_table, error_handler_t>
try_clone(size_t n=1)const noexcept
requires(
        concepts::copyable_or_cloneable<key_t>
        && concepts::copyable_or_cloneable<mapped_t>
)

[[nodiscard]] constexpr btree_table clone(size_t n=1)const noexcept
requires(
        concepts::copyable_or_cloneable<key_t>
        && concepts::copyable_or_cloneable<mapped_t>
)
```

Costruisce una copia composta da `size() * n` elementi
Se `is_multi` è falso `n` viene impostato a 1 per `n > 1`

```
[[nodiscard]] constexpr allocator_t unsafe_allocator()const noexcept
```

Ritorna l'allocatore
Il metodo non è sicuro perchè `allocator_t` non è sicuro

```
[[nodiscard]] constexpr seq_t seq()noexcept
[[nodiscard]] constexpr cseq_t seq()const noexcept
[[nodiscard]] constexpr cseq_t cseq()const noexcept
```

Ritorna una sequenza di elementi a valori ordinati usando `key_less_t`

```
[[nodiscard]] constexpr rseq_t rseq()noexcept
[[nodiscard]] constexpr crseq_t rseq()const noexcept
[[nodiscard]] constexpr crseq_t crseq()const noexcept
```

Ritorna una sequenza di elementi a valori ordinati all'inverso usando `key_less_t`

```
[[nodiscard]] constexpr seq_t find(const key_t& k)noexcept
[[nodiscard]] constexpr cseq_t find(const key_t& k)const noexcept
[[nodiscard]] constexpr cseq_t cfind(const key_t& k)const noexcept
```

Ritorna una sequenza con il primo valore uguale a `k` usando `key_less_t`
Se `k` non viene trovato ritorna una sequenza vuota

```
template <typename K,typename M>
[[nodiscard]] constexpr result<std::pair<seq_t,bool>, error_handler_t>
try_insert(K&& key,M&& mapped)noexcept
requires (
        has_mapped
        && std::constructible_from<key_t,K>
        && std::constructible_from<mapped_t,M>
)

template <typename K,typename M>
constexpr std::pair<seq_t,bool> insert(K&& key,M&& mapped)noexcept
requires (
        has_mapped
        && std::constructible_from<key_t,K>
        && std::constructible_from<mapped_t,M>
)
```

Inserisce `key + mapped` nel contenitore, non ci devono essere referenti attivi
Ritorna una sequenza il cui primo elemento è uguale a `k` usando `key_less_t`
ed un valore booleano che indica se `key + mapped` è stato inserito
L'operazione è transazionale

```
template <typename K>
[[nodiscard]] constexpr result<std::pair<seq_t, bool>, error_handler_t>
try_insert(K&& key)noexcept
requires (
        !has_mapped
        && std::constructible_from<key_t,K>
)

template <typename K>
constexpr std::pair<seq_t, bool> insert(K&& key)noexcept
requires (
        !has_mapped
        && std::constructible_from<key_t, K>
)
```

Inserisce `key` nel contenitore, non ci devono essere referenti attivi
Ritorna una sequenza il cui primo elemento è uguale a `k` usando `key_less_t`
ed un valore booleano che indica se `key` è stato inserito
L'operazione è transazionale

```
template <typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t> try_insert_range(
                SEQ& seq
)noexcept
        requires(
                has_mapped
                && concepts::sequence<SEQ>
                && !std::is_const_v<SEQ>
                && concepts::std_pair<typename SEQ::value_t>
                && concepts::constructible_at<key_t, std::decay_t<typename
                SEQ::value_t::first_type>&&>
                && concepts::constructible_at<mapped_t, std::decay_t<typename
        SEQ::value_t::second_type>&&>
        )
```

```
template <typename SEQ>
constexpr size_t insert_range(SEQ& seq)noexcept
requires(
        has_mapped
        && concepts::sequence<SEQ>
        && concepts::std_pair<typename SEQ::value_t>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t::first_type>&&>
        && concepts::constructible_at<mapped_t,std::decay_t<typename
        SEQ::value_t::second_type>&&>
        )
```

```
template <typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_insert_range(SEQ& seq)noexcept
        requires(
        !has_mapped
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

```
template <typename SEQ>
constexpr size_t insert_range(SEQ& seq)noexcept
requires(
        !has_mapped
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<key_t,std::decay_t<typename
        SEQ::value_t>&&>
        )
```

Inserisce i valori di una sequenza, non ci devono essere referenti attivi
I valori vengono copiati e viene ritornato il numero di elementi inseriti
`seq` sarà consumato alla fine dell'operazione se non ci sono errori
L'operazione non è transazionale

```
template <typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_transactional_insert_range(
                SEQ& seq
)noexcept
        requires(
                has_mapped
                && concepts::sequence<SEQ>
                && !std::is_const_v<SEQ>
                && concepts::std_pair<typename SEQ::value_t>
                && concepts::constructible_at<key_t, std::decay_t<typename
                SEQ::value_t::first_type>&&>
                && concepts::constructible_at<mapped_t, std::decay_t<typename
        SEQ::value_t::second_type>&&>
        )
```

```
template <typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_transactional_insert_range(SEQ& _seq)noexcept
        requires(
        !has_mapped
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

```
template <typename SEQ>
constexpr size_t transactional_insert_range(SEQ& seq)noexcept
requires(
        has_mapped
        && concepts::sequence<SEQ>
        && concepts::std_pair<typename SEQ::value_t>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t::first_type>&&>
        && concepts::constructible_at<mapped_t,std::decay_t<typename
        SEQ::value_t::second_type>&&>
        )
```

```
template <typename SEQ>
constexpr size_t transactional_insert_range(SEQ& seq)noexcept
        requires(
        !has_mapped
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

Come `insert_range` ma transazionale

```
constexpr btree_table& reset()noexcept
```

Distrugge tutti gli elementi, non ci devono essere referenti attivi
Dopo questa operazione `empty()` ritorna vero

```
template <typename K,typename... ARGS >
constexpr size_t erase(const K& key,const ARGS&... args)noexcept
requires (
        std::is_nothrow_convertible_v<std::decay_t<K>, key_t>
        &&
        (std::is_nothrow_convertible_v<std::decay_t<ARGS>, key_t>& ...)
)
```

Rimuove una lista di chiavi, non ci devono essere referenti attivi
Ritorna il numero di elementi rimossi

```
[[nodiscard]] constexpr result<size_t,error_handler_t>
try_merge(btree_table& oth)noexcept
```

```
constexpr size_t merge(btree_table& oth)noexcept
```

Inserisce elementi da `oth`
E' piu efficiente di un `insert_range` perchè i valori di `oth` vengono spostati e gli elementi rimossi
Non ci devono essere referenti attivi sia nell' istanza corrente che in `oth`
Ritorna il numero di elementi inseriti
L'operazione non è transazionale

```
template <typename... ARGS>
[[nodiscard]] constexpr result<adapter_box_t, error_handler_t>
try_release_list(const ARGS&... keys)noexcept
requires (std::is_nothrow_convertible_v<std::decay_t<ARGS>, key_t>& …)
```

```
template <typename... ARGS>
[[nodiscard]] constexpr adapter_box_t release_list(const ARGS&... keys)noexcept
requires (std::is_nothrow_convertible_v<std::decay_t<ARGS>, key_t>& …)
```

Esporta gli elementi con chiave presente nell' elenco `keys` usando `key_less_t` per uguaglianza
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr result<adapter_box_t, error_handler_t>
try_release_all()noexcept
```

```
[[nodiscard]] constexpr adapter_box_t release_all()noexcept
```

Esporta tutti gli elementi, non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr const tree_node_t* unsafe_data()const noexcept
[[nodiscard]] constexpr const tree_node_t* unsafe_cdata()const noexcept
[[nodiscard]] constexpr tree_node_t* unsafe_data()noexcept
```

Ritorna il puntatore al nodo radice interno
L' operazione non è sicura

```
template <
        typename KEY1
        , typename MAPPED1
        , typename LT1
        , std::size_t ORDER1
        , typename A1
        , bool MULTI1
        , bool HAS_MAPPED1
>
[[nodiscard]] constexpr bool operator==(const btree_table<KEY1,MAPPED1,LT1,ORDER1,
        A1,MULTI1,HAS_MAPPED1>& r)const noexcept
requires(
        has_mapped == HAS_MAPPED1
        && std::is_same_v<key_t,KEY1>
        && std::is_same_v<key_less_t, LT1>
        && (!has_mapped || std::is_same_v<mapped_t,MAPPED1>)
        && (!has_mapped || concepts::nothrow_equality_comparable<mapped_t>)
)
```

Confronta il contenitore con r
Ritorna vero se sono uguali

cfg

Uso:

fornire informazioni sulla configurazione del sistema

Testata:

```
iron/cfg.h
```

Sinossi:

```
class cfg final
```

Alias:

```
typedef struct version {
        using number_t = std::uint32_t;
        number_t major, minor, patch;
} version_t

using size_t=std::size_t

typedef enum class enum_target_arch {
        unknow
        , x86
        , x86_64
        , powerpc
        , powerpc64
        , arm
        , arm64
        , arm64ec
        , aarch64
        ,first=unknow
        ,last= aarch64
} arch_t

typedef enum class enum_feature {
        unknow
        , avx
        , avx2
        , crt_static
        , rdrand
        , sse
        , sse2
        , sse4_1
        , first = unknow
        , last = sse4_1
} feature_t


typedef enum class enum_target_os {
        unknow
        ,windows
        ,macos
        ,ios
        ,linux
        ,android
        ,freebsd
        ,dragonfly
        ,openbsd
        ,netbsd
        , first = unknow
        , last = netbsd
} os_t

typedef enum class enum_endian {
        unknow
        ,big
        ,little
        , first = unknow
        , last = little
} endian_t

typedef enum class enum_family {
        unknow
        ,unix
        ,windows
        , first = unknow
        , last= windows
} family_t
```

```
typedef enum class enum_target_env {
        unknow
        ,gnu
        ,msvc
        ,clang
        ,first = unknow
        ,last = clang
} env_t
```

Costruttori:

```
cfg() = delete
```

Costanti statiche:

```
constexpr std::uint32_t char_bits = <private>
```

Il numero dei bits in un char

Metodi statici:

```
[[nodiscard]] constexpr version_t iron_version()noexcept
```

Ritorna la versione della libreria

```
[[nodiscard]] consteval arch_t arch()noexcept
```

Ritorna l'architettura della cpu

```
[[nodiscard]] consteval feature_t feature()noexcept
```

Ritorna particolari operazioni della cpu
Attualmente ritorna sempre unknow

```
[[nodiscard]] consteval os_t os()noexcept
```

Ritorna il nome del sistema operativo
Attualmente ritorna sempre unknow

```
[[nodiscard]] consteval endian_t endian()noexcept
```

Ritorna la direzione dei bit in un intero

```
[[nodiscard]] consteval family_t family()noexcept
```

Ritorna la famiglia del sistema operativo
Attualmente ritorna sempre unknow

```
[[nodiscard]] consteval env_t env()noexcept
```

Ritorna il tipo di compilatore

```
[[nodiscard]] consteval std::uint32_t pointer_width()noexcept
```

Ritorna la grandezza di un puntatore in bits,0 per valore sconosciuto

```
[[nodiscard]] consteval std::uint32_t cpp_std()noexcept
```

Ritorna la versione std del compilatore,0 per sconosciuto

```
[[nodiscard]] consteval std::uint32_t compiler_version()noexcept
```

Ritorna la versione del compilatore, 0 per sconosciuta

```
[[nodiscard]] const char* vendor()noexcept
```

Ritorna il nome del venditore del compilatore,stringa vuota in caso sia sconosciuto

```
[[nodiscard]] const char* str(arch_t _v)noexcept
```

Converte in stringa arch_t

```
[[nodiscard]] const char* str(feature_t _v)noexcept
```

Converte in stringa feature_t

```
[[nodiscard]] const char* str(os_t _v)noexcept
```

Converte in stringa os_t

```
[[nodiscard]] const char* str(endian_t _v)noexcept
```

Converte in stringa endian_t

```
[[nodiscard]] const char* str(family_t _v)noexcept
```

Converte in stringa family_t

```
[[nodiscard]] const char* str(env_t _v)noexcept
```

Converte in stringa env_t

```
[[nodiscard]] static consteval size_t hash_prime()noexcept
[[nodiscard]] static consteval size_t hash_prime_start()noexcept
```

Queste funzioni servono per la distribuzione di valori per una funzione hash
Ritorna 0 per architettura sconosciuta

chr

Uso:

Rappresentazione di un `UTF32`

Testata:

`iron/chr.h`

Sinossi:

```
class chr final
```

Alias:

```
using u8_t = std::uint8_t
using u32_t = std::uint32_t
using size_t = std::size_t
using radix_t = u32_t
using unicode_version_t = unsafe::unicode::version_t
class sequence
using seq_t=sequence
```

Costanti:

```
constexpr char32_t max_chr = 0x0010fffe
```

Il massimo valore di un `UTF32`

```
constexpr char32_t default_replacement_chr = 0x00FFFD
```

Il valore di rimpiazzo per un `UTF32` non valido

```
constexpr radix_t default_radix = 10
```

La base numerica predefinita

```
constexpr radix_t min_radix = 2
```

La minima base numerica

```
constexpr radix_t max_radix = 36
```

La massima base numerica

Metodi statici:

```
template <typename T>
[[nodiscard]] static constexpr bool is_unicode(T _v)noexcept
requires(concepts::utf32_charset<T>)
```

Ritorna vero se `v` è un `UTF32` valido

```
template <typename T>
[[nodiscard]] static constexpr bool is_ascii(T v)noexcept
requires(concepts::utf32_charset<T>)
```

Ritorna vero se `v` è un `ASCII` valido

```
[[nodiscard]] static constexpr unicode_version_t unicode_version()noexcept
```

Ritorna la versione di unicode usata

```
[[nodiscard]] static constexpr bool is_radix(radix_t r)noexcept
```

Ritorna vero se `r` è compreso nell' intervallo `[min_radix..max_radix]`

Costruttori:

```
constexpr chr()noexcept=default
constexpr chr(const chr&)noexcept
constexpr chr(chr&&)noexcept
```

```
template <typename T>
[[nodiscard]] static constexpr option<chr> from(T)noexcept
requires(concepts::utf32_charset<_T>)
```

Costruisce da un `UTF32`

```
[[nodiscard]] static constexpr option<chr> from_u32(u32_t)noexcept
```

Costruisce da un `u32_t`

```
[[nodiscard]] static constexpr option<chr> from(char)noexcept
```

Costruisce da un `basic charset`

```
template <typename T>
[[nodiscard]] static constexpr chr unchecked_from(T v)noexcept
requires(concepts::utf32_charset<_T>)
```

Costruisce da un `UTF32` senza verificare la validità di `v`

```
[[nodiscard]] static constexpr chr unchecked_from_u32(u32_t v)noexcept
```

Costruisce da un `u32_t` senza verificare la validità di `v` come `UTF32`

```
template <typename T1,typename T2>
[[nodiscard]] static constexpr chr from(T1 c1,T2 c2)noexcept
requires(concepts::utf32_charset<T1> && concepts::utf32_charset<T2>)
```

Costruisce da `c1` se valido altrimenti da `c2`
Se `c2` non è valido viene chiamata `PANIC()`

```
[[nodiscard]] static constexpr chr from_u32(u32_t v1, u32_t v2)noexcept
```

Costruisce da `v1` se è un `UTF32` valido altrimenti da `v2`
Se `v2` non è un `UTF32` valido viene chiamata `PANIC()`

```
[[nodiscard]] static constexpr option<chr>
from_digit(u32_t num, radix_t radix)noexcept
```

Costruisce convertendo da num avente come base radix

```
[[nodiscard]] static constexpr option<chr> from_digit(radix_t r)noexcept
```

Costruisce convertendo da r

Distruttore:

```
constexpr ~chr() = default
```

Metodi:

```
constexpr chr& operator=(const chr&)noexcept
constexpr chr& operator=(chr&&)noexcept
```

```
[[nodiscard]] constexpr u32_t to_u32()const noexcept
```

Ritorna la conversione in `u32_t`

```
template <typename T=char32_t>
[[nodiscard]] constexpr T to_utf32()const noexcept
requires(concepts::utf32_charset<T>)
```

Ritorna la conversione in `UTF32`

```
[[nodiscard]] constexpr option<char> to_basic()const noexcept
```

Ritorna la conversione in carattere basico

```
[[nodiscard]] constexpr option<u32_t> to_digit(radix_t radix)const noexcept
```

Ritorna la conversione in cifra
Se `radix` non è valido viene chiamata `PANIC()`

```
[[nodiscard]] constexpr bool is_unicode()const noexcept
```

Ritorna vero per un `UTF32` valido

```
[[nodiscard]] constexpr bool is_digit(radix_t rx)const noexcept
```

Ritorna vero se puo essere convertito in cifra avente come radice `rx`
Se `rx` non è valido viene chiamata `PANIC()`

```
[[nodiscard]] bool is_numeric()const noexcept
```

Ritorna vero se è una cifra `UTF32`

```
[[nodiscard]] bool is_alphabetic()const noexcept
```

Ritorna vero se è un lettera `UTF32`

```
[[nodiscard]] bool is_lowercase()const noexcept
```

Ritorna vero se è una lettera minuscola `UTF32`

`[[nodiscard]] bool is_uppercase()const noexcept`

Ritorna vero se è una lettera maiuscola `UTF32`

`[[nodiscard]] bool is_whitespace()const noexcept`

Ritorna vero se è uno spazio `UTF32`

`[[nodiscard]] bool is_alphanumeric()const noexcept`

Ritorna vero se è una lettera o cifra `UTF32`

`[[nodiscard]] bool is_control()const noexcept`

Ritorna vero se è un carattere di controllo `UTF32`

`[[nodiscard]] constexpr bool eq_ignore_case(const chr&)const noexcept`

Ritorna vero per uguaglianza al di la delle lettere minuscole/maiuscole

`[[nodiscard]] constexpr bool lt_ignore_case(const chr&)const noexcept`

Ritorna vero per minore al di la delle lettere minuscole/maiuscole

`[[nodiscard]] constexpr bool is_ascii()const noexcept`

Ritorna vero se è un carattere `ASCII`

`[[nodiscard]] constexpr bool is_ascii_alphabetic()const noexcept`

Ritorna vero se è una lettera `ASCII`

`[[nodiscard]] constexpr bool is_ascii_uppercase()const noexcept`

Ritorna vero se è una lettera maiuscola `ASCII`

`[[nodiscard]] constexpr bool is_ascii_lowercase()const noexcept`

Ritorna vero se è una lettera minuscola `ASCII`

`[[nodiscard]] constexpr bool is_ascii_alphanumeric()const noexcept`

Ritorna vero se è una lettera o una cifra `ASCII`

`[[nodiscard]] constexpr bool is_ascii_digit()const noexcept`

Ritorna vero se è una cifra `ASCII`

`[[nodiscard]] constexpr bool is_ascii_hexdigit()const noexcept`

Ritorna vero se è una cifra esadecimale `ASCII`

`[[nodiscard]] constexpr bool is_ascii_punctuation()const noexcept`

Ritorna vero se è un carattere di punteggiatura `ASCII`

`[[nodiscard]] constexpr bool is_ascii_graphic()const noexcept`

Ritorna vero se è un carattere stampabile `ASCII`

`[[nodiscard]] constexpr bool is_ascii_control()const noexcept`

Ritorna vero se è un carattere di controllo ASCII

`[[nodiscard]] constexpr chr to_ascii_uppercase()const noexcept`

Ritorna la conversione in lettera maiuscola `ASCII`
Se non è una lettera minuscola `ASCII` ritorna se stesso

`[[nodiscard]] constexpr chr to_ascii_lowercase()const noexcept`

Ritorna la conversione in lettera minuscola `ASCII`
Se non è una lettera maiuscola `ASCII` ritorna se stesso

`[[nodiscard]] constexpr bool eq_ignore_ascii_case(const chr& v)const noexcept`

Ritorna vero per uguaglianza al di la delle lettere minuscole/maiuscole
`ASCII`

`[[nodiscard]] constexpr bool lt_ignore_ascii_case(const chr& v)const noexcept`

Ritorna vero per minore al di la delle lettere minuscole/maiuscole
`ASCII`

`constexpr chr& make_ascii_uppercase()noexcept`

Converte in lettera maiuscola `ASCII` se è una lettera minuscola `ASCII`

`constexpr chr& make_ascii_lowercase()noexcept`

Converte in lettera minuscola `ASCII` se è una lettera maiuscola `ASCII`

`[[nodiscard]] constexpr const chr max(const chr& v)const noexcept`

Se è maggiore di `v` ritorna se stesso altrimenti `v`

`[[nodiscard]] constexpr const chr min(const chr& v)const noexcept`

Se è minore di `v` ritorna se stesso altrimenti `v`

`[[nodiscard]] constexpr size_t  size_utf8()const noexcept`

Ritorna il numero di `char8_t` necessari per la conversione
Se non è un `UTF32` valido ritorna zero

```
template <size_t N>
[[nodiscard]] constexpr size_t encode_utf8(char8_t(&buffer)[N])const noexcept
requires(N > 4)
```

Converte in `UTF8`, ritorna il numero di `char8_t` immagazzinati in buffer
senza conteggiare il `\0` finale
Se non è un `UTF32` valido ritorna zero

```
[[nodiscard]] constexpr size_t  size_utf16()const noexcept
```

Ritorna il numero di `char16_t` necessari per la conversione
Se non è un `UTF32` valido ritorna zero

```
template <typename T,size_t N>
[[nodiscard]] constexpr size_t encode_utf16(T(&buffer)[N])const noexcept
requires (N > 2 && concepts::utf16_charset<T>)
```

Converte in `UTF16`, ritorna il numero di `char16_t` immagazzinati in buffer
senza conteggiare il `\0` finale
Se non è un `UTF32` valido ritorna zero

```
[[nodiscard]] seq_t to_lowercase()const noexcept
```

Ritorna una sequenza per avere tutti i valori di lettere minuscole corrispondenti

```
[[nodiscard]] seq_t to_uppercase()const noexcept
```

Ritorna una sequenza per avere tutti i valori di lettere maiuscole corrispondenti

```
[[nodiscard]] seq_t escape_unicode()const noexcept
```

Ritorna una sequenza che contiene i valori per l'esadecimale
del valore corrente con il prefisso `0x`

```
[[nodiscard]] seq_t escape_debug()const noexcept
```

Ritorna una sequenza che contiene i valori per una rappresentazione
di caratteri non stampabile

```
[[nodiscard]] seq_t escape_default()const noexcept
```

Come escape debug

```
[[nodiscard]] constexpr bool operator==(const chr&)const noexcept
[[nodiscard]] constexpr auto operator<=>(const chr&)const noexcept
```

Confronto

chr::sequence

Uso:

sequenza per la classe `chr`

Testata:

```
iron/chr.h
```

Sinossi:

```
class chr::sequence final
```

Alias:

```
using value_t = chr
using iter_t = const value_t*
```

Costruttori:

```
sequence(const sequence&)noexcept
sequence(sequence&&)noexcept
```

Distruttore:

```
~sequence() = default
```

Metodi:

```
sequence& operator=(const sequence&)noexcept
sequence& operator=(sequence&&)noexcept
```

```
[[nodiscard]] iter_t begin()const noexcept
[[nodiscard]] iter_t end()const noexcept
```

Ritorna gli iteratori
Sono due metodi di basso livello, esistono solo per poter usare la sequenza in un ciclo for
Sconsigliato: meglio usare `while(seq) { auto v=*seq++;...}`

```
[[nodiscard]] option<value_t> next()noexcept
```

Ritorna il valore corrente ed avanza

```
[[nodiscard]] bool empty()const noexcept
```

Ritorna vero se la sequenza è vuota

```
[[nodiscard]] size_t size()const noexcept
```

Ritorna il numero di elementi

```
[[nodiscard]] explicit operator bool()const noexcept
```

Ritorna vero se la sequenza non è vuota

```
[[nodiscard]] value_t operator*()const noexcept
```

Ritorna il valore corrente
In caso di sequenza vuota viene chiamata `PANIC()`

```
[[nodiscard]] option<value_t> peek()const noexcept
```

Ritorna il valore corrente

```
sequence& operator++()noexcept
sequence operator++(int)noexcept
```

operatori di avanzamento

clone

Uso:

copiare una istanza

Testata:

`iron/clone.h`

Sinossi:

`class clone final`

Alias:

`using error_handler_t=error_handler`

Metodi statici:

```
template <typename T>
[[nodiscard]] constexpr result<T, error_handler_t> try_clone_from(const T&)noexcept
```

```
template <typename T>
[[nodiscard]] constexpr T clone_from(const T&)noexcept
```

Ritorna una copia

```
template <typename U, typename T>
[[nodiscard]] constexpr result<U, error_handler_t>
try_clone_from_to(const T&)noexcept;
```

```
template <typename U, typename T>
[[nodiscard]] constexpr U clone_from_to(const T&)noexcept
```

Ritorna una copia di `T` convertita nel tipo `U`

```
template <typename T,std::size_t N>
void try_clone_from(const T(&)[N]) = delete
```

```
template <typename T, std::size_t N>
void clone_from(const T(&)[N]) = delete
```

```
template <typename U, typename T,std::size_t N>
U try_clone_from_to(const T(&)[N])=delete
```

```
template <typename U, typename T, std::size_t N>
U clone_from_to(const T(&)[N]) = delete
```

Non è possibile clonare un `builtin_array`

Costruttori:

`clone()=delete`

Concetti esterni (spazio dei nomi `concepts`):

`copyable_or_cloneable<T>`

Vero se e' possibile usare `clone::try_clone_from(const T&)`

`nothrow_copyable_or_cloneable<T>`

Vero per `std::is_nothrow_copy_constructible_v<T> || cloneable<T>`

`std_tuple_copyable_or_cloneable<T>`

Vero se e' possibile usare la tupla `T` in `clone::try_clone_from(const T&)`

collector

Uso:

raccogliere dati in un contenitore

Testata:

`iron/collector.h`

Sinossi:

```
class collector final
```

Alias:

```
using error_handler_t=error_handler
```

Metodi statici:

```
template <typename T, std::size_t N>
[[nodiscard]] constexpr array<T, N> make_array()noexcept
requires(std::is_nothrow_default_constructible_v<T>)
```

Ritorna un array di `N` elementi inizializzati al valore predefinito

```
template <size_t N,typename T>
[[nodiscard]] constexpr result<array<std::decay_t<T>,N>,
error_handler_t> try_make_array_fill(T&& v)noexcept

template <size_t N,typename T>
[[nodiscard]] constexpr array<std::decay_t<T>,N>
make_array_fill(T&& v)noexcept
```

Ritorna un array di `N` elementi inizializzati al valore `v`

```
template <typename T,typename... ARGS>
[[nodiscard]] constexpr result<array<std::decay_t<T>, sizeof…
(ARGS) + 1>,error_handler_t> try_make_array(T&& v,ARGS&&... args)noexcept
requires (
        (sizeof...(ARGS) + 1 <=
                unsafe::raw_storage<std::decay_t<_T>>::max_size())
        &&
        (std::convertible_to<std::decay_t<ARGS>,std::decay_t<T>>&& ...)
)

template <typename T, typename... ARGS>
[[nodiscard]] constexpr array<std::decay_t<T>, sizeof...(ARGS) + 1>
make_array(T&& v,ARGS&&... args)noexcept
requires (
        (sizeof...(ARGS) + 1 <=
                unsafe::raw_storage<std::decay_t<T>>::max_size())
        &&
        (std::convertible_to<std::decay_t<ARGS>,std::decay_t<T>>&& ...)
)
```

Ritorna un array di `ARGS + 1` elementi inizializzati al valore `v` e `args`

```
template <typename T,typename... ARGS>
[[nodiscard]] constexpr result<vector<T>,error_handler_t>
try_make_vector(ARGS&&... args)noexcept
requires (!concepts::sequence<T> && !concepts::adapter_box<T>)

template <typename T,typename... ARGS>
[[nodiscard]] constexpr vector<T> make_vector(ARGS&&... args)noexcept
requires (!concepts::sequence<T> && !concepts::adapter_box<T>)
```

Ritorna un `vector` inizializzato da `args`
`ARGS` puo essere un `T`, oppure una sequenza di `T` oppure un `adapter_box`
`rvalue` di `T`

```
template <typename T, typename... ARGS>
[[nodiscard]] constexpr result<fwd_list<T>, error_handler_t>
try_make_fwd_list(ARGS&&... args)noexcept
requires (!concepts::sequence<T> && !concepts::adapter_box<T>)

template <typename T,typename... ARGS>
[[nodiscard]] constexpr fwd_LIST<T> make_fwd_list(ARGS&&... args)noexcept
requires (!concepts::sequence<T> && !concepts::adapter_box<T>)
```

Ritorna un `fwd_list` inizializzato da `args`
`ARGS` puo essere un `T`, oppure una sequenza di `T` oppure un `adapter_box` `rvalue` di `T`

```
template <typename T, typename... ARGS>
[[nodiscard]] constexpr result<iron::list<T>, error_handler_t>
try_make_list(ARGS&&... args)noexcept
requires (!concepts::sequence<T> && !concepts::adapter_box<T>)

template <typename T,typename... ARGS>
[[nodiscard]] constexpr list<T> make_list(ARGS&&... args)noexcept
requires (!concepts::sequence<T> && !concepts::adapter_box<T>)
```

Ritorna un `list` inizializzato da `args`
`ARGS` puo essere un `T`, oppure una sequenza di `T` oppure un `adapter_box` `rvalue` di `T`

```
template <typename T,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t> try_make_str(ARGS&&...
args)noexcept
requires (concepts::str<T>)

template <typename T,typename... ARGS>
[[nodiscard]] constexpr T make_str(ARGS&&...args)noexcept
requires (concepts::str<T>)

template <typename T=ustrb,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_ustrb(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustrb>)

template <typename T=ustrb,typename... ARGS>
[[nodiscard]] constexpr T make_ustrb(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustrb>)

template <typename T=ustr8,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_ustr8(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustr8>)

template <typename T=ustr8,typename... ARGS>
[[nodiscard]] constexpr T make_ustr8(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustr8>)

template <typename T=ustr16,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_ustr16(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustr16>)

template <typename T=ustr16,typename... ARGS>
[[nodiscard]] constexpr T make_ustr16(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustr16>)

template <typename T=ustrw,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_ustrw(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustrw>)

template <typename T=ustrw,typename... ARGS>
[[nodiscard]] constexpr T make_ustrw(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustrw>)

template <typename T=ustr32,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_ustr32(ARGS&&... args)noexcept
requires(std::is_same_v<T,ustr32>)
```

```
template <typename T=strb,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_strb(ARGS&&... args)noexcept
requires(std::is_same_v<T,strb>)

template <typename T=strb,typename... ARGS>
[[nodiscard]] constexpr T make_strb(ARGS&&... args)noexcept
requires(std::is_same_v<T,strb>)

template <typename T=str8,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_str8(ARGS&&... args)noexcept
requires(std::is_same_v<T,str8>)


template <typename T=str8,typename... ARGS>
[[nodiscard]] constexpr T make_str8(ARGS&&... args)noexcept
requires(std::is_same_v<T,str8>)

template <typename T=str16,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_str16(ARGS&&... args)noexcept
requires(std::is_same_v<T,str16>)

template <typename T=str16,typename... ARGS>
[[nodiscard]] constexpr T make_str16(ARGS&&... args)noexcept
requires(std::is_same_v<T,str16>)

template <typename T=strw,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_strw(ARGS&&... args)noexcept
requires(std::is_same_v<T,strw>)

template <typename T=strw,typename... ARGS>
[[nodiscard]] constexpr T make_strw(ARGS&&... args)noexcept
requires(std::is_same_v<T,strw>)

template <typename T=str32,typename... ARGS>
[[nodiscard]] constexpr result<T, error_handler_t>
try_make_str32(ARGS&&... args)noexcept
requires(std::is_same_v<T,str32>)

template <typename T=str32,typename... ARGS>
[[nodiscard]] constexpr T make_str32(ARGS&&... args)noexcept
requires(std::is_same_v<T,str32>)
```

Ritorna una stringa con i valori inizializzati da `args`
`ARGS` deve essere un `charset` oppure una sequenza di `charset` oppure un `rvalue` `adapter_box` di `charset`

```
template <typename K,typename M,typename... ARGS>
[[nodiscard]] constexpr result<hash_map<K,M>, error_handler_t>
try_make_hash_map(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>
&& !concepts::sequence<M> && !concepts::adapter_box<M>)

template <typename K,typename M,typename... ARGS>
[[nodiscard]] constexpr hash_map<K,M> make_hash_map(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>
&& !concepts::sequence<M> && !concepts::adapter_box<M>)
```

Ritorna un `hash map` con i valori inizializzati da `args`
Il tipo `ARGS` viene alternato tra `K` e `M`

```
template <typename K,typename M,typename... ARGS>
[[nodiscard]] constexpr result<multi_hash_map<K,M>, error_handler_t>
try_make_multi_hash_map(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>
&& !concepts::sequence<M> && !concepts::adapter_box<M>)

template <typename K,typename M,typename... ARGS>
[[nodiscard]] constexpr multi_hash_map<K,M> make_multi_hash_map(ARGS&&...
args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>
&& !concepts::sequence<M> && !concepts::adapter_box<M>)
```

Ritorna un `multi hash map` con i valori inizializzati da `args`
Il tipo di `ARGS` viene alternato tra `K` e `M`

```
template <typename K,typename... ARGS>
[[nodiscard]] constexpr result<hash_set<K>, error_handler_t>
try_make_hash_set(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>)

template <typename K,typename... ARGS>
[[nodiscard]] constexpr hash_set<K> make_hash_set(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>)
```

Ritorna un `hash set` con i valori inizializzati da `args`

```
template <typename K,typename... ARGS>
[[nodiscard]] constexpr result<multi_hash_set<K>, error_handler_t>
try_make_multi_hash_set(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>)

template <typename K,typename... ARGS>
[[nodiscard]] constexpr multi_hash_set<K> make_multi_hash_set(
        ARGS&&... args
)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>)
```

Ritorna un `multi hash set` con i valori inizializzati da `args`

```
template <typename K,typename M,typename... ARGS>
[[nodiscard]] constexpr result<btree_map<K,M>, error_handler_t>
try_make_btree_map(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>
&& !concepts::sequence<M> && !concepts::adapter_box<M>)

template <typename K,typename M,typename... ARGS>
[[nodiscard]] constexpr btree_map<K,M> make_btree_map(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>
&& !concepts::sequence<M> && !concepts::adapter_box<M>)
```

Ritorna un `btree map` con i valori inizializzati da `args`
Il tipo `ARGS` viene alternato tra `K` e `M`

```
template <typename K,typename M,typename... ARGS>
[[nodiscard]] constexpr result<multi_btree_map<K,M>, error_handler_t>
try_make_multi_btree_map(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>
&& !concepts::sequence<M> && !concepts::adapter_box<M>)

template <typename K,typename M,typename... ARGS>
[[nodiscard]] constexpr multi_btree_map<K,M> make_multi_btree_map(
        ARGS&&... args
)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>
&& !concepts::sequence<M> && !concepts::adapter_box<M>)
```

Ritorna un `multi btree map` con i valori inizializzati da `args`
Il tipo di `ARGS` viene alternato tra `K` e `M`

```
template <typename K,typename... ARGS>
[[nodiscard]] constexpr result<btree_set<K>, error_handler_t>
try_make_btree_set(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>)

template <typename K,typename... ARGS>
[[nodiscard]] constexpr btree_set<K> make_btree_set(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>)
```

Ritorna un `btree set` con i valori inizializzati da `args`

```
template <typename K,typename... ARGS>
[[nodiscard]] constexpr result<multi_btree_set<K>, error_handler_t>
try_make_multi_btree_set(ARGS&&... args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>)
```

```
template <typename K,typename... ARGS>
[[nodiscard]] constexpr multi_btree_set<K> make_multi_btree_set(ARGS&&...
args)noexcept
requires (!concepts::sequence<K> && !concepts::adapter_box<K>)
```

Ritorna un `multi btree set` con i valori inizializzati da `args`

Costruttori:

`collector()=delete`

comparables::equal_to

Uso:

classe per confronto di uguaglianza

Testata:

```
iron/comparables.h
```

Sinossi:

```
template <typename T> class equal_to final
```

Alias:

```
using value_t = T
using value_type = value_t
```

Costruttori:

```
constexpr equal_to()noexcept=default
constexpr equal_to(const equal_to&)noexcept
constexpr equal_to(equal_to&&)noexcept
```

Distruttore:

```
constexpr ~equal_to()=default
```

Metodi:

```
constexpr equal_to& operator=(const equal_to&)noexcept
constexpr equal_to& operator=(equal_to&&)noexcept
```

```
[[nodiscard]] constexpr bool operator()(const value_t&, const value_t&)const
noexcept
requires(concepts::nothrow_equality_comparable<value_t>)
```

Ritorna vero per uguaglianza

comparables::less_to

Uso:

classe per confronto di minoranza

Testata:

```
iron/comparables.h
```

Sinossi:

```
template <typename T> class less_to final
```

Alias:

```
using value_t = T
using value_type = value_t
```

Costruttori:

```
constexpr less_to()noexcept=default
constexpr less_to(const less_to&)noexcept
constexpr less_to(less_to&&)noexcept
```

Distruttore:

```
constexpr ~less_to()=default
```

Metodi:

```
constexpr less_to& operator=(const less_to&)noexcept
constexpr less_to& operator=(less_to&&)noexcept
```

```
[[nodiscard]] constexpr bool operator()(const value_t&, const value_t&)const noexcept
requires(concepts::nothrow_less_comparable<value_t>)
```

Ritorna vero per minoranza

enum_set

Uso:

`wrapper` su `bitset` per usare come un insieme di bit i valori di un `enum`.
La `enum` deve avere come `underlying_type` un tipo non segnato,avere valori consecutivi ed il primo valore numerico a zero

Testata:

```
iron/enum_set.h
```

Sinossi:

```
template <typename T> class enum_set final
```

Alias:

```
using value_t = T;
using underlying_type_t = std::underlying_type_t<T>
```

Vincoli:

```
std::is_enum_v<value_t>
std::is_unsigned_v<underlying_type_t>
```

Metodi statici:

```
[[nodiscard]] static constexpr underlying_type_t numeric_value(value_t v)noexcept
```

Converte il valore di `v` in numero

```
[[nodiscard]] static constexpr bool is_bit(value_t v)noexcept
```

Ritorna vero se il valore di `v` è valido

Costruttori:

```
constexpr enum_set()noexcept=default
constexpr enum_set(const enum_set&)noexcept=default
constexpr enum_set(enum_set&&)noexcept=default
```

Costruttori esterni:

```
template <typename T>
[[nodiscard]] constexpr enum_set<T> make_enum_set()noexcept
requires(std::is_enum_v<T>)

template <typename T,typename... ARGS>
[[nodiscard]] constexpr enum_set<T> make_enum_set(T,ARGS...)noexcept
requires(std::is_enum_v<T> && (std::is_same_v<T,ARGS>&& …))
```

Distruttore:

```
constexpr ~enum_set()noexcept=default
```

Metodi:

```
constexpr enum_set& operator=(const enum_set&)noexcept=default
constexpr enum_set& operator=(enum_set&&)noexcept=default
constexpr enum_set& swap(enum_set& _v)noexcept
```

```
constexpr enum_set& set(value_t v)noexcept
```

Imposta a vero il bit corrispondente a `v`

```
constexpr enum_set& reset(value_t v)noexcept
```

Imposta a falso il bit corrispondente a `v`

```
[[nodiscard]] constexpr bool test(value_t v)const noexcept
```

Ritorna vero se il bit corrispondente a `v` è impostato

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se almeno un bit è impostato

```
[[nodiscard]] constexpr bool test_or(const std::initializer_list<value_t>& v)const
noexcept
```

Ritorna vero se almeno un bit dei corrispondenti valori di `v` è impostato

```
[[nodiscard]] constexpr bool test_and(const std::initializer_list<value_t>& v)const
noexcept
```

Ritorna falso se almeno un bit dei corrispondenti valori di `v` non è impostato

error_handler

Uso:

gestione degli errori

Testata:

```
iron/error_handler.h
```

Sinossi:

```
class error_handler final
```

Alias:

```
using line_number_t = std::uint_least32_t
using size_t=std::size_t
using stacktrace_t = memory::stacktrace
using base_t = unsafe_errors::base
typedef enum class enum_error_type {
        no_error
        ,posix
        , os
        , exception
        , user
} error_type_t
```

Costanti statiche:

```
const char* stacktrace_enable_env = "IRON_STACKTRACE"
```

Il nome della variabile d'ambiente per abilitare la stampa dello `stack trace`

```
constexpr auto max_count = std::numeric_limits<size_t>::max()
```

Il massimo valore per il conteggio delle istanza condivise

Costruttori:

```
constexpr error_handler()noexcept=default
constexpr error_handler(error_handler&&)noexcept
```

```
constexpr error_handler(const error_handler&)noexcept
```

Costruttore di copia
In realtà non viene effettuata una copia dell' istanza dell'errore interno ma viene solo condiviso
L'accesso al contatore delle copie condivise è sicuro rispetto ai `threads`
Se il contatore tenta di superare il valore `max_count` viene chiamata `PANIC()`

```
template <typename T>
error_handler(T&& v)noexcept
requires (std::is_base_of_v<unsafe_errors::base, T>)
```

Costruisce da `v`

```
[[nodiscard]] static error_handler make(
               unsafe_errors::posix::code_t code
               ,const char* file
               ,line_number_t line
               ,const char* func
)noexcept
```

Costruisce un errore `POSIX`.
Se `code` è `posix::code_ok` viene ritornata una istanza senza errore impostato
`file` e `func` possono essere `nullptr`

```
[[nodiscard]] static error_handler make(
               unsafe_errors::os::code_t code
               ,const char* file
               ,line_number_t line
               ,const char* func)noexcept
```

Costruisce un errore da un codice d' errore del sistema operativo
Se `code` è `os::code_t{}` viene ritornata una istanza senza errore impostato.
`file` e `func` possono essere `nullptr`

```
[[nodiscard]] static error_handler make(
               std::exception_ptr&& excp
               ,const char* file
               ,line_number_t line
               ,const char* func
               )noexcept
```

Costruisce un errore da `std::exception_ptr`
`file` e `func` possono essere `nullptr`
Se `excp` e' convertibile in `posix` o `os` ritorna un errore impostato
su uno di questi.

Distruttore:

```
constexpr ~error_handler()
```

Se l'errore è impostato non è possibile usarlo in valutazione a tempo di compilazione

Metodi:

```
constexpr error_handler& operator=(const error_handler&)noexcept
```

Come per il costruttore di copia usa una istanza condivisa per l'errore

```
constexpr error_handler& operator=(error_handler&&)noexcept
constexpr error_handler& swap(error_handler&)noexcept
```

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se nessun errore è impostato

```
[[nodiscard]] const base_t* unsafe_error()const noexcept
```

Ritorna una istanza di `errors::base` se impostata altrimenti un puntatore nullo.
Il metodo non è sicuro perché ritorna un puntatore

```
[[nodiscard]] error_type_t error_type()const noexcept
```

Ritorna il tipo dell'errore

`[[nodiscard]] unsafe::errors::os::code_t numeric_error()const noexcept`

Ritorna il codice numerico dell'errore.
Se l'errore non è impostato ritorna `os::code_t{}`
Se l'errore impostato e' un errore del sistema operativo o `POSIX`
ritorna il codice numerico altrimenti un codice per indicare errore sconosciuto
che dipende dal sistema operativo.

`[[nodiscard]] const char* unsafe_code()const noexcept`

Ritorna il codice di errore in formato stringa
Il metodo non è sicuro perché ritorna un puntatore
Il valore ritornato non è mai `nullptr`

`[[nodiscard]] const char8_t* unsafe_description()const noexcept`

Ritorna la descrizione dell'errore
Il metodo non è sicuro perché ritorna un puntatore
Il valore ritornato non è mai `nullptr`

`[[nodiscard]] const char* unsafe_file()const noexcept`

Ritorna il nome del `file` in cui si è verificato l'errore
Il valore ritornato non è mai `nullptr`

`[[nodiscard]] line_number_t line()const noexcept`

Ritorna il numero di riga del `file` in cui si è verificato l'errore

`[[nodiscard]] const char* unsafe_func()const noexcept`

Ritorna il nome della funzione in cui si è verificato l'errore
Il valore ritornato non è mai `nullptr`

`[[nodiscard]] constexpr bool has_stacktrace()const noexcept`

Ritorna vero se il tracciamento dello `stack trace` è impostato
Il tracciamento è impostato se la variabile d'ambiente impostata in
`stacktrace_enable_env` esiste ed è a valore non zero

`[[nodiscard]] const stacktrace_t& unsafe_stacktrace()const noexcept`

Ritorna lo `stacktrace_t` impostato nel punto in cui l'istanza `error_handler` è
costruita con errore impostato
Ritorna uno `stacktrace` vuoto se `has_stacktrace()` ritorna falso
il metodo non è sicuro perché ritorna un riferimento

`[[noreturn]] void do_panic()const noexcept`

Forza la terminazione del programma.
E' simile alla chiamata della macro `PANIC()`

hasher

Uso:

classe per calcolare un valore `hash`

Testata:

```
iron/hasher.h
```

Sinossi:

```
template <typename T> class hasher final
```

Alias:

```
using value_t = T
using value_type = value_t
using size_t=std::size_t
using size_type = size_t
```

Costruttori:

```
constexpr hasher()noexcept=default
constexpr hasher(const hasher&)noexcept
constexpr hasher(hasher&&)noexcept
```

Distruttore:

```
constexpr ~hasher()=default
```

Metodi:

```
constexpr hasher& operator=(const hasher&)noexcept
constexpr hasher& operator=(hasher&&)noexcept
```

```
[[nodiscard]] constexpr size_t operator()(const value_t&)const noexcept
```

Calcola il valore di hash

hash_table

Uso:

Implementazione di un `hash table set` o `map`

Testata:

`iron/hash_table.h`

Sinossi:

```
template <
        typename KEY
        ,typename MAPPED
        ,typename EQ
        ,typename HASHER
        ,typename A
        ,bool MULTI
        ,bool HAS_MAPPED
>
class hash_table final
```

Alias:

```
using key_t = KEY
using key_type = key_t
using mapped_t = MAPPED
using mapped_type = mapped_t
using allocator_t = A
using allocator_type = allocator_t
using key_equal_t = EQ
using key_equal_type = key_equal_t
using hasher_t = HASHER
using hasher_type = hasher_t
using value_t = <private>
using value_type = value_t
using exported_value_t = <private>
using exported_value_type = exported_value_t
using size_t = std::size_t
using size_type = size_t
using ptr_t = value_t*
using pointer = ptr_t
using cptr_t = const value_t*
using const_pointer = cptr_t
using diff_t = std::ptrdiff_t
using difference_type = diff_t
using reference = value_t&;
using const_reference = const value_t&
using ref_t = ref_wrapper<value_t>
using cref_t = ref_wrapper<const value_t>
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using load_factor_t = float
using load_factor_type = load_factor_t
using adapter_box_t = <private>
using iterator = <private>
using const_iterator = <private>
using seq_t = sequence<iterator>
using cseq_t = sequence<const_iterator>
using list_t = <private>
```

Alias esterni:

```
template <
        typename KEY
        ,typename MAPPED
        ,typename EQ
        ,typename HASHER
        ,typename A
>
using base_hash_map = hash_table<KEY, MAPPED, EQ, HASHER,A,false, true>


template <
        typename KEY
        , typename MAPPED
        , typename EQ = comparables::equal_to<KEY>
        , typename HASHER = hasher<KEY>
        , typename A =unsafe::allocator<std::pair<KEY, MAPPED>>
>
using hash_map = base_hash_map<KEY, MAPPED, EQ, HASHER, A>
```

```
template <
        typename KEY
        , typename EQ
        , typename HASHER
        , typename A
>
using base_hash_set = hash_table<KEY, <private>, EQ, HASHER,A,false,false>
```

```
template <
        typename KEY
        , typename EQ = comparables::equal_to<KEY>
        , typename HASHER = hasher<KEY>
        , typename A =unsafe::allocator<KEY>
>
using hash_set = base_hash_set<KEY,EQ, HASHER, A>
```

```
template <
        typename KEY
        , typename MAPPED
        , typename EQ
        , typename HASHER
        , typename A
>
using base_multi_hash_map = hash_table<KEY, MAPPED, EQ, HASHER, A,true, true>
```

```
 template <
        typename KEY
        , typename MAPPED
        , typename EQ = comparables::equal_to<KEY>
        , typename HASHER = hasher<KEY>
        , typename A =unsafe::allocator<std::pair<KEY, MAPPED>>
 >
 using multi_hash_map = base_multi_hash_map<KEY, MAPPED, EQ, HASHER, A>
```

```
 template <
        typename KEY
        , typename EQ
        , typename HASHER
        , typename A
  >
 using base_multi_hash_set = hash_table<KEY, <private>, EQ, HASHER, A, true, false>
```

```
 template <
        typename KEY
        , typename EQ = comparables::equal_to<KEY>
        , typename HASHER = hasher<KEY>
        , typename A =unsafe::allocator<KEY>
 >
using multi_hash_set = base_multi_hash_set<KEY, EQ, HASHER, A>
```

Costanti statiche:

```
constexpr bool is_multi = MULTI
constexpr bool has_mapped = HAS_MAPPED
```

Metodi statici:

```
[[nodiscard]] constexpr load_factor_t default_load_factor()noexcept
```

Ritorna il fattore di carico predefinito

```
[[nodiscard]] constexpr size_t max_size()noexcept
```

Ritorna il massimo numero di elementi caricabili

```
[[nodiscard]] constexpr size_t max_bucket_count()noexcept
```

Ritorna il massimo numero di secchi caricabili

```
[[nodiscard]] constexpr load_factor_t max_load_factor()
```

Ritorna il massimo fattore di carico

Vincoli:

```
sizeof(key_t) > 0
!std::is_reference_v<key_t>
!std::is_array_v<key_t>
!std::is_const_v<key_t>
std::is_nothrow_destructible_v<key_t>
std::is_nothrow_move_constructible_v<key_t>
std::is_nothrow_swappable_v<key_t>
!has_mapped || !std::is_reference_v<mapped_t>
!has_mapped || !std::is_array_v<mapped_t>
!has_mapped || !std::is_const_v<mapped_t>
!has_mapped || std::is_nothrow_destructible_v<mapped_t>
!has_mapped || std::is_nothrow_move_constructible_v<mapped_t>
!has_mapped || std::is_nothrow_swappable_v<mapped_t>
concepts::allocator<allocator_t>
concepts::hashable<hasher_t, key_t>
concepts::comparable<key_equal_t, key_t>
```

Costruttori:

```
[[nodiscard]] static constexpr result<hash_table, error_handler_t>
try_from(adapter_box_t&& box,load_factor_t lf = default_load_factor())noexcept

[[nodiscard]] static constexpr hash_table
from(adapter_box_t&& box,load_factor_t lf = default_load_factor())noexcept
```

Costruisce da `rvalue adapter_box_t` con fattore di carico `lf`

```
[[nodiscard]] static constexpr hash_table unchecked_from(allocator_t)noexcept
```

Costruisce il contenitore senza elementi impostando l'allocatore
Il metodo non è sicuro perchè `allocator_t` non è sicuro

```
constexpr hash_table()noexcept
hash_table(const hash_table&)=delete
constexpr hash_table(hash_table&&)noexcept
```

Distruttore:

```
constexpr ~hash_table()=default
```

Metodi:

```
hash_table& operator=(const hash_table&)=delete
constexpr hash_table& operator=(hash_table&&)noexcept
constexpr hash_table& swap(hash_table&)noexcept
```

```
[[nodiscard]] constexpr load_factor_t load_factor_threshold()const noexcept
```

Ritorna il fattore di carico impostato

```
[[nodiscard]] constexpr load_factor_t load_factor()const noexcept
```

Ritorna il fattore di carico effettivo

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se il contenitore ha elementi

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se il contenitore non ha elementi

```
[[nodiscard]] constexpr size_t size()const noexcept
```

Ritorna il numero di elementi

```
[[nodiscard]] constexpr size_t bucket_count()const noexcept
```

Ritorna il numero di secchi

```
[[nodiscard]] constexpr allocator_t unsafe_allocator()const noexcept
```

Ritorna l'allocatore
Il metodo non è sicuro perche `allocator_t` non è sicuro

```
[[nodiscard]] constexpr seq_t seq()noexcept
[[nodiscard]] constexpr cseq_t seq()const noexcept
[[nodiscard]] constexpr cseq_t cseq()const noexcept
```

Ritorna una sequenza di elementi
Gli elementi non sono ordinati

```
[[nodiscard]] constexpr result<hash_table, error_handler_t>
try_clone()const noexcept
requires(
	concepts::copyable_or_cloneable<key_t>
	&& concepts::copyable_or_cloneable<mapped_t>
)

[[nodiscard]] constexpr hash_table clone()const noexcept
requires(
	concepts::copyable_or_cloneable<key_t>
	&& concepts::copyable_or_cloneable<mapped_t>
)
```

Ritorna una copia mantenendo il fattore di carico effettivo

```
[[nodiscard]] constexpr result<hash_table, error_handler_t>
try_clone(load_factor_t lf)const noexcept
requires(
	concepts::copyable_or_cloneable<key_t>
	&& concepts::copyable_or_cloneable<mapped_t>
)

[[nodiscard]] constexpr hash_table clone(load_factor_t lf)const noexcept
requires(
	concepts::copyable_or_cloneable<key_t>
	&& concepts::copyable_or_cloneable<mapped_t>
)
```

Ritorna una copia con impostato il fattore di carico `lf`
Questa operazione è piu lenta del semplice `clone()`

```
[[nodiscard]] constexpr error_handler_t try_rehash(load_factor_t lf)noexcept
constexpr hash_table& rehash(load_factor_t lf)noexcept
```

Riorganizza gli elementi in modo da avere un fattore di carico impostato a `lf`
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t try_reserve(size_t addictional)noexcept
constexpr hash_table& reserve(size_t addictional)noexcept
```

Riorganizza gli elementi in modo da mantere il fattore di carico impostato aggiungendo `addictional` elementi
L'operazione è transazionale

```
[[nodiscard]] constexpr seq_t find(const key_t& k)noexcept
[[nodiscard]] constexpr cseq_t find(const key_t& k)const noexcept
[[nodiscard]] constexpr cseq_t cfind(const key_t& k)const noexcept
```

Ritorna una sequenza di elementi il cui primo elemento è uguale `k` usando `equal_t`
In caso non esista nessun elemento con chiave `k` viene ritornata
una sequenza vuota

```
template <typename K,typename M>
[[nodiscard]] constexpr result<std::pair<seq_t,bool>, error_handler_t>
try_insert(K&& key,M&& mapped)noexcept
requires (
	has_mapped
	&& std::constructible_from<key_t,K>
	&& std::constructible_from<mapped_t,M>
)

template <typename K,typename M>
constexpr std::pair<seq_t,bool> insert(K&& key,M&& mapped)noexcept
requires (
	has_mapped
	&& std::constructible_from<key_t,K>
	&& std::constructible_from<mapped_t,M>
)
```

Inserisce `key` + `mapped`
Non ci devono essere referenti attivi
Ritorna una sequenza la cui chiave del primo elemento è uguale a `key` usando `key_equal_t`
ed un valore booleano che indica se `key` + `mapped` è stato inserito
L'operazione è transazionale.

```
template <typename K>
[[nodiscard]] constexpr result<std::pair<seq_t, bool>, error_handler_t>
try_insert(K&& key)noexcept
requires (
	!has_mapped
	&& std::constructible_from<key_t,K>
)

template <typename K>
constexpr std::pair<seq_t, bool> insert(K&& key)noexcept
requires (
	!has_mapped
	&& std::constructible_from<key_t, K>
)
```

Inserisce `key` nel contenitore
Non ci devono essere referenti attivi
Ritorna una sequenza la cui primo elemento è uguale a `key` usando `key_equal_t`
ed un valore booleano che indica se `key` è stato inserito
L'operazione è transazionale

```
template <typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t> try_insert_range(
                SEQ& seq
)noexcept
        requires(
                has_mapped
                && concepts::sequence<SEQ>
                && !std::is_const_v<SEQ>
                && concepts::std_pair<typename SEQ::value_t>
                && concepts::constructible_at<key_t, std::decay_t<typename
                SEQ::value_t::first_type>&&>
                && concepts::constructible_at<mapped_t, std::decay_t<typename
                SEQ::value_t::second_type>&&>
        )
```

```
template <typename SEQ>
constexpr size_t insert_range(SEQ& seq)noexcept
requires(
        has_mapped
        && concepts::sequence<SEQ>
        && concepts::std_pair<typename SEQ::value_t>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t::first_type>&&>
        && concepts::constructible_at<mapped_t,std::decay_t<typename
        SEQ::value_t::second_type>&&>
        )
```

Inserisce i valori di una sequenza
Non ci devono essere referenti attivi
I valori vengono copiati e viene ritornato il numero di elementi inseriti
`seq` sarà consumato alla fine dell'operazione se non ci sono errori
L'operazione non è transazionale

```
template <typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_insert_range(SEQ& seq)noexcept
        requires(
        !has_mapped
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

```
template <typename SEQ>
constexpr size_t insert_range(SEQ& seq)noexcept
requires(
        !has_mapped
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<key_t,std::decay_t<typename
        SEQ::value_t>&&>
        )
```

Inserisce i valori di una sequenza
Non ci devono essere referenti attivi
I valori vengono copiati e viene ritornato il numero di elementi inseriti
`seq` sarà consumato alla fine dell'operazione se non ci sono errori
L'operazione non è transazionale

```
template <typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_transactional_insert_range(
                SEQ& seq
)noexcept
        requires(
                has_mapped
                && concepts::sequence<SEQ>
                && !std::is_const_v<SEQ>
                && concepts::std_pair<typename SEQ::value_t>
                && concepts::constructible_at<key_t, std::decay_t<typename
                SEQ::value_t::first_type>&&>
                && concepts::constructible_at<mapped_t, std::decay_t<typename
        SEQ::value_t::second_type>&&>
        )
```

```
template <typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_transactional_insert_range(SEQ& seq)noexcept
        requires(
        !has_mapped
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

```
template <typename SEQ>
constexpr size_t transactional_insert_range(SEQ& seq)noexcept
requires(
        has_mapped
        && concepts::sequence<SEQ>
        && concepts::std_pair<typename SEQ::value_t>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t::first_type>&&>
        && concepts::constructible_at<mapped_t,std::decay_t<typename
        SEQ::value_t::second_type>&&>
        )
```

```
template <typename SEQ>
constexpr size_t transactional_insert_range(SEQ& seq)noexcept
        requires(
        !has_mapped
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<key_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

Come `insert_range` ma transazionale

`constexpr btree_table& clear()noexcept`

Elimina tutti gli elementi
Non ci devono essere referenti attivi
Dopo questa operazione `empty()` ritorna vero
La memoria interna non viene tutta deallocata
ed il fattore di carico impostato rimane invariato.

`constexpr btree_table& reset()noexcept`

Elimina tutti gli elementi
Non ci devono essere referenti attivi
Dopo questa operazione `empty()` ritorna vero
Tutta la memoria interna viene deallocata
e viene impostato il fattore di carico predefinito

```
template <typename K,typename... ARGS >
constexpr size_t erase(const K& key,const ARGS&... args)noexcept
requires (
        std::is_nothrow_convertible_v<std::decay_t<K>, key_t>
        &&
        (std::is_nothrow_convertible_v<std::decay_t<ARGS>, key_t>& ...)
)
```

Rimuove una lista di chiavi
Non ci devono essere referenti attivi
Ritorna il numero di elementi rimossi

```
[[nodiscard]] constexpr result<size_t,error_handler_t>
try_merge(hash_table& oth)noexcept

constexpr size_t merge(hash_table& oth)noexcept
```

Inserisce elementi da `oth`
E' piu efficiente di un `insert_range` perchè i valori di `oth` vengono spostati
Non ci devono essere referenti attivi sia nel contenitore che in `oth`
Ritorna il numero di elementi inseriti
L'operazione è transazionale

```
template <typename... ARGS>
[[nodiscard]] constexpr result<adapter_box_t, error_handler_t>
try_release_list(const ARGS&... keys)noexcept
requires (std::is_nothrow_convertible_v<std::decay_t<ARGS>, key_t>& …)

template <typename... ARGS>
[[nodiscard]] constexpr adapter_box_t release_list(const ARGS&... keys)noexcept
requires (std::is_nothrow_convertible_v<std::decay_t<ARGS>, key_t>& ...)
```

Esporta gli elementi con chiave presente nell' elenco `keys`.
Non ci devono essere referenti attivi
L'operazione è transazionale.

```
[[nodiscard]] constexpr result<adapter_box_t, error_handler_t>
try_release_all()noexcept

[[nodiscard]] constexpr adapter_box_t release_all()noexcept
```

Esporta tutti gli elementi
Non ci devono essere referenti attivi
L'operazione è transazionale.

```
[[nodiscard]] constexpr const list_t* unsafe_data()const noexcept
[[nodiscard]] constexpr const list_t* unsafe_cdata()const noexcept
[[nodiscard]] constexpr list_t* unsafe_data()noexcept
```

Ritorna il puntatore alla prima lista interna
L'operazione non è sicura perché accedere alla lista interna non è sicuro

integrals

Uso:

Per operazioni arimetiche e bit sicure

Testata:

```
iron/integrals.h
```

Sinossi:

```
template <typename T> class base_integral final
```

Alias:

```
using value_t = T
using value_type = value_t
using uvalue_t = <private>
using unsigned_value_type = uvalue_t
using svalue_t = <private>
using signed_value_type = svalue_t
using base32_t = base_integral<std::uint32_t>
using base32_type = base32_t
```

Alias esterni:

```
using i8 = base_integral<std::int8_t>
using i16 = base_integral<std::int16_t>
using i32 = base_integral<std::int32_t>
using i64 = base_integral<std::int64_t>
using isize = base_integral<std::ptrdiff_t>
using u8 = base_integral<std::uint8_t>
using u16 = base_integral<std::uint16_t>
using u32 = base_integral<std::uint32_t>
using u64 = base_integral<std::uint64_t>
using usize = base_integral<std::size_t>
```

Vincoli:

```
concepts::numeric_integral<value_t>
```

Costanti statiche:

```
constexpr std::u32_t char_bits = <private>
```

Il numero di `bits` in un `char`

Metodi statici:

```
[[nodiscard]] constexpr value_t min()noexcept
```

Ritorna il valore minimo di `value_t`

```
[[nodiscard]] constexpr value_t max()noexcept
```

Ritorna il valore massimo di `value_t`

```
[[nodiscard]] constexpr bool is_signed()noexcept
```

Ritorna vero se `value_t` è segnato

```
[[nodiscard]] static constexpr bool is_unsigned()noexcept
```

Ritorna vero se `value_t` non è segnato

```
[[nodiscard]] constexpr base32_t bits()noexcept
```

Ritorna il numero di `bits` in `value_t`

Costruttori:

```
constexpr base_integral()noexcept
constexpr base_integral(value_t)noexcept
constexpr base_integral(const base_integral&)noexcept
constexpr base_integral(base_integral&&)noexcept
```

```
[[nodiscard]] static constexpr base_integral from_be(const base_integral& v)noexcept
```

Ritorna un `integral` il cui valore è convertito da un `big endian`

```
[[nodiscard]] static constexpr base_integral from_le(const base_integral& v)noexcept
```

Ritorna un `integral` il cui valore è convertito da un `little endian`

Distruttore:

```
constexpr ~base_integral()=default
```

Metodi:

```
constexpr base_integral& operator=(const base_integral&)noexcept
constexpr base_integral& operator=(base_integral&&)noexcept
constexpr base_integral& swap(base_integral&)noexcept
```

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se il valore non è zero

```
[[nodiscard]] constexpr value_t value()const noexcept
```

Ritorna il valore al tipo `value_t`

```
[[nodiscard]] constexpr base32_t count_ones()const noexcept
```

Ritorna il conteggio dei bit impostati

```
[[nodiscard]] constexpr base32_t leading_ones()const noexcept
```

Ritorna il conteggio dei bits impostati fino ad incontrare un bit non impostato iniziando a contare dal bit piu significativo

```
[[nodiscard]] constexpr base32_t trailing_ones()const noexcept
```

Ritorna il conteggio dei bits impostati fino ad incontrare un bit non impostato iniziando a contare dal bit meno significativo

```
[[nodiscard]] constexpr base32_t count_zeros()const noexcept
```

Ritorna il conteggio dei bits non impostati

```
[[nodiscard]] constexpr base32_t leading_zeros()const noexcept
```

Ritorna il conteggio dei bit non impostati fino ad incontrare un bit impostato iniziando a contare dal bit piu significativo

```
[[nodiscard]] constexpr base32_t trailing_zeros()const noexcept
```

Ritorna il conteggio dei bit non impostati fino ad incontrare un bit impostato iniziando a contare dal bit meno significativo

```
[[nodiscard]] constexpr base_integral rotate_left(const base32_t& n)const noexcept
```

Ritorna il valore ruotato a sinistra di `n` bit

```
[[nodiscard]] constexpr base_integral rotate_right(const base32_t& n)const noexcept
```

Ritorna il valore ruotato a destra di `n` bit

```
[[nodiscard]] constexpr base_integral swap_bytes()const noexcept
```

Ritorna un valore con i byte scambiati simmetricamente

```
[[nodiscard]] constexpr base_integral reverse_bits()const noexcept
```

Ritorna un valore con i bit invertiti

```
[[nodiscard]] constexpr base_integral to_be()const noexcept
```

Ritorna un valore convertito in `big endian`

```
[[nodiscard]] constexpr base_integral to_le()const noexcept
```

Ritorna un valore convertito in `little endian`

```
[[nodiscard]] constexpr option<base_integral>
try_add(const base_integral&)const noexcept

[[nodiscard]] constexpr option<base_integral>
try_sub(const base_integral&)const noexcept

[[nodiscard]] constexpr option<base_integral>
try_mul(const base_integral&)const noexcept

[[nodiscard]] constexpr option<base_integral>
try_div(const base_integral&)const noexcept

[[nodiscard]] constexpr option<base_integral>
try_rem(const base_integral&)const noexcept

[[nodiscard]] constexpr base_integral operator+(const base_integral&)const noexcept
[[nodiscard]] constexpr base_integral operator-(const base_integral&)const noexcept
[[nodiscard]] constexpr base_integral operator*(const base_integral&)const noexcept
[[nodiscard]] constexpr base_integral operator/(const base_integral&)const noexcept
[[nodiscard]] constexpr base_integral operator%(const base_integral&)const noexcept
constexpr base_integral& operator+=(const base_integral&)noexcept
constexpr base_integral& operator-=(const base_integral&)noexcept
constexpr base_integral& operator*=(const base_integral&)noexcept
constexpr base_integral& operator/=(const base_integral&)noexcept
constexpr base_integral& operator%=(const base_integral&)noexcept
```

Ritorna un valore su una delle 5 comuni operazioni aritmetiche

```
[[nodiscard]] constexpr option<base_integral>
try_neg()const noexcept
requires(is_signed())
```

Ritorna il valore negativo

```
[[nodiscard]] constexpr option<base_integral>
try_shl(const base32_t& n)const noexcept

[[nodiscard]] constexpr base_integral operator<<(const base32_t& n)const noexcept
constexpr base_integral& operator<<=(const base32_t& _n)noexcept
```

Ritorna il valore slittando `n` bit in direzione significativa

```
[[nodiscard]] constexpr option<base_integral>
try_shr(const base32_t& n)const noexcept

[[nodiscard]] constexpr base_integral operator>>(const base32_t& n)const noexcept
constexpr base_integral& operator>>=(const base32_t& _n)noexcept
```

Ritorna il valore slittando `n` bit in direzione non significativa

```
[[nodiscard]] constexpr option<base_integral>
try_abs()const noexcept

[[nodiscard]] constexpr base_integral abs()const noexcept
```

Ritorna il valore assoluto

```
[[nodiscard]] constexpr option<base_integral>
try_pow(const base32_t& n)const noexcept

[[nodiscard]] constexpr base_integral pow(const base32_t& n)const noexcept
```

Ritorna l’elevamento a potenza di `n`

```
[[nodiscard]] constexpr base_integral
saturating_add(const base_integral&)const noexcept

[[nodiscard]] constexpr base_integral
saturating_sub(const base_integral&)const noexcept

[[nodiscard]] constexpr base_integral
saturating_mul(const base_integral&)const noexcept

[[nodiscard]] constexpr base_integral
saturating_div(const base_integral&)const noexcept
```

Ritorna il valore massimo in caso di traboccamento positivo
o il valore minimo in caso di traboccamento negativo
La divisione per zero ritorna il valore massimo se il valore corrente è positivo od uguale a zero o il valore negativo se negativo

```
[[nodiscard]] contexpr int signum()const noexcept
```

Ritorna 1 se il valore è positivo, zero se il valore è zero, -1 se il valore è negativo

```
[[nodiscard]] constexpr bool positive()const noexcept
```

Ritorna vero per valore positivo

```
[[nodiscard]] constexpr bool negative()const noexcept
```

Ritorna vero per valore negativo

```
[[nodiscard]] constexpr bool zero()const noexcept
```

Ritorna vero per valore zero

```
[[nodiscard]] constexpr base_integral operator&(const base_integral& v)const noexcept
constexpr base_integral& operator&=(const base_integral& v)noexcept
```

Ritorna il `bitand` con `v`

```
[[nodiscard]] constexpr base_integral operator|(const base_integral& v)const noexcept
constexpr base_integral& operator|=(const base_integral& v)noexcept
```

Ritorna il `bitor` con `v`

```
[[nodiscard]] constexpr base_integral operator^(const base_integral& v)const noexcept
constexpr base_integral& operator^=(const base_integral& v)noexcept
```

Ritorna il `bitxor` con `v`

```
[[nodiscard]] constexpr base_integral operator~()const noexcept
```

Ritorna il `bitnot`

```
[[nodiscard]] constexpr bool operator&&(const base_integral& v)const noexcept
```

Ritorna l' `and` logico con `v`

```
[[nodiscard]] constexpr bool operator||(const base_integral& v)const noexcept
```

Ritorna l' `or` logico con `v`

```
[[nodiscard]] constexpr bool operator!()const noexcept
```

Ritorna la negazione logica

```
template <typename U>
[[nodiscard]] constexpr bool operator==(const base_integral<U>&)const noexcept
requires (is_signed() == base_integral<_U>::is_signed())

template <typename U>
[[nodiscard]] constexpr bool operator==(U)const noexcept
requires (
               concepts::numeric_integral<U>
               && is_signed() == base_integral<U>::is_signed()
)

template <typename U>
[[nodiscard]] constexpr bool operator==(const ref_wrapper<U>&)const noexcept
requires (
               concepts::numeric_integral<U>
               && is_signed() == base_integral<U>::is_signed()
)
template <typename U>
[[nodiscard]] constexpr bool operator==(const ref_wrapper<U>&)const noexcept
requires (
               concepts::base_integral<U>
               && is_signed() == U::is_signed()
)
```

Ritorna il confronto per uguaglianza

```
template <typename U>
[[nodiscard]] constexpr auto operator<=>(const base_integral<U>&)const noexcept
requires (is_signed() == base_integral<U>::is_signed())

template <typename U>
[[nodiscard]] constexpr auto operator<=>(U)const noexcept
requires (
        concepts::numeric_integral<U>
        && is_signed() == base_integral<U>::is_signed()
)

template <typename U>
[[nodiscard]] constexpr auto operator<=>(const ref_wrapper<U>&)const noexcept
requires (concepts::base_integral<_U>&& is_signed() == _U::is_signed())

template <typename U>
[[nodiscard]] constexpr auto operator<=>(const ref_wrapper<U>&)const noexcept
requires (
        concepts::numeric_integral<U>
        && is_signed() == base_integral<U>::is_signed()
)
```

Ritorna il confronto per comparatore a tre vie

integrals bounded

Uso:

Per valutare l'esito di una expressione arimetica o bit

Testata:

```
iron/integrals.h
```

Sinossi:

```
template <typename T> class base_integral_bounded final
```

Alias:

```
using value_t = T
using value_type = value_t
using uvalue_t = <private>
using unsigned_value_type = uvalue_t
using svalue_t = <private>
using signed_value_type = svalue_t
using base32_t = base_integral_bounded<std::uint32_t>
using base32_type = base32_t
typedef enum class enum_status {
        ok
        , overflow
        , illegal_operation
        , division_by_zero
        ,first=ok
        ,last= division_by_zero
} status_t;
```

Alias esterni:

```
using i8b = base_integral_bounded<std::int8_t>
using i16b = base_integral_bounded<std::int16_t>
using i32b = base_integral_bounded<std::int32_t>
using i64v = base_integral_bounded<std::int64_t>
using isizeb = base_integral_bounded<std::ptrdiff_t>
using u8b = base_integral_bounded<std::uint8_t>
using u16b = base_integral_bounded<std::uint16_t>
using u32b = base_integral_bounded<std::uint32_t>
using u64b = base_integral_bounded<std::uint64_t>
using usizeb = base_integral_bounded<std::size_t>
```

Vincoli:

```
concepts::numeric_integral<value_t>
```

Costanti statiche:

```
constexpr std::u32_t char_bits = <private>
```

Il numero di `bits` in un `char`

Metodi statici:

```
[[nodiscard]] static constexpr const char* unsafe_str(status_t)noexcept
```

Ritorna `status_t` convertito in stringa
Il valore non è mai un puntatore nullo
Il metodo non è sicuro perche ritorna un puntatore

```
[[nodiscard]] constexpr value_t min()noexcept
```

Ritorna il valore minimo di `value_t`

```
[[nodiscard]] constexpr value_t max()noexcept
```

Ritorna il valore massimo di `value_t`

```
[[nodiscard]] constexpr bool is_signed()noexcept
```

Ritorna vero se `value_t` è segnato

```
[[nodiscard]] static constexpr bool is_unsigned()noexcept
```

Ritorna vero se `value_t` non è segnato

```
[[nodiscard]] constexpr base32_t bits()noexcept
```

Ritorna il numero di `bits` in `value_t`

Costruttori:

```
constexpr base_integral_bounded(value_t v={})noexcept
constexpr base_integral_bounded(value_t v, status_t s)noexcept
constexpr base_integral(const base_integral&)noexcept
constexpr base_integral(base_integral&&)noexcept
```

Distruttore:

```
constexpr ~base_integral_bounded() = default
```

Metodi:

```
constexpr base_integral_bounded& operator=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator=(base_integral_bounded&&)noexcept
constexpr base_integral_bounded& swap(base_integral_bounded&)noexcept
```

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se `status` è `ok`

```
[[nodiscard]] constexpr status_t status()const noexcept
```

Ritorna il valore di `status`

```
[[nodiscard]] constexpr base32_t count_ones()const noexcept
```

Ritorna il conteggio dei bit impostati

```
[[nodiscard]] constexpr base32_t leading_ones()const noexcept
```

Ritorna il conteggio dei bits impostati fino ad incontrare un bit non impostato iniziando a contare dal bit piu significativo

```
[[nodiscard]] constexpr base32_t trailing_ones()const noexcept
```

Ritorna il conteggio dei bits impostati fino ad incontrare un bit non impostato iniziando a contare dal bit meno significativo

```
[[nodiscard]] constexpr base32_t count_zeros()const noexcept
```

Ritorna il conteggio dei bits non impostati

```
[[nodiscard]] constexpr base32_t leading_zeros()const noexcept
```

Ritorna il conteggio dei bit non impostati fino ad incontrare un bit impostato iniziando a contare dal bit piu significativo

```
[[nodiscard]] constexpr base32_t trailing_zeros()const noexcept
```

Ritorna il conteggio dei bit non impostati fino ad incontrare un bit impostato iniziando a contare dal bit meno significativo

```
[[nodiscard]] constexpr base_integral_bounded rotate_left(const base32_t& n)const
noexcept
```

Ritorna il valore ruotato a sinistra di `n` bit

```
[[nodiscard]] constexpr base_integral_bounded rotate_right(const base32_t& n)const
noexcept
```

Ritorna il valore ruotato a destra di `n` bit

```
[[nodiscard]] constexpr base_integral_bounded swap_bytes()const noexcept
```

Ritorna un valore con i byte scambiati simmetricamente

```
[[nodiscard]] constexpr base_integral_bounded reverse_bits()const noexcept
```

Ritorna un integral con valore dei bits invertiti

```
[[nodiscard]] constexpr base_integral_bounded to_be()const noexcept
```

Ritorna il valore convertito in `big endian`

```
[[nodiscard]] constexpr base_integral_bounded to_le()const noexcept
```

Ritorna il valore convertito in `little endian`

```
[[nodiscard]] constexpr option<value_t> value()const noexcept
```

Ritorna il valore

```
[[nodiscard]] constexpr value_t operator*()const noexcept
```

Ritorna il valore, in caso `status` non sia `ok` viene chiamato `PANIC()`

```
[[nodiscard]] constexpr value_t unchecked_value()const noexcept
```

Ritorna il valore senza verificare `status`

```
[[nodiscard]] contexpr base_integral_bounded<int> signum()const noexcept
```

Ritorna 1 se il valore è positivo, zero se il valore è zero, -1 se il valore è negativo

```
[[nodiscard]] constexpr option<bool> positive()const noexcept
```

Ritorna vero per positivo

```
[[nodiscard]] constexpr bool unchecked_positive()const noexcept
```

Ritorna vero per positivo senza verificare `status`

```
[[nodiscard]] constexpr option<bool> negative()const noexcept
```

Ritorna vero per negativo

```
[[nodiscard]] constexpr bool unchecked_negative()const noexcept
```

Ritorna vero per negativo senza verificare `status`

```
[[nodiscard]] constexpr option<bool> zero()const noexcept
```

Ritorna vero per zero

```
[[nodiscard]] constexpr bool unchecked_zero()const noexcept
```

Ritorna vero per zero senza verificare `status`

```
[[nodiscard]] constexpr base_integral_bounded abs()const noexcept
[[nodiscard]] constexpr base_integral_bounded pow(const base32_t& n)const noexcept
constexpr base_integral_bounded& operator+=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator-=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator*=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator/=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator%=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator&=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator|=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator^=(const base_integral_bounded&)noexcept
constexpr base_integral_bounded& operator<<=(const base32_t&)noexcept
constexpr base_integral_bounded& operator>>=(const base32_t&)noexcept

[[nodiscard]] constexpr base_integral_bounded operator+(
        const base_integral_bounded&)const noexcept

[[nodiscard]] constexpr base_integral_bounded operator-(
        const base_integral_bounded&)const noexcept

[[nodiscard]] constexpr base_integral_bounded operator*(
        const base_integral_bounded&)const noexcept

[[nodiscard]] constexpr base_integral_bounded operator/(
        const base_integral_bounded&)const noexcept

[[nodiscard]] constexpr base_integral_bounded operator%(
        const base_integral_bounded&)const noexcept

[[nodiscard]] constexpr base_integral_bounded operator&(
        const base_integral_bounded&)const noexcept

[[nodiscard]] constexpr base_integral_bounded operator|(
        const base_integral_bounded&)const noexcept

[[nodiscard]] constexpr base_integral_bounded operator^(
        const base_integral_bounded&)const noexcept

[[nodiscard]] constexpr base_integral_bounded operator~()const noexcept

[[nodiscard]] constexpr base_integral_bounded operator<<(const base32_t&)const
noexcept
[[nodiscard]] constexpr base_integral_bounded operator>>(const base32_t&)const
noexcept
```

Le comuni operazioni come per `base_integral` ma in caso di errore viene impostato lo stato di errore

list

Uso:

Implementazione di una lista

Testata:

```
iron/list.h
```

Sinossi:

```
template <typename T,typename A, bool ISBD>
class base_list final
```

Alias:

```
using value_t = T
using value_type = value_t
using allocator_t = A
using allocator_type = allocator_t
using size_t = std::size_t
using size_type = size_t
using diff_t = std::ptrdiff_t
using difference_type = diff_t
using ref_t = ref_wrapper<value_t>
using cref_t = ref_wrapper<const value_t>
using ptr_t = value_t*
using cptr_t = const value_t*
using reference = value_t&
using const_reference = const value_t&
using pointer = value_t*
using const_pointer = const value_t*
using iterator = <private>
using const_iterator = <private>
using seq_t = sequence<iterator>
using cseq_t = sequence<const_iterator>
using rseq_t = sequence<_riter_t>
using crseq_t = sequence<_criter_t>
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using adapter_box_t = <private>
using adapter_box_type = adapter_box_t
```

Alias esterni:

```
template <typename T, typename A=unsafe::allocator<T>>
using list = base_list<T, A,true>

template <typename T, typename A=unsafe::allocator<T>>
using fwd_list = base_list<T, A,false>
```

Costanti statiche:

```
constexpr bool is_bidirectional = ISBD
```

Vero se la lista è bidirezionale

Metodi statici:

```
[[nodiscard]] constexpr size_t max_size()noexcept
```

Ritorna il massimo numero di elementi caricabili

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
std::is_nothrow_move_constructible_v<value_t>
std::is_nothrow_swappable_v<value_t>
std::is_nothrow_destructible_v<value_t>
concepts::allocator<allocator_t>
```

Costruttori:

```
[[nodiscard]] static constexpr result<base_list, error_handler_t>
try_from(adapter_box_t&& box)noexcept
```

```
[[nodiscard]] static constexpr base_list from(adapter_box_t&& box)noexcept
```

Costruisce da `rvalue adapter_box_t`

```
[[nodiscard]] static constexpr base_list unchecked_from(allocator_t)noexcept
```

Costruisce il contenitore senza elementi impostando l'allocatore
Il metodo non è sicuro perche `allocator_t` non è sicuro

```
constexpr base_list()noexcept = default
constexpr base_list(base_list&&)noexcept
base_list(const base_list&)=delete
```

Distruttore:

```
constexpr ~base_list()=default
```

Metodi:

```
base_list& operator=(const base_list&)=delete
constexpr base_list& operator=(base_list&&)noexcept
constexpr base_list& swap(base_list&)noexcept
```

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se il contenitore ha elementi

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se il contenitore non ha elementi

```
[[nodiscard]] constexpr size_t size()const noexcept
requires (is_bidirectional)
```

Ritorna il numero di elementi

```
[[nodiscard]] constexpr size_t count()const noexcept
```

Ritorna il numero di elementi
Se `is_bidirectional` è vero ritorna `size()`
altrimenti è equivalente a `cseq().count()`

```
[[nodiscard]] constexpr allocator_t unsafe_allocator()const noexcept
```

Ritorna l'allocatore
L'operazione non è sicura in quanto `allocator_t` non è sicuro

```
[[nodiscard]] constexpr result<base_list, error_handler_t>
try_clone(size_t n = 1)const noexcept
requires(concepts::copyable_or_cloneable<value_t>)
```

```
[[nodiscard]] constexpr base_list clone(size_t n=1)const noexcept
requires(concepts::copyable_or_cloneable<value_t>)
```

Operazione di copia, `n` è il fattore di moltiplicazione per gli elementi

```
[[nodiscard]] constexpr value_t& unchecked_front()noexcept
[[nodiscard]] constexpr const value_t& unchecked_front()const noexcept
[[nodiscard]] constexpr const value_t& unchecked_cfront()const noexcept
```

Ritorna il primo elemento
L'operazione non è sicura perche la lista puo essere vuota e perché ritornare una referenza non è sicuro

```
[[nodiscard]] constexpr option<ref_t> front()noexcept
[[nodiscard]] constexpr option<cref_t> front()const noexcept
[[nodiscard]] constexpr option<cref_t> cfront()const noexcept
```

Ritorna il primo elemento

```
[[nodiscard]] constexpr value_t& unchecked_back()noexcept
requires (is_bidirectional)

[[nodiscard]] constexpr const value_t& unchecked_back()const noexcept
requires (is_bidirectional)

[[nodiscard]] constexpr const value_t& unchecked_cback()const noexcept
requires (is_bidirectional)
```

Ritorna l'ultimo elemento
L'operazione non è sicura perche la lista puo essere vuota e perché ritornare una referenza non è sicuro

```
[[nodiscard]] constexpr option<ref_t> back()noexcept
requires (is_bidirectional)

[[nodiscard]] constexpr option<cref_t> back()const noexcept
requires (is_bidirectional)

[[nodiscard]] constexpr option<cref_t> cback()const noexcept
requires (is_bidirectional)
```

Ritorna l'ultimo elemento

```
template<typename... ARGS>
[[nodiscard]] constexpr result<seq_t, error_handler_t> try_push_front(ARGS&&... args)
requires (concepts::constructible_at<value_t,ARGS&&...>)

template<typename... ARGS>
constexpr seq_t push_front(ARGS&&... args)
requires (concepts::constructible_at<value_t,ARGS&&...>)
```

Immette in testa un elemento costruito da `args`
L'operazione è transazionale
Ritorna una sequenza il cui primo valore è l'elemento inserito

```
template<typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_push_front_range(SEQ& seq)noexcept
requires(
        concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<value_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

```
template<typename SEQ>
constexpr size_t push_front_range(SEQ& seq)noexcept
requires(
        concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<value_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

Immette in testa, elementi costruiti da `seq`
Ritorna il numero di elementi inseriti e `seq` sarà consumata se non ci sono errori
L'operazione non è transazionale

```
template<typename... ARGS>
[[nodiscard]] constexpr result<seq_t, error_handler_t> try_push_back(ARGS&&... args)
requires (
        is_bidirectional
        && concepts::constructible_at<value_t,ARGS&&...>
)
```

```
template<typename... ARGS>
constexpr seq_t push_back(ARGS&&... args)
requires (
        is_bidirectional
        && concepts::constructible_at<value_t,ARGS&&...>
)
```

Immette in coda un elemento costruito da `args`
Ritorna una sequenza il cui primo valore è l'elemento inserito
L'operazione è transazionale

```
template<typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_push_back_range(SEQ& seq)noexcept
requires(
        is_bidirectional
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<value_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

```
template<typename SEQ>
constexpr size_t push_back_range(SEQ& seq)noexcept
requires(
        is_bidirectional
        && concepts::sequence<SEQ>
        && !std::is_const_v<SEQ>
        && concepts::constructible_at<value_t, std::decay_t<typename
        SEQ::value_t>&&>
        )
```

Immette in coda elementi costruiti da `seq`
Ritorna il numero di elementi inseriti e `seq` sarà consumata se non ci sono errori
L'operazione non è transazionale

```
template <typename CONTAINER_SEQ, typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_insert_range_before(const CONTAINER_SEQ& container_seq, SEQ& seq)noexcept
requires(
               concepts::sequence<CONTAINER_SEQ>
               && concepts::sequence<SEQ>
               && !std::is_const_v<SEQ>
               && concepts::constructible_at<value_t, std::decay_t<typename
       SEQ::value_t>&&>
)

template <typename CONTAINER_SEQ,typename SEQ>
constexpr size_t insert_range_before(
               const CONTAINER_SEQ& container_seq
               ,SEQ& seq
)noexcept
requires(
       concepts::sequence<CONTAINER_SEQ>
       && !std::is_const_v<SEQ>
       && concepts::sequence<SEQ>
       && concepts::constructible_at<value_t, std::decay_t<typename
       SEQ::value_t>&&>
)
```

Immette prima del valore corrente di `container_seq` elementi costruiti da `seq`
Ritorna il numero di elementi inseriti e `seq` sarà consumata se non ci sono errori
L'operazione non è transazionale

```
template <typename SEQ,typename... ARGS>
[[nodiscard]] constexpr result<seq_t, error_handler_t> try_insert_before(
       const SEQ& seq, ARGS&&... args
)noexcept
requires (
       concepts::sequence<SEQ>
       && concepts::constructible_at<value_t, ARGS&&...>
)

template <typename SEQ,typename... ARGS>
constexpr seq_t insert_before(
       const SEQ& seq, ARGS&&... args
)noexcept
requires (
       concepts::sequence<SEQ>
       && concepts::constructible_at<value_t, ARGS&&...>
)
```

Immette prima del valore corrente di `seq` un elemento costruito da `args`
`seq` deve essere servitrice dell'istanza corrente
Ritorna una sequenza il cui valore corrente è quello costruito
L'operazione è transazionale

```
template <typename CONTAINER_SEQ, typename SEQ>
[[nodiscard]] constexpr result<size_t, error_handler_t>
try_insert_range_after(const CONTAINER_SEQ& container_seq, SEQ& seq)noexcept
requires(
               concepts::sequence<CONTAINER_SEQ>
               && concepts::sequence<SEQ>
               && !std::is_const_v<SEQ>
               && concepts::constructible_at<value_t, std::decay_t<typename
       SEQ::value_t>&&>
)

template <typename CONTAINER_SEQ,typename SEQ>
constexpr size_t insert_range_after(
               const CONTAINER_SEQ& container_seq
               ,SEQ& seq
)noexcept
requires(
       concepts::sequence<CONTAINER_SEQ>
       && !std::is_const_v<SEQ>
       && concepts::sequence<SEQ>
       && concepts::constructible_at<value_t, std::decay_t<typename
       SEQ::value_t>&&>
)
```

Immette dopo il valore corrente di `container_seq` elementi costruiti da `seq`
Ritorna il numero di elementi inseriti e `seq` sarà consumata se non ci sono errori
L'operazione non è transazionale

```
template <typename SEQ,typename... ARGS>
[[nodiscard]] constexpr result<seq_t, error_handler_t> try_insert_after(
        const SEQ& seq, ARGS&&... args
)noexcept
requires (
        concepts::sequence<SEQ>
        && concepts::constructible_at<value_t, ARGS&&...>)

template <typename SEQ,typename... ARGS>
constexpr seq_t insert_after(
        const SEQ& seq, ARGS&&... args
)noexcept
requires (
        concepts::sequence<SEQ>
        && concepts::constructible_at<value_t, ARGS&&...>)
```

Immette dopo il valore corrente di `seq` un elemento costruito da `args`
`seq` deve essere referente dell'istanza corrente
Ritorna la sequenza il cui valore corrente è quello costruito
L'operazione è transazionale

```
constexpr value_t unchecked_pop_front()noexcept
```

Ritorna, rimuovendolo, l'elemento di testa
L'operazione non è sicura perché la lista potrebbe essere vuota

```
constexpr option<value_t> pop_front()noexcept
```

Ritorna, rimuovendolo, l'elemento di testa
Non ci devono essere referenti attivi

```
constexpr value_t unchecked_pop_back()noexcept
requires(is_bidirectional)
```

Ritorna, rimuovendolo, l'elemento di coda
L'operazione non è sicura perché la lista potrebbe essere vuota

```
constexpr option<value_t> pop_back()noexcept
requires(is_bidirectional)
```

Ritorna, rimuovendolo, l'elemento di coda
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr result<adapter_box_t,error_handler_t>
try_release()noexcept

[[nodiscard]] constexpr adapter_box_t release()noexcept
```

Esporta tutti gli elementi, non ci devono essere referenti attivi
L'operazione è transazionale

```
constexpr size_t erase_n(seq_t& seq, size_t n)noexcept
requires(is_bidirectional)

constexpr size_t erase_n(rseq_t& seq, size_t n)noexcept
requires(is_bidirectional)
```

Rimuove fino a `n` elementi dal valore corrente di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il numero di elementi rimossi,se `seq` è vuota ritorna zero
Non ci devono essere referenti attivi oltre a `seq`

```
constexpr size_t erase_n_after(seq_t& seq, size_t n)noexcept
```

Rimuove fino a `n` elementi dopo il valore corrente di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il numero di elementi rimossi,se `seq` è vuota ritorna zero
Non ci devono essere referenti attivi oltre a `seq`
Dopo questa operazione `seq` sarà consumata

```
constexpr size_t erase_all(seq_t& seq)noexcept
requires(is_bidirectional)

constexpr size_t erase_all(rseq_t& seq)noexcept
requires(is_bidirectional)
```

Rimuove tutti gli elementi dal valore corrente di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il numero di elementi rimossi,se `seq` è vuota ritorna zero
Non ci devono essere referenti attivi oltre a `seq`
Dopo questa operazione `seq` sarà consumata

```
constexpr size_t erase_all_after(seq_t& seq)noexcept
```

Rimuove tutti elementi dal valore dopo di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il numero di elementi rimossi,se `seq` è vuota ritorna zero
Non ci devono essere referenti attivi oltre a `seq`
Dopo questa operazione `seq` sarà consumata

```
constexpr option<value_t> erase_first(seq_t& seq)noexcept
requires(is_bidirectional)

constexpr option<value_t> erase_first(rseq_t& seq)noexcept
requires(is_bidirectional)
```

Rimuove l'elemento dal valore di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il valore rimosso se `seq` non è vuota
Non ci devono essere referenti attivi oltre a `seq`
Dopo questa operazione `seq` sarà avanzata di un elemento

```
constexpr option<value_t> erase_first_after(seq_t& seq)noexcept
```

Rimuove l'elemento successivo al valore di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il valore rimosso se esiste
Non ci devono essere referenti attivi oltre a `seq`
Dopo questa operazione `seq` avrà il valore dell' elemento successivo a quello cancellato, se esiste

```
constexpr value_t unchecked_erase_first(seq_t& seq)noexcept
requires(is_bidirectional)

constexpr value_t unchecked_erase_first(rseq_t& seq)noexcept
requires(is_bidirectional)
```

Rimuove l'elemento dal valore di `seq`
Ritorna il valore rimosso
Il metodo non è sicuro perche `seq` potrebbe essere vuota
Dopo questa operazione `seq` sarà avanzata di un elemento

```
constexpr value_t unchecked_erase_first_after(seq_t& seq)noexcept
```

Rimuove l'elemento successivo al valore di `seq`
Ritorna il valore rimosso
Il metodo non è sicuro perche `seq` potrebbe essere vuota
Dopo questa operazione `seq` si riferirà valore dell' elemento successivo a quello cancellato,

```
template <typename F>
constexpr size_t erase_if(F f)noexcept
requires(concepts::bool_predicate<F, value_t>)
```

Rimuove gli elementi che soddisfano `f`
Ritorna il numero di elementi rimossi
Non ci devono essere referenti attivi

```
template <typename F>
constexpr size_t erase_if(seq_t& seq,F f)noexcept
requires(is_bidirectional && concepts::bool_predicate<F, value_t>)

template <typename F>
constexpr size_t erase_if(rseq_t& seq, F f)noexcept
requires(is_bidirectional && concepts::bool_predicate<F, value_t>)
```

Rimuove gli elementi che soddisfano `f` a partire dal valore corrente di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il numero di elementi rimossi,se `seq` è vuota ritorna zero.
Non ci devono essere referenti attivi oltre a `seq`
Dopo questa operazione `seq` sarà consumata

```
template <typename F>
constexpr size_t erase_if_after(seq_t& seq, F f)noexcept
requires(concepts::bool_predicate<F, value_t>)
```

Rimuove tutti gli elementi che soddisfano `f` a partire dal valore successivo di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il numero di elementi rimossi, se `seq` è vuota ritorna zero
Non ci devono essere referenti attivi
Dopo questa operazione `seq` sarà consumata

```
constexpr size_t erase(const value_t& v)noexcept
requires (concepts::nothrow_equality_comparable<value_t>)
```

Rimuove tutti gli elementi che abbiano il valore `v`
Ritorna il numero di elementi rimossi
Non ci devono essere referenti attivi

```
constexpr size_t erase(seq_t& seq, const value_t& v)noexcept
requires (is_bidirectional && concepts::nothrow_equality_comparable<value_t>)

constexpr size_t erase(rseq_t& seq, const value_t& v)noexcept
requires (is_bidirectional && concepts::nothrow_equality_comparable<value_t>)
```

Rimuove gli elementi che abbiano valore `v` a partire dal valore corrente di `seq`
`seq` deve essere referente dell'istanza corrente o vuota
Ritorna il numero di elementi rimossi,se `seq` è vuota ritorna zero
Non ci devono essere referenti attivi oltre a `seq`
Dopo questa operazione `seq` sarà consumata

```
constexpr size_t erase_after(seq_t& seq, const value_t& v)noexcept
requires (concepts::nothrow_equality_comparable<value_t>)
```

Rimuove gli elementi che abbiano valore `v` a partire dal valore successivo di `seq`
Ritorna il numero di elementi rimossi
Non ci devono essere referenti attivi oltre a `seq`
Dopo questa operazione `seq` sarà consumata

```
constexpr size_t unique()noexcept
requires(concepts::nothrow_equality_comparable<value_t>)
```

Elimina i valori duplicati
Il contenitore deve avere i valori degli elementi raggruppati per uguaglianza.
Ritorna il valore di `count()` dopo l'operazione.
Non ci devono essere referenti attivi.

```
constexpr base_list& reverse()noexcept
```

Inverte gli elementi

```
constexpr base_list& reset()noexcept
```

Rimuove tutti gli elementi
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr error_handler_t try_resize(size_t n)noexcept
requires(std::is_default_constructible_v<value_t>)

base_list& resize(size_t n)noexcept
requires(std::is_default_constructible_v<value_t>)
```

Porta il numero di elementi effettivi a `n` con l'eccedenza a valore predefinito.
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t try_resize(size_t n,const value_t& v)noexcept
requires(concepts::copyable_or_cloneable<value_t>)

base_list& resize(size_t n,const value_t& v)noexcept
requires(concepts::copyable_or_cloneable<value_t>)
```

Porta il numero di elementi effettivi a `n` con l'eccedenza a valore `v`.
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr seq_t seq()noexcept
[[nodiscard]] constexpr cseq_t seq()const noexcept
[[nodiscard]] constexpr cseq_t cseq()noexcept
```

Ritorna una sequenza di valori

```
[[nodiscard]] constexpr rseq_t rseq()noexcept
requires (is_bidirectional)

[[nodiscard]] constexpr crseq_t rseq()const noexcept
requires (is_bidirectional)

[[nodiscard]] constexpr crseq_t crseq()noexcept
requires (is_bidirectional)
```

Ritorna una sequenza di valori invertiti

```
template <typename A1,bool ISBD1>
[[nodiscard]] constexpr bool operator==(const base_list<value_t,A1,ISBD1>& r)const
noexcept
requires(concepts::nothrow_equality_comparable<value_t>)
```

Ritorna vero se i due contenitori hanno lo stesso numero di elementi a valore uguale

memory::condition_variable

Uso:

primitiva di sincronizzazione utilizzata con un `mutex` per bloccare uno o più `thread` finché un altro `thread` non modifica una variabile condivisa (la condizione) e avviene la notifica.

Testata:

```
iron/memory/condition_variable.h
```

Sinossi:

```
template <typename _MUTEX>
class base_condition_variable final
```

Alias:

```
using mutex_t=_MUTEX
using error_handler_t=error_handler
using lock_t = std::unique_lock<mutex_t>
```

Alias esterni:

```
using condition_variable = base_condition_variable<mutex>
```

Metodi statici:

```
template<typename F>
[[nodiscard]] consteval bool is_bool_predicate()noexcept
```

Costruttori:

```
base_condition_variable()noexcept
base_condition_variable(base_condition_variable&&)=delete
base_condition_variable(const base_condition_variable&)=delete
```

Distruttore:

```
~base_condition_variable()
```

Metodi:

```
base_condition_variable& operator=(const base_condition_variable&)=delete
base_condition_variable& operator=(base_condition_variable&&)=delete
```

`notify_one()noexcept`

Il produttore notifica ad un consumatore che i dati sono disponibili

`notify_all()noexcept`

Il produttore notifica a tutti i consumatori che i dati sono disponibili

`[[nodiscard]] error_handler_t try_wait(lock_t&)noexcept`

`base_condition_variable& wait(lock_t&)noexcept`

blocca il thread corrente finché la variabile di condizione non viene risvegliata

```
template<typename PREDICATE>
[[nodiscard]] error_handler_t try_wait(lock_t&,PREDICATE pred)noexcept
requires(is_bool_predicate<PREDICATE>())
```

```
template<typename PREDICATE>
base_condition_variable& wait(lock_t&,PREDICATE pred )noexcept
requires(is_bool_predicate<PREDICATE>())
```

Blocca il thread corrente finché la variabile di condizione non viene risvegliata e `pred` ritorna vero

```
[[nodiscard]] const auto native_handle()const noexcept
```

Ritorna l'`handle` interno

memory::lock_guard

Uso:

un `wrapper` per una classe di `lock` usando il paradigma `RAII`

Testata:

```
iron/memory/lock_guard.h
```

Sinossi:

```
template <typename T>
class lock final
```

Alias:

```
using value_t = T
using value_type = value_t
```

Vincoli:

```
concepts::lock<value_t>
```

Costruttore:

```
explicit lock_guard(value_t&)noexcept
lock_guard(const lock_guard&)=delete
lock_guard(lock_guard&&)=delete
```

Costruttori esterni:

```
template <typename T>
[[nodiscard]]  lock_guard<T> make_lock_guard(T&)noexcept
```

Distruttore:

```
~lock_guard()
```

Metodi:

```
lock_guard& operator=(const lock_guard&)=delete
lock_guard& operator=(lock_guard&&)=delete
```

memory::mutex

Uso:

sincronizzare l'accesso a dati condivisi in modo bloccante

Testata:

```
iron/memory/mutex.h
```

Sinossi:

```
template <bool RECURSIVE>
class base_mutex final
```

Alias:

```
using error_handler_t=error_handler
```

Alias esterni:

```
using mutex = base_mutex<false>
using recursive_mutex = base_mutex<true>
```

Costanti statiche:

```
constexpr bool recursive = RECURSIVE
```

Costruttore:

```
base_mutex()noexcept
base_mutex(const base_mutex&)=delete
base_mutex(base_mutex&&) = delete
```

Distruttore:

```
~base_mutex()
```

Metodi:

```
base_mutex& operator=(const base_mutex&)=delete
base_mutex& operator=(base_mutex&&) = delete
```

```
[[nodiscard]] error_handler_t try_acquire()noexcept
base_mutex& acquire()noexcept
void lock()noexcept
```

Acquisisce il `lock`

```
[[nodiscard]] result<bool, error_handler_t> try_can_acquired()noexcept
[[nodiscard]] bool can_acquired()noexcept
```

Tenta di acquisire il `lock`, ritorna vero se puo essere acquisito

```
[[nodiscard]] error_handler_t try_release()noexcept
base_mutex& release()noexcept
void unlock()noexcept
```

Rilascia il `lock`

```
[[nodiscard]] const void* native_handle()const noexcept
```

Ritorna l'`handle` interno

memory::spin_lock

Uso:

sincronizzare l'accesso a dati condivisi in modo non bloccante

Testata:

```
iron/memory/spin_lock.h
```

Sinossi:

```
template <bool RECURSIVE>
class base_spin_lock final
```

Alias:

```
using error_handler_t=error_handler
```

Alias esterni:

```
using spin_lock = base_spin_lock<false>
using recursive_spin_lock = base_spin_lock<true>
```

Costanti statiche:

```
constexpr bool recursive = RECURSIVE
```

Vincoli:

```
!recursive || noexcept(std::this_thread::get_id())
```

Costruttore:

```
base_spin_lock()noexcept
base_spin_lock(const base_spin_lock&)=delete
base_spin_lock(base_spin_lock&&) = delete
```

Distruttore:

```
~base_spin_lock()
```

Metodi:

```
base_spin_lock& operator=(const base_spin_lock&)=delete
base_spin_lock& operator=(base_spin_lock&&) = delete
```

```
base_spin_lock& acquire()noexcept
void lock()noexcept
```

Acquisisce il `lock`

```
base_spin_lock& release()noexcept
void unlock()noexcept
```

Rilascia il `lock`

option

Uso:

memorizzare un valore in modo opzionale

Testata:

`iron/option.h`

Sinossi:

```
template <typename T>
class option final
```

Alias:

```
using value_t = T
using value_type = value_t
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using ref_t = <private>&
using reference = ref_t
using cref_t = const <private>&
using const_reference = cref_t
using safe_ref_t = ref_wrapper<<private>>
using safe_reference = safe_ref_t
using safe_cref_t = ref_wrapper<const <private>>
using safe_const_reference = safe_cref_t
```

Costanti statiche:

```
[[nodiscard]] constexpr size_t char_max_size()noexcept
```

Il massimo numero di `char32_t` per un `builtin array` come ingresso per quei metodi che lo richiedono

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
std::is_nothrow_move_constructible_v<value_t>
std::is_nothrow_swappable_v<value_t>
std::is_nothrow_destructible_v<value_t>
```

Costruttori:

```
constexpr option()noexcept

constexpr option(option<void*>&&)noexcept
requires(!std::is_same_v<value_t,void*>)
```

```
[[nodiscard]] static constexpr option none()noexcept
```

Costruisce senza elemento attivo

```
constexpr option(option&&)noexcept
constexpr option(const option&)noexcept
requires (std::is_nothrow_copy_constructible_v<value_t>)

template <typename... ARGS>
[[nodiscard]] static constexpr option some(ARGS&&... args)noexcept
requires (concepts::nothrow_constructible_at<value_t,ARGS&&...>)
```

Costruisce e ritorna un `option` contenente una istanza di `value_t` costruita da `args`

Costruttori esterni:

```
template <typename T=void*>
[[nodiscard]]  constexpr option<T> none()noexcept
```

Costruisce e ritorna un `option` senza elemento attivo

```
template <typename T>
[[nodiscard]]  constexpr option<std::decay_t<T>> some(T&& v)noexcept
requires (concepts::nothrow_constructible_at<std::decay_t<T>,T&&>)
```

Costruisce e ritorna un `option` contenente una istanza di `T` costruita da `v`

```
template <typename T, typename... ARGS>
[[nodiscard]]  constexpr option<T>  some_emplace(ARGS&&... args)noexcept
requires (concepts::nothrow_constructible_at<T,ARGS&&...>)
```

Costruisce e ritorna un `option` contenente una istanza di `T` costruita da `args`

Distruttore:

```
constexpr ~option()
```

Metodi:

```
constexpr option& operator=(option&&)noexcept

option& operator=(const option&)noexcept
requires (std::is_nothrow_copy_constructible_v<value_t>)

constexpr option& swap(option&)noexcept
```

```
[[nodiscard]] constexpr result<option, error_handler_t>
try_clone()const noexcept
requires (concepts::copyable_or_cloneable<value_t>)

[[nodiscard]] constexpr option clone()const noexcept
requires (concepts::copyable_or_cloneable<value_t>)
```

Ritorna una copia

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
[[nodiscard]] constexpr bool is_some()const noexcept
```

Ritorna vero se l'elemento interno è impostato

```
[[nodiscard]] constexpr bool is_none()const noexcept
```

Ritorna vero se l'elemento interno non è impostato

```
template <typename F>
[[nodiscard]] constexpr bool is_some_and(F f)const noexcept
requires concepts::bool_predicate<F,value_t>
```

Ritorna vero se l'elemento interno è impostato e il predicato `f` ritorna vero

```
constexpr void reset()noexcept
```

Distrugge l'elemento interno se impostato
Dopo questa operazione `is_some()` ritorna falso
Non ci devono essere referenti attivi

```
[[nodiscard]] value_t release()noexcept
```

Rilascia e ritorna l'elemento interno se impostato altrimenti viene chiamata `PANIC()`
Dopo questa operazione `is_some()` ritorna falso
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr value_t release_or()noexcept
requires (std::is_nothrow_default_constructible_v<value_t>)
```

Rilascia e ritorna l'elemento interno se impostato altrimenti ritorna una istanza con valore predefinito
Dopo questa operazione `is_some()` ritorna falso
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr safe_cref_t unwrap()const noexcept
[[nodiscard]] constexpr safe_cref_t cunwrap()const noexcept
[[nodiscard]] constexpr safe_cref_t operator*()const noexcept
[[nodiscard]] constexpr safe_ref_t unwrap()noexcept
[[nodiscard]] constexpr safe_ref_t operator*()noexcept
```

Ritorna l'istanza interna se impostata altrimenti viene chiamata `PANIC()`

```
template <typename M,std::size_t N>
[[nodiscard]] constexpr safe_cref_t expect(const M(&v)[N])const noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)

template <typename M,std::size_t N>
[[nodiscard]] constexpr safe_cref_t cexpect(const M(&v)[N])const noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)

template <typename M,std::size_t N>
[[nodiscard]] constexpr safe_ref_t expect(const M(&v)[N])noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)
```

Come `unwrap()` ma in caso l'istanza interna non sia impostata chiama `PANIC()` con il messaggio `v`

```
[[nodiscard]] constexpr cref_t unchecked_unwrap()const noexcept
[[nodiscard]] constexpr cref_t unchecked_cunwrap()const noexcept
[[nodiscard]] constexpr ref_t unchecked_unwrap()noexcept
```

Ritorna un riferimento dell'istanza interna che deve essere impostata altrimenti il risultato è indefinito
La chiamata non è sicura perché ritorna un riferimento e/o l'istanza interna potrebbe non essere impostata

```
[[nodiscard]] constexpr value_t unwrap_or(value_t v)const noexcept
requires(std::is_nothrow_copy_constructible_v<value_t>)
```

Ritorna il valore dell'istanza interna se impostata
altrimenti ritorna `v`

```
[[nodiscard]] constexpr const value_t& unchecked_unwrap_or(const value_t& v)const
noexcept
[[nodiscard]] constexpr const value_t& unchecked_cunwrap_or(const value_t& v)const
noexcept
[[nodiscard]] constexpr value_t& unchecked_unwrap_or(value_t& v)noexcept
```

Ritorna un riferimento dell'istanza interna se impostata
altrimenti ritorna `v`
Questi tre metodi non sono sicuri perché ritornano un riferimento

```
[[nodiscard]] constexpr value_t unwrap_or_default()const noexcept
requires (
               std::is_nothrow_default_constructible_v<value_t>
               && std::is_nothrow_copy_constructible_v<value_t>
        )
```

Ritorna il valore dell'istanza interna se impostata
altrimenti ritorna una istanza predefinita

ref_wrapper

Uso:

memorizzare un riferimento

Testata:

```
iron/ref_wrapper.h
```

Sinossi:

```
template <typename T> class ref_wrapper final
```

Alias:

```
using value_t = T
using value_type = value_t
```

Vincoli:

```
std::is_object_v<value_t>
!concepts::ref_wrapper<value_t>
```

Costruttori:

```
template <typename MASTER>
[[nodiscard]] static constexpr ref_wrapper from(
        const MASTER& master
        ,value_t& v
)noexcept
#if IRON_SECURITY_CHECKER
requires(concepts::has_security_checker<MASTER>)
#endif
```

Ritorna un `wrapper` costruito da `value_t`
`Master` serve unicamente al `security_checker` per tenere traccia della vita dell'istanza della classe principale, la proprietaria degli elementi.
`MASTER` può essere anche una classe referente di una classe principale

```
constexpr ref_wrapper(const ref_wrapper&)noexcept
constexpr ref_wrapper(ref_wrapper&&)noexcept
```

Costruttori esterni:

```
template <typename MASTER,typename T>
[[nodiscard]] inline constexpr auto make_ref(const MASTER&,T&)noexcept

template <typename T>
[[nodiscard]] inline constexpr auto make_ref(T&)noexcept
requires(concepts::ref_wrapper<T>)

template <typename MASTER, typename T>
[[nodiscard]] inline constexpr auto make_cref(const MASTER&,const T&)noexcept

template <typename T>
[[nodiscard]] inline constexpr auto make_cref(const T&)noexcept
requires(concepts::ref_wrapper<T>)
```

Distruttore:

```
constexpr ~ref_wrapper() = default
```

Metodi:

```
constexpr ref_wrapper& operator=(const ref_wrapper&)noexcept
constexpr ref_wrapper& operator=(ref_wrapper&&)noexcept
constexpr ref_wrapper& swap(ref_wrapper&)noexcept


[[nodiscard]] constexpr value_t& value()const noexcept
[[nodiscard]] constexpr value_t& operator*()const noexcept
[[nodiscard]] constexpr operator value_t& ()const noexcept
```

Ritorna il valore dell'elemento

```
[[nodiscard]] constexpr value_t* operator->()const noexcept
requires(std::is_class_v<value_t>)
```

Ritorna il puntatore all'istanza interna
Da usare solo con `value_t` classe o struttura per accedere ad un valore od un metodo

result

Uso:

memorizzare un valore di tipo `T` o `E`

Testata:

```
iron/result.h
```

Sinossi:

```
template <typename T,typename E>
class result final
```

Alias:

```
using value_t = T
using value_type = value_t
using err_t=E
using err_type=err_t
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using ref_value_t = <private>&
using value_reference = ref_value_t
using cref_value_t = const <private>
using const_value_reference = cref_value_t
using safe_ref_value_t = ref_wrapper<<private>>
using safe_value_reference = safe_ref_value_t
using safe_cref_value_t = ref_wrapper<const <private>>
using safe_const_value_reference = safe_cref_value_t
using ref_err_t = <private>&
using err_reference = ref_err_t
using cref_err_t = const <private>&
using const_err_reference = cref_err_t
using safe_ref_err_t = ref_wrapper<<private>>
using safe_err_reference = safe_ref_err_t
using safe_cref_err_t = ref_wrapper<const <private>>
using safe_const_err_reference = safe_cref_err_t
```

Costanti statiche:

```
[[nodiscard]] constexpr size_t char_max_size()noexcept
```

Il massimo numero di `char32_t` per un `array` come ingresso per quei metodi che lo richiedono

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
std::is_nothrow_move_constructible_v<value_t>
std::is_nothrow_swappable_v<value_t>
std::is_nothrow_destructible_v<value_t>
!std::is_reference_v<err_t>
!std::is_array_v<err_t>
!std::is_const_v<err_t>
std::is_nothrow_move_constructible_v<err_t>
std::is_nothrow_swappable_v<err_t>
std::is_nothrow_destructible_v<err_t>
```

Costruttori:

```
constexpr result()noexcept
requires (std::is_nothrow_default_constructible_v<value_t>)


constexpr result(result&&)noexcept
constexpr result(const result&) = delete

constexpr result(result<value_t, void*>&&)noexcept
constexpr result(result<void*,err_t>&&)noexcept


[[nodiscard]] static constexpr result ok(value_t&&)noexcept

[[nodiscard]] static constexpr result ok(const value_t&)noexcept
requires (std::is_nothrow_copy_constructible_v<value_t>)
```

```
[[nodiscard]] static constexpr result err(err_t&&)noexcept


[[nodiscard]] static constexpr result err(const err_t&)noexcept
requires (std::is_nothrow_copy_constructible_v<err_t>)
```

Costruttori esterni:

```
template <typename T, typename E=void*>
[[nodiscard]]  constexpr auto ok(T&&)noexcept

template <typename E, typename T=void*>
[[nodiscard]]  constexpr auto err(E&&)noexcept
```

Distruttore:

```
constexpr ~result()
```

Metodi:

```
constexpr result& operator=(result&&)noexcept
result& operator=(const result&)=delete
constexpr result& swap(result&)noexcept

[[nodiscard]] constexpr bool is_ok()const noexcept
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se l' istanza `value_t` è impostata

```
[[nodiscard]] constexpr bool is_err()const noexcept {
```

Ritorna vero se l'istanza `err_t` è impostata

```
[[nodiscard]] constexpr safe_cref_value_t unwrap()const noexcept
[[nodiscard]] constexpr safe_cref_value_t cunwrap()const noexcept
[[nodiscard]] constexpr safe_ref_value_t unwrap()noexcept
```

Ritorna l'istanza interna di `value_t` se impostata
altrimenti viene chiamata `PANIC()`

```
template <typename M,std::size_t N>
[[nodiscard]] constexpr safe_cref_value_t expect(const M(&v)[N])const noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)

template <typename M,std::size_t N>
[[nodiscard]] constexpr safe_cref_value_t cexpect(const M(&v)[N])const noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)


template <typename M,std::size_t N>
[[nodiscard]] constexpr safe_ref_value_t expect(const M(&v)[N])noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)
```

Come `unwrap()` ma in caso l'istanza interna non sia impostata
chiama `PANIC()` con il messaggio `v`.

```
[[nodiscard]] constexpr cref_value_t unchecked_unwrap()const noexcept
[[nodiscard]] constexpr cref_value_t unchecked_cunwrap()const noexcept
[[nodiscard]] constexpr ref_value_t  unchecked_unwrap()noexcept
```

Ritorna un riferimento dell'istanza interna di `value_t` che deve essere impostata altrimenti il risultato è indefinito
La chiamata non è sicura perché ritorna un riferimento e/o l'istanza interna potrebbe non essere impostata

```
[[nodiscard]] constexpr safe_cref_err_t unwrap_err()const noexcept
[[nodiscard]] constexpr safe_cref_err_t cunwrap_err()const noexcept
[[nodiscard]] constexpr safe_ref_err_t unwrap_err()noexcept
```

Ritorna l'istanza interna di `err_t` se impostata
altrimenti viene chiamata `PANIC()`

```
template <typename M,std::size_t N>
[[nodiscard]] constexpr auto expect_err(const M(&v)[N])const noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)

template <typename M,std::size_t N>
[[nodiscard]] constexpr auto cexpect_err(const M(&v)[N])const noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)

template <typename M,std::size_t N>
[[nodiscard]] constexpr auto expect_err(const M(&v)[N])noexcept
requires (
        (std::is_same_v<M, char> || std::is_same_v<M, char32_t>)
        && N < char_max_size()
)
```

Come `unwrap_err()` ma in caso l'istanza interna di `err_t` non sia impostata chiama `PANIC()` con il messaggio `v`

```
[[nodiscard]] constexpr cref_err_t unchecked_unwrap_err()const noexcept
[[nodiscard]] constexpr cref_err_t unchecked_cunwrap_err()const noexcept
[[nodiscard]] constexpr ref_err_t unchecked_unwrap_err()noexcept
```

Ritorna un riferimento dell'istanza interna di `err_t` che deve essere impostata
La chiamata non è sicura perché ritorna un riferimento e/o l'istanza interna potrebbe non essere impostata

```
[[nodiscard]] constexpr value_t unwrap_or_default()const noexcept
requires (
                std::is_nothrow_default_constructible_v<value_t>
                && std::is_nothrow_copy_constructible_v<value_t>
        )
```

Ritorna il valore dell'istanza interna di `value_t` se è impostata
altrimenti ritorna una istanza predefinita di `value_t`

```
[[nodiscard]] constexpr err_t unwrap_err_or_default()const noexcept
requires (
                std::is_nothrow_default_constructible_v<err_t>
                && std::is_nothrow_copy_constructible_v<err_t>
        )
```

Ritorna il valore dell'istanza interna di `err_t` è impostata
altrimenti ritorna una istanza predefinita di `err_t`

sequence

Uso:

memorizzare riferimenti ad una sequenza di elementi

Testata:

```
iron/sequence.h
```

Sinossi:

```
template <typename ITER> class sequence
```

Alias:

```
using iter_type=ITER
using iter_t = iter_type
using value_t = <private>
using value_type = value_t
using ref_t = <private>
using reference = ref_t
using difference_type = std::ptrdiff_t
using difference_t = difference_type
using size_type = std::size_t
using size_t = size_type
using error_handler_t = error_handler
using error_handler_type = error_handler_t
```

Costanti statiche:

```
constexpr bool is_forward_iterator=<private>
```

Ritorna vero se `iter_t` è un iteratore in avanti di passo uno

```
constexpr bool is_bidirectional_iterator=<private>
```

Ritorna vero se `iter_t` è un iteratore in avanti ed indietro di passo uno

```
constexpr bool is_random_iterator=<private>
```

Ritorna vero se `iter_t` è un iteratore in avanti ed indietro di passo `N`

```
constexpr bool is_contiguous_iterator=<private>
```

Ritorna vero se `iter_t` è un iteratore in avanti ed indietro di passo `N`
con i riferimenti interni ad elementi memorizzati in memoria contigua

```
constexpr bool is_ref=<private>
```

Ritorna vero se `iter_t::operator*()` ritorna un riferimento

```
constexpr bool is_const=<private>
```

Ritorna vero se `iter_t::operator*()` non ritorna un riferimento o un riferimento a valore immutabile

Metodi statici:

```
template<typename F>
[[nodiscard]] consteval bool is_bool_predicate()noexcept
```

Ritorna vero se `F` è un predicato nella forma `bool F(const value_t&)noexcept`

```
template <typename F,typename OTHVALUE>
[[nodiscard]] consteval bool is_cmp_predicate()noexcept
```

Ritorna vero se `F` è un predicato nella forma

```
bool F(const value_t&,const OTHVALUE&)noexcept
```

```
template<typename F>
[[nodiscard]] consteval bool is_transform_predicate()noexcept
```

Ritorna vero se `F` è un predicato nella forma

```
result<bool,error_handler_t> F(value_t&)noexcept
```

Vincoli:

```
is_forward_iterator<iter_t>
```

Costruttori:

```
constexpr sequence()noexcept=default
constexpr sequence(sequence&&)noexcept
constexpr sequence(const sequence&)noexcept
```

```
template <typename MASTER>
[[nodiscard]] static constexpr sequence unchecked_from(
        const MASTER& master
        ,const iter_type& b
        ,const iter_type& e
)noexcept
#if IRON_SECURITY_CHECKER
requires(concepts::has_security_checker<MASTER>)
#endif
```

Ritorna una sequenza costruita da una coppia di iteratori
`Master` serve unicamente al `security_checker` per tenere traccia della vita dell'istanza della classe principale, la proprietaria degli elementi.
`MASTER` può essere anche una classe referente di una classe principale
Il metodo non è sicuro ed è da usare solo per quelle classi contenitori di sequenze di elementi
Quando la sequenza è consumata non è piu una istanza referente.

Distruttore:

```
constexpr ~sequence()=default
```

Metodi:

```
constexpr sequence& operator=(const sequence&)noexcept
constexpr sequence& operator=(sequence&&)noexcept
constexpr sequence& swap(sequence&)noexcept
```

```
[[nodiscard]] constexpr sequence clone()const noexcept
```

Ritorna una copia della sequenza

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se la sequenza non contiene nessun riferimento

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero sequenza la sequenza contiene almeno un riferimento

```
[[nodiscard]] constexpr option<size_t> size()const noexcept
```

Ritorna il numero di elementi se `iter_t` è un `contiguos_iterator`

```
[[nodiscard]] constexpr iter_t begin()const noexcept
[[nodiscard]] constexpr iter_t& begin()noexcept
```

Ritorna l'iteratore all' inizio della sequenza
Questo metodo non è sicuro ma esiste solo per poter utilizzare la sequenza in un ciclo `for` (non consigliato) oppure in codice di basso livello.
Notare che un ciclo `for` non consuma la sequenza perché effettua prima una copia dell'iteratore, il motivo per cui usare un `for` in questo caso non è consigliato
Meglio usare `while(seq) { auto r=*seq++;...}`

```
[[nodiscard]] constexpr iter_t end()const noexcept
```

Ritorna l'iteratore oltre l'ultimo elemento della sequenza
Questo metodo non è sicuro ma esiste solo per poter utilizzare la sequenza in un ciclo `for` (non consigliato) oppure in codice di basso livello.

```
constexpr sequence& operator++()noexcept
constexpr sequence operator++(int)noexcept
```

avanza la sequenza di un passo,prefisso e suffisso

```
constexpr sequence& operator+=(size_t n)noexcept
constexpr sequence& skip(size_t n=1)noexcept
```

Avanza la sequenza di n

```
[[nodiscard]] constexpr value_t& unsafe_peek()const noexcept
requires(is_ref)
```

Ritorna una referenza all'elemento corrente
Il metodo non è sicuro perche non controlla se la sequenza è consumata e perché ritorna una referenza

```
[[nodiscard]] constexpr value_t unsafe_peek()const noexcept
requires(!is_ref)
```

Ritorna l'elemento corrente
Il metodo non è sicuro perche non controlla se la sequenza è consumata

```
[[nodiscard]] constexpr option<ref_t> peek()const noexcept
```

Ritorna l'elemento corrente

```
[[nodiscard]] constexpr ref_t operator*()const noexcept
```

Ritorna l'elemento corrente, se la sequenza è consumata viene chiamata `PANIC()`

```
[[nodiscard]] constexpr auto& unchecked_next()noexcept
requires(is_ref)
```

Ritorna l'elemento corrente e poi avanza la sequenza di un passo
Il metodo non è sicuro perche non controlla se la sequenza è consumata
e perché ritorna una referenza

```
[[nodiscard]] constexpr auto unchecked_next()noexcept
requires(!is_ref)
```

Ritorna l'elemento corrente e poi avanza la sequenza di un passo
Il metodo non è sicuro perche non controlla se la sequenza è consumata

```
[[nodiscard]] constexpr option<ref_t> next()noexcept
```

Ritorna l'elemento corrente e poi avanza la sequenza di un passo
se la sequenza non è consumata

```
[[nodiscard]] constexpr option<ref_t> last()noexcept
```

Consuma la sequenza e ritorna l'ultimo elemento della sequenza

```
[[nodiscard]] constexpr size_t count()noexcept
```

Consuma la sequenza e ritorna il numero di elementi effettivi.

```
template <typename F>
[[nodiscard]] constexpr size_t count_if(F predicate)noexcept
requires(is_bool_predicate<F>())
```

Consuma la sequenza e ritorna il numero di elementi che soddisfano
il predicato `predicate`

```
[[nodiscard]] constexpr auto reverse()noexcept
requires(is_bidirectional_iterator)
```

Consuma la sequenza e ritorna una sequenza con gli elementi invertiti
di posizione

```
template <typename F>
[[nodiscard]] constexpr option<ref_t> for_each_condition(F predicate)noexcept
requires(is_cmp_predicate<F,value_t>())
```

Consuma la sequenza
Ritorna l'ultimo elemento che soddisfa il predicato

```
template <typename F>
constexpr sequence& all(F predicate)noexcept
requires(is_bool_predicate<F>())
```

Avanza la sequenza fino a che `predicate` sia soddisfatta

```
template <typename F>
constexpr sequence& any(F predicate)noexcept
requires(is_bool_predicate<F>())
```

Avanza la sequenza fino a che `predicate` non sia soddisfatta

```
template <typename ENUMERATE>
[[nodiscard]] constexpr auto enumerate(ENUMERATE start)noexcept
requires concepts::numeric_integral<ENUMERATE>
```

Consuma la sequenza e ritorna un' altra sequenza che ha come elementi un `std::pair<ENUMERATE,reference>` con il primo valore di `ENUMERATE` uguale a `start`
In caso di traboccamento ricomincia a contare da `start`

```
[[nodiscard]] constexpr auto step_by(size_t n)noexcept
```

Consuma la sequenza e ritorna un' altra sequenza che avanza internamente per `n` ad ogni passo
`n` deve essere maggiore di zero altrimenti viene chiamata `PANIC()`

```
[[nodiscard]] constexpr auto slice(
        size_t from
        ,size_t n=std::numeric_limits<size_t>::max()
)noexcept
```

Consuma la sequenza e ritorna un' altra sequenza avanzata di `from` con al massimo `n` elementi

```
template <typename F>
[[nodiscard]] constexpr auto filter(F predicate)noexcept
requires(is_bool_predicate<F>())
```

Consuma la sequenza e ritorna un' altra sequenza con solo elementi che soddisfano `predicate`

```
template <typename F>
constexpr sequence& for_each(F predicate)noexcept
requires(is_bool_predicate<F>())
```

Avanza la sequenza fino a che `predicate` non sia soddisfatto

```
template <typename F>
constexpr sequence& find(F predicate, size_t n=0)noexcept
requires (is_bool_predicate<F>())
```

Avanza la sequenza fino a che l'ennesimo elemento soddisfa il predicato
Il conteggio degli elementi inizia da zero

```
constexpr sequence& find(const value_t& v, size_t n=0)noexcept
requires(iron::concepts::nothrow_equality_comparable<value_t>)
```

Avanza la sequenza fino a che l'ennesimo elemento sia uguale a `v`
Il conteggio degli elementi inizia da zero

```
template <typename F>
[[nodiscard]] constexpr option<size_t> position(F predicate, size_t n=0)noexcept
requires (is_bool_predicate<F>())
```

Avanza la sequenza e ritorna la posizione dell' ennesimo elemento che soddisfa il predicato
Il conteggio degli elementi inizia da zero

```
constexpr auto take(size_t n)noexcept
```

Consuma la sequenza e ritorna un'altra sequenza che contiene al massimo `n` elementi

```
template <typename F>
constexpr sequence& skip_if(F predicate)noexcept
requires (is_bool_predicate<F>())
```

Avanza di un passo la sequenza se il predicato è soddisfatto

```
template <typename F>
constexpr sequence& take_if(F predicate)noexcept
requires (is_bool_predicate<F>())
```

Avanza di un passo la sequenza se il predicato non è soddisfatto

```
template <typename F>
[[nodiscard]] constexpr error_handler_t try_transform(F predicate)noexcept
requires(is_transform_predicate<F>() && !is_const)

template <typename F>
constexpr sequence& transform(F predicate)noexcept
requires(is_transform_predicate<F>() && !is_const)
```

Consuma la sequenza effettuando una modifica dei valori usando `predicate`

```
constexpr sequence& reset()noexcept
```

Consuma la sequenza senza avanzamenti

```
template <typename SEQ>
[[nodiscard]] constexpr bool eq(SEQ&)noexcept
requires(
	concepts::sequence<SEQ>
	&& !std::is_const_v<SEQ>
	&& concepts::nothrow_equality_comparable_with<value_t, typename
	SEQ::value_t>
)
```

Consuma le sequenze e ritorna vero se il numero di elementi è uguale
sia per numero che per valore

```
template <typename SEQ>
[[nodiscard]] constexpr bool ne(SEQ&)noexcept
requires(
	concepts::sequence<SEQ>
	&& !std::is_const_v<SEQ>
	&& concepts::nothrow_equality_comparable_with<value_t, typename
	SEQ::value_t>
)
```

Consuma le sequenze e ritorna vero se il numero di elementi non è uguale
sia per numero che per valore

```
template <typename SEQ,typename F>
[[nodiscard]] constexpr sequence& cmp(SEQ& oth, F predicate)noexcept
requires(
	concepts::sequence<SEQ>
	&& !std::is_const_v<SEQ>
	&& is_cmp_predicate<F, typename SEQ::value_t>()
)
```

Avanza le due sequenze fino a che predicate non sia soddisfatto

```
[[nodiscard]] constexpr bool operator==(const sequence& seq)const noexcept
```

Ritorna vero se le due sequenze si riferiscono allo stesso elemento corrente.
`seq` deve avere come istanza principale la stessa della sequenza corrente

shared_box

Uso:

Un wrapper per la memoria dinamica con elemento condiviso

Testata:

`iron/shared_box.h`

Sinossi:

```
template <typename T,typename D,int FLAGS>
class base_shared_box final
```

La base di `shared_box`

Alias esterni:

```
template <typename T, typename D>
using base_rc = base_shared_box<T, D, <private>>
```

La base di `rc`

```
template <typename T, typename D = box_deleter<T,unsafe::allocator<T>>>
using rc = base_rc<T,D>
```

Non ha la sicurezza sui `thread` per l'accesso al contatore interno delle istanze condivise

```
template <typename T, typename D = box_deleter<T,unsafe::allocator<T>>>
using rcw = typename base_rc<T,D>::weak_t
```

Il tipo `weak` per `rc`

```
template <typename T, typename D>
using base_arc = base_shared_box<T,D,<private>>
```

La base di `arc`

```
template <typename T, typename D = box_deleter<T,unsafe::allocator<T>>>
using arc = base_arc<T,D>
```

Ha la sicurezza sui `thread` per l'accesso al contatore interno delle istanze condivise

```
template <typename T, typename D = box_deleter<T,unsafe::allocator<T>>>
using arcw = typename base_arc<T,D>::weak_t
```

Il tipo `weak` per `arc`

Alias interni:

```
using deleter_t = D
using deleter_type = deleter_t
using value_t = T
using value_type = value_t
using ptr_t = value_t* const
using pointer = ptr_t
using cptr_t = const value_t* const
using const_pointer = cptr_t
using counter_t = std::size_t
using counter_type = counter_t
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using box_t = base_box<value_t, deleter_t>
using size_t = std::size_t
using size_type = size_t
using ref_t = typename std::add_lvalue_reference<value_t>::type
using reference = ref_t
using cref_t = typename std::add_lvalue_reference<const value_t>::type
```

```
using const_reference = cref_t
template <typename T1>
using rebind_t = base_shared_box<T1, typename deleter_t::template
rebind_t<T1>,FLAGS,T1>
template <typename T1>
using rebind_type = rebind_t<T1>
using weak_t = base_shared_box<value_t, deleter_t,<private>>
using weak_type = weak_t
```

Metodi statici:

```
[[nodiscard]] static constexpr bool thread_safe()noexcept
```

Ritorna vero se l'accesso al contatore delle istanze condivise è sicuro in un ambiente `multithread`

```
[[nodiscard]] static constexpr bool weak()noexcept
```

Ritorna vero se il tipo della classe è `weak_t`

```
[[nodiscard]] static constexpr counter_t max_count()noexcept
```

Ritorna il massimo valore del contatore delle istanze condivise

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
!std::is_same_v<value_t*,void*>
concepts::box_deleter<deleter_t>
std::is_same_v<value_t, typename deleter_t::value_t>
```

Costruttori:

```
[[nodiscard]] static constexpr result<base_shared_box, error_handler_t>
try_from(box_t&&)noexcept
requires (!weak())

[[nodiscard]] static constexpr base_shared_box from(box_t&&)noexcept
requires (!weak())
```

Costruisce da una istanza di `box_t`

```
constexpr base_shared_box()noexcept = default
constexpr explicit base_shared_box(std::nullptr_t)noexcept
constexpr base_shared_box(base_shared_box&&)noexcept
```

```
constexpr base_shared_box(const base_shared_box& v)noexcept
```

Costruttore di copia
In caso il contatore interno di `v` sia uguale a `max_count()`
l'istanza sarà vuota

Costruttori esterni:

```
template <typename T, typename... ARGS>
[[nodiscard]] constexpr result<rc<T>,error_handler>
try_make_rc(ARGS&&...)noexcept

template <typename T, typename... ARGS>
[[nodiscard]] constexpr rc<T> make_rc(ARGS&&...)noexcept

template <typename T, typename... ARGS>
[[nodiscard]] result<arc<T>,error_handler>
try_make_arc(ARGS&&...)noexcept
```

```
template <typename T, typename... ARGS>
[[nodiscard]] arc<T> make_arc(ARGS&&...)noexcept
```

Distruttore:

```
constexpr ~base_shared_box()
```

Metodi:

```
constexpr base_shared_box& operator=(std::nullptr)noexcept
requires(sizeof(value_t) > 0 && std::is_nothrow_destructible_v<value_t>)
```

Equivalente a `reset()`

```
constexpr base_shared_box& operator=(const base_shared_box& v)noexcept
requires(sizeof(value_t) > 0 && std::is_nothrow_destructible_v<value_t>)
```

Operatore di assegnamento
In caso il contatore interno di `v` sia uguale a `max_count()`
l'istanza corrente sarà vuota

```
constexpr base_shared_box& operator=(base_shared_box&&)noexcept
constexpr base_shared_box& swap(base_shared_box&)noexcept
```

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se l'istanza corrente è vuota

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se l'istanza corrente non è vuota

```
[[nodiscard]] constexpr counter_t use_count()const noexcept
```

Ritorna il contatore delle istanze condivise di `value_t`
Le condivisioni di tipo `weak_t` non sono conteggiate

```
[[nodiscard]] constexpr counter_t use_weak_count()const noexcept
```

Ritorna il contatore delle istanze condivise di tipo `weak_t`

```
[[nodiscard]] constexpr ptr_t unsafe_data()noexcept
requires(!weak())

[[nodiscard]] constexpr cptr_t unsafe_data()const noexcept
requires(!weak())

[[nodiscard]] constexpr cptr_t unsafe_cdata()const noexcept
requires(!weak())
```

Ritorna un puntatore della istanza condivisa
Se nessuna istanza è impostata ritorna un valore nullo
Il metodo non è sicuro perché ritorna un puntatore

```
[[nodiscard]] constexpr option<ref_wrapper<value_t>> value()noexcept
requires(!weak())
```

```
[[nodiscard]] constexpr option<ref_wrapper<const value_t>> value()const noexcept
requires(!weak())
```

```
[[nodiscard]] constexpr option<ref_wrapper<const value_t>> cvalue()const noexcept
requires(!weak())
```

Ritorna un riferimento al valore della istanza condivisa

```
[[nodiscard]] constexpr ref_wrapper<value_t> operator*()noexcept
requires(!weak())
```

```
[[nodiscard]] constexpr ref_wrapper<const value_t> operator*()const noexcept
requires(!weak())
```

Ritorna un riferimento al valore della istanza condivisa
Se nessuna istanza è impostata viene chiamata `PANIC()`

```
[[nodiscard]] constexpr value_t* operator->()noexcept
requires(!weak() && sizeof(value_t) > 0 && std::is_class_v<value_t>)
```

```
[[nodiscard]] constexpr const value_t* operator->()const noexcept
requires(!weak() && sizeof(value_t) > 0 && std::is_class_v<value_t>)
```

Ritorna il puntatore all'istanza interna
Se non è impostata viene chiamata `PANIC()`
Da usare solo con `value_t` classe o struttura per accedere ad un valore od un metodo

```
[[nodiscard]] constexpr weak_t make_weak()const noexcept
```

Costruisce e ritorna una istanza `weak_t`
Il tipo `weak_t` non interviene sul conteggio delle istanze condivise
ma solo sul conteggio `weak`
Se il conteggio `weak` corrente è uguale a `max_count()` o l'istanza corrente è scaduta
ritorna una istanza non impostata.

```
[[nodiscard]] constexpr auto lock()const noexcept
requires(weak())
```

Costruisce e ritorna una istanza effettiva da una istanza `weak`
Se il conteggio corrente è uguale a `max_count()` o l'istanza corrente è scaduta viene ritornata una istanza non impostata.

```
[[nodiscard]] constexpr base_box<base_value_t, deleter_t> release()noexcept
requires(!weak())
```

Se il conteggio delle istanze condivise è uguale a uno l'istanza viene rilasciata
e ritorna una istanza che la contiene altrimenti ritorna una istanza vuota
In caso di successo l'istanza corrente sarà non impostata.

```
constexpr base_shared_box& reset()noexcept
```

Distrugge l'istanza impostata.
Se il contatore delle istanze condivise raggiunge lo zero distrugge anche l'istanza condivisa.
Dopo questa operazione l'istanza sarà non impostata

```
template <typename U>
[[nodiscard]] auto upcast()const noexcept
requires (
                !weak()
                && (std::is_same_v<U, value_t> || std::is_base_of_v<U, value_t>)
                && std::has_virtual_destructor_v<U>
)
```

Ritorna una istanza con tipo `U`, una classe base o uguale a `value_t`
Se l'istanza corrente è impostata il contatore interno viene incrementato ma se è uguale a `max_count()` ritorna una istanza non impostata

```
template <typename U>
[[nodiscard]] auto downcast()const noexcept
requires (
        !weak()
        && (std::is_same_v<U, value_t> || std::is_base_of_v<value_t,U>)
        && std::has_virtual_destructor_v<U>
)
```

Ritorna una istanza con tipo `U`, una classe derivata o uguale a `value_t`
In caso di successo il contatore interno viene incrementato ma se è uguale a `max_count()` ritorna una istanza vuota

```
template <typename U>
[[nodiscard]] auto cast()const noexcept
requires(
        !weak()
        && (
                std::is_same_v<U, value_t>
                || std::is_base_of_v<value_t,U>
                || std::is_base_of_v<U, value_t>
        )
        && std::has_virtual_destructor_v<U>
)
```

Ritorna una istanza con tipo `U`, una classe base,derivata o uguale a `value_t`
In caso di successo il contatore interno viene incrementato ma se è uguale a `max_count()` ritorna una istanza vuota

```
template <typename T1,typename D1,int FLAGS1>
[[nodiscard]] constexpr bool
operator==(const base_shared_box<T1,D1,FLAGS1>& v)const noexcept
requires(
        (!weak() && !base_shared_box<T1,D1,FLAGS1>::weak())
        && (
                std::is_same_v<T1,value_t>
                || std::is_base_of_v<value_t,T1>
                || std::is_base_of_v<T1,value_t>
        )
)
```

Ritorna vero se l'istanza interna è la stessa di `v`.

str

Uso:

stringa di caratteri
Il tipo di carattere deve soddisfare il concetto `charset`
Il contenuto di una stringa puo essere sicuro o no inteso come valore corretto
dei caratteri , non degli elementi contenuti.
Se si tenta di inserire una sequenza di caratteri non validi per la codifica impostata in una stringa sicura viene ritornato l'errore `POSIX illegal_byte_sequence`, mentre in una stringa non sicura l'inserimento è indeterminato e senza segnalazione di errore.

Testata:

`iron/str.h`

Sinossi:

```
template <typename T, typename A, std::size_t FIXED_SIZE,bool SAFE>
class base_str final
```

Costanti esterne:

```
constexpr const std::size_t str_default_fixedsize = 30
```

Dimensione predefinita della parte fissa interna

Alias esterni:

```
template <typename T, typename A, std::size_t FIXED_SIZE>
using  str = base_str<T, A, FIXED_SIZE,true>
```

Base di `str` sicura

```
template <typename T, typename A, std::size_t FIXED_SIZE>
using  ustr = base_str<T, A, FIXED_SIZE,false>
```

Base di `str` non sicura

```
template <
        typename A =unsafe::allocator<char>
        , std::size_t FIXED_SIZE = str_default_fixedsize
>
using base_ustrb = ustr<char, A, FIXED_SIZE>

using ustrb = base_ustrb<>
```

Una stringa a contenuto non sicuro con codifica base

```
template <
        typename A =unsafe::allocator<char>
        , std::size_t FIXED_SIZE = str_default_fixedsize
>
using base_strb = str<char, A, FIXED_SIZE>

using strb = base_strb<>
```

Una stringa a contenuto sicuro con codifica base

```
template <
        typename A=unsafe::allocator<char8_t>
        ,std::size_t FIXED_SIZE= str_default_fixedsize
```

```
>
using base_ustr8 = ustr<char8_t, A, FIXED_SIZE>
using ustr8 = base_ustr8<>
```

Una stringa a contenuto non sicuro con codifica `UTF8`

```
template <
        typename A =unsafe::allocator<char8_t>
        , std::size_t FIXED_SIZE = str_default_fixedsize
>
using base_str8 = str<char8_t, A, FIXED_SIZE>
using str8 = base_str8<>
```

Una stringa a contenuto sicuro con codifica `UTF8`

```
template <
        typename A =unsafe::allocator<wchar_t>
        , std::size_t FIXED_SIZE = str_default_fixedsize
>
using base_ustrw = ustr<wchar_t, A, FIXED_SIZE>
using ustrw = base_ustrw<>
```

Una stringa a contenuto non sicuro di `wchar_t`
Il tipo di codifica dipende da `size(wchar_t)`.
Se è `2` viene assunto `UTF16` altrimenti, essendo `4`, viene assunto `UTF32`
Questa classe esiste solo per essere usata in casi particolari come ad esempio le `API win32`

```
template <
        typename A =unsafe::allocator<wchar_t>
        , std::size_t FIXED_SIZE = str_default_fixedsize
>
using base_strw = str<wchar_t, A, FIXED_SIZE>
using strw = base_strw<>
```

Una stringa a contenuto sicuro di `wchar_t`
Il tipo di codifica dipende `size(wchar_t)`.
Se è `2` viene assunto `UTF16` altrimenti, essendo `4`, viene assunto `UTF32`
Questa classe esiste solo per essere usata in casi particolari come ad esempio le `API win32`

```
template <
        typename A =unsafe::allocator<char16_t>
        , std::size_t FIXED_SIZE = str_default_fixedsize
>
using base_ustr16 = ustr<char16_t, A, FIXED_SIZE>
using ustr16 = base_ustr16<>
```

Una stringa a contenuto non sicuro con codifica `UTF16`

```
template <
        typename A =unsafe::allocator<char16_t>
        , std::size_t FIXED_SIZE = str_default_fixedsize
>
using base_str16 = str<char16_t, A, FIXED_SIZE>
using str16 = base_str16<>
```

Una stringa a contenuto sicuro con codifica `UTF16`

```
template <
        typename A =unsafe::allocator<char32_t>
        , std::size_t FIXED_SIZE = str_default_fixedsize
>
using base_ustr32 = ustr<char32_t, A, FIXED_SIZE>
using ustr32 = base_ustr32<>
```

Una stringa a contenuto non sicuro con codifica `UTF32`

```
template <
        typename A =unsafe::allocator<char32_t>
        , std::size_t FIXED_SIZE = str_default_fixedsize
```

```
>
using base_str32 = str<char32_t, A, FIXED_SIZE>
using str32 = base_str32<>
```

Una stringa a contenuto sicuro con codifica `UTF32`

Alias interni:

```
using value_t = T
using value_type = value_t
using allocator_t = A
using allocator_type = allocator_t
using size_t = typename _raw_str_t::size_t
using size_type = size_t
using adapter_box_t = <private>
using view_t = base_strview<value_t,SAFE>
using view_type = view_t
using cview_t = base_strview<const value_t,SAFE>
using const_view_type = cview_t
using char_t = std::conditional_t<concepts::fixed_charset<value_t>, value_t,
       char32_t>
using char_type = char_t
using diff_t = std::ptrdiff_t
using difference_type_t = diff_t
using ptr_t = value_t*
using cptr_t = const value_t*
using pointer = ptr_t
using ref_t = ref_wrapper<value_t>
using cref_t = ref_wrapper<const value_t>
using reference = value_t&
using const_reference = const value_t&
using iterator = <private>
using const_iterator = <private>
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using raw_seq_type = raw_seq_t
using craw_seq_t = typename view_t::craw_seq_t
using const_raw_seq_type = craw_seq_t
using rev_raw_seq_t = typename view_t::rev_raw_seq_t
using rev_raw_seq_type = rev_raw_seq_t
using crev_raw_seq_t = typename view_t::crev_raw_seq_t
using const_rev_raw_seq_type = crev_raw_seq_t
using seq_t = typename view_t::seq_t
using seq_type = seq_t
using cseq_t = typename view_t::cseq_t
using const_seq_type = cseq_t
```

Metodi statici:

```
[[nodiscard]] constexpr size_t max_size()noexcept
```

Ritorna il massimo numero di elementi

```
[[nodiscard]] constexpr size_t fixed_size()noexcept
```

Ritorna il numero di elementi preallocati in modo fisso.
Il valore può essere zero

```
[[nodiscard]] constexpr bool is_safe()noexcept
```

Ritorna vero se la stringa deve contenere caratteri validi per la codifica impostata

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_const_v<value_t>
concepts::charset<value_t>
fixed_size() < max_size()
```

Costruttori:

```
[[nodiscard]] static constexpr base_str unchecked_from(allocator_t a)noexcept
```

Costruisce una stringa vuota dall’allocatore `a`

Il metodo non è sicuro perché `allocator_t` non è sicuro

```
template <typename U>
[[nodiscard]] static constexpr result<base_str,error_handler_t>
try_unchecked_from(U c,size_t n,allocator_t a)noexcept
requires concepts::fixed_charset<U>

template <typename U>
[[nodiscard]] static constexpr base_str
unchecked_from(U c,size_t n,allocator_t a)noexcept
requires concepts::fixed_charset<U>
```

Costruisce una stringa composta da `n` caratteri `c` usando l'allocatore `a`
Il metodo non è sicuro perché `allocator_t` non è sicuro

```
template <typename U>
[[nodiscard]] static constexpr result<base_str, error_handler_t>
try_from(U c, size_t n = 1)noexcept
requires concepts::fixed_charset<U>

template <typename U>
[[nodiscard]] static constexpr base_str from(U c,size_t n=1)noexcept
requires concepts::fixed_charset<U>
```

Costruisce una stringa composta da `n` caratteri `c`

```
template <typename ITER>
[[nodiscard]] static constexpr result<base_str, error_handler_t>
try_unchecked_from(ITER& b, const ITER& e, size_t n=1,allocator_t a = {})noexcept
requires (
        concepts::charset_forward_iterator<ITER>
        && !std::is_const_v<ITER>
)

template <typename ITER>
[[nodiscard]] static constexpr base_str
unchecked_from(ITER& b, const ITER& e, size_t n=1,allocator_t a = {})noexcept
requires (
               concepts::charset_forward_iterator<ITER>
               && !std::is_const_v<ITER>
)
```

Costruisce una stringa composta da una coppia di iteratori ripetuta `n` volte usando l'allocatore `a`
Il metodo non è sicuro perché `allocator_t` e `ITER` non sono sicuri

```
template <typename U, size_t N>
[[nodiscard]] static constexpr result<base_str, error_handler_t>
try_unchecked_from(const U(&v)[N], size_t n, allocator_t a)noexcept
requires (concepts::charset<U>&& N < max_size())

template <typename U, size_t N>
[[nodiscard]] static constexpr base_str
unchecked_from(const U(&v)[N], size_t n, allocator_t a)noexcept
requires (concepts::charset<U>&& N < max_size())
```

Costruisce una stringa da un `builtin array` ripetuto `n` volte usando l'allocatore `a`
Se il valore di `v[N - 1]` è `\0` viene escluso dal caricamento
Il metodo non è sicuro perché `allocator_t` non è sicuro

```
template <typename U, size_t N>
[[nodiscard]] static constexpr result<base_str, error_handler_t>
try_from(const U(&v)[N], size_t n = 1)noexcept
requires (concepts::charset<U> && N < max_size())
```

```
template <typename U, size_t N>
[[nodiscard]] static constexpr base_str from(const U(&v)[N], size_t n = 1)noexcept
requires (concepts::charset<U> && N < max_size())
```

Costruisce una stringa da un `builtin array` ripetuto `n` volte
Se il valore di `v[N - 1]` è `\0` viene escluso dal caricamento

```
template <typename SEQ>
[[nodiscard]] static constexpr result<base_str, error_handler_t>
try_from(const SEQ& seq,size_t n=1)noexcept
requires concepts::charset_sequence<SEQ>
```

```
template <typename SEQ>
[[nodiscard]] static constexpr base_str from(const SEQ& seq,size_t n=1)noexcept
requires concepts::charset_sequence<SEQ>
```

Costruisce una stringa da una sequenza ripetuta `n` volte

```
[[nodiscard]] static constexpr result<base_str, error_handler>
try_from(adapter_box_t&&)noexcept
```

```
[[nodiscard]] static constexpr base_str from(adapter_box_t&&)noexcept
```

Costruisce una stringa da un `rvalue adapter_box_t`

```
template <typename T1, typename A1, std::size_t FIXED_SIZE1,bool SAFE1>
[[nodiscard]] static constexpr result<base_str, error_handler_t>
try_clone_from(const base_str<T1,A1,FIXED_SIZE1,SAFE1>& s,size_t n=1)noexcept
```

```
template <typename T1, typename A1, std::size_t FIXED_SIZE1,bool SAFE1>
[[nodiscard]] static constexpr base_str
clone_from(const base_str<T1,A1,FIXED_SIZE1,SAFE1>& s,size_t n=1)noexcept
```

Costruisce una stringa da `s` ripetuta `n` volte

```
constexpr base_str()noexcept=default
constexpr base_str(base_str&&)noexcept
base_str(const base_str&)=delete
```

Costruttori esterni:

Queste funzioni chiamano `PANIC()` in caso di errore e sono definite nello spazio dei nomi `literals`

```
constexpr ustrb operator""_ub(const char* v, std::size_t sz)noexcept
```

Ritorna una stringa `basic charset` a contenuto non sicuro da `v` per lunghezza `sz`

```
constexpr strb operator""_sb(const char* v, std::size_t sz)noexcept
```

Ritorna una stringa `basic charset` a contenuto sicuro da `v` per lunghezza `sz`

```
constexpr ustr8 operator""_u8(const char* v, std::size_t sz)noexcept
constexpr ustr8 operator""_u8(const char8_t* v, std::size_t sz)noexcept
constexpr ustr8 operator""_u8(const wchar_t* v, std::size_t sz)noexcept
constexpr ustr8 operator""_u8(const char16_t* v, std::size_t sz)noexcept
constexpr ustr8 operator""_u8(const char32_t* v, std::size_t sz)noexcept
```

Ritorna una stringa `utf8 charset` a contenuto non sicuro da `v`

```
constexpr str8 operator""_s8(const char* v, std::size_t sz)noexcept
constexpr str8 operator""_s8(const char8_t* v, std::size_t sz)noexcept
constexpr str8 operator""_s8(const wchar_t* v, std::size_t sz)noexcept
constexpr str8 operator""_s8(const char16_t* v, std::size_t sz)noexcept
constexpr str8 operator""_s8(const char32_t* v, std::size_t sz)noexcept
```

Ritorna una stringa `utf8 charset` a contenuto sicuro da `v` per lunghezza `sz`

```
constexpr ustr16 operator""_u16(const char* v, std::size_t sz)noexcept
constexpr ustr16 operator""_u16(const char8_t* v, std::size_t sz)noexcept
constexpr ustr16 operator""_u16(const wchar_t* v, std::size_t sz)noexcept
constexpr ustr16 operator""_u16(const char16_t* v, std::size_t sz)noexcept
constexpr ustr16 operator""_u16(const char32_t* v, std::size_t sz)noexcept
```

Ritorna una stringa `utf16 charset` a contenuto non sicuro da `v` per lunghezza `sz`

```
constexpr str16 operator""_s16(const char* v, std::size_t sz)noexcept
constexpr str16 operator""_s16(const char8_t* v, std::size_t sz)noexcept
constexpr str16 operator""_s16(const wchar_t* v, std::size_t sz)noexcept
constexpr str16 operator""_s16(const char16_t* v, std::size_t sz)noexcept
constexpr str16 operator""_s16(const char32_t* v, std::size_t sz)noexcept
```

Ritorna una stringa `utf16 charset` a contenuto sicuro da `v`

```
constexpr ustrw operator""_uw(const char* v, std::size_t sz)noexcept
constexpr ustrw operator""_uw(const char8_t* v, std::size_t sz)noexcept
constexpr ustrw operator""_uw(const wchar_t* v, std::size_t sz)noexcept
constexpr ustrw operator""_uw(const char16_t* v, std::size_t sz)noexcept
constexpr ustrw operator""_uw(const char32_t* v, std::size_t sz)noexcept
```

Ritorna una stringa `wchar_t` a contenuto non sicuro da `v` per lunghezza `sz`

```
constexpr strw operator""_sw(const char* v, std::size_t sz)noexcept
constexpr strw operator""_sw(const char8_t* v, std::size_t sz)noexcept
constexpr strw operator""_sw(const wchar_t* v, std::size_t sz)noexcept
constexpr strw operator""_sw(const char16_t* v, std::size_t sz)noexcept
constexpr strw operator""_sw(const char32_t* v, std::size_t sz)noexcept
```

Ritorna una stringa `wchar_t` a contenuto sicuro da `v` per lunghezza `sz`

```
constexpr ustr32 operator""_u32(const char* v, std::size_t sz)noexcept
constexpr ustr32 operator""_u32(const char8_t* v, std::size_t sz)noexcept
constexpr ustr32 operator""_u32(const wchar_t* v, std::size_t sz)noexcept
constexpr ustr32 operator""_u32(const char16_t* v, std::size_t sz)noexcept
constexpr ustr32 operator""_u32(const char32_t* v, std::size_t sz)noexcept
```

Ritorna una stringa `utf32 charset` a contenuto non sicuro da `v` per lunghezza `sz`

```
constexpr str32 operator""_s32(const char* v, std::size_t sz)noexcept
constexpr str32 operator""_s32(const char8_t* v, std::size_t sz)noexcept
constexpr str32 operator""_s32(const wchar_t* v, std::size_t sz)noexcept
constexpr str32 operator""_s32(const char16_t* v, std::size_t sz)noexcept
constexpr str32 operator""_s32(const char32_t* v, std::size_t sz)noexcept
```

Ritorna una stringa `utf32 charset` a contenuto sicuro da `v` per lunghezza `sz`

Distruttore:

```
constexpr ~base_str()=default
```

Metodi:

```
constexpr base_str& operator=(base_str&&)noexcept
base_str& operator=(const base_str&)=delete
constexpr base_str& swap(base_str&)noexcept
```

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se il numero di elementi effettivi è zero

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se il numero di elementi effettivi non è zero

```
[[nodiscard]] constexpr size_t size()const noexcept
```

Ritorna il numero di elementi effettivi

```
[[nodiscard]] constexpr size_t capacity()const noexcept
```

Ritorna il numero di elementi allocati

```
[[nodiscard]] constexpr size_t count()const noexcept
```

Ritorna il numero di caratteri effettivi

```
[[nodiscard]] constexpr allocator_t unsafe_allocator()const noexcept
```

Ritorna l'allocatore
Il metodo non è sicuro perche `allocator_t` non è sicuro

```
[[nodiscard]] constexpr bool use_fixed()const noexcept
```

Ritorna vero se viene usata solo la parte fissa di memoria interna per memorizzare gli elementi.

```
constexpr base_str& clear()noexcept
```

Riporta il numero di elementi effettivi a zero ma senza alterare la capacità.
Dopo questa operazione `size()` ritorna zero.
Non ci devono essere referenti attivi.

```
constexpr base_str& reset()noexcept
```

Riporta il numero di elementi effettivi a zero eventualmente liberando lo spazio occupato nella memoria dinamica.
Dopo questa operazione `capacity()` ritorna il valore di `fixed_size()` e `size()` ritorna zero
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr result<base_str,error_handler_t>
try_clone(size_t n = 1)const noexcept
```

```
[[nodiscard]] constexpr base_str clone(size_t n = 1)const noexcept
```

Ritorna una stringa copiata `n` volte

```
[[nodiscard]] constexpr error_handler_t
try_reserve_at_least(size_t additional, size_t n = 1)noexcept
```

```
constexpr base_str& reserve_at_least(size_t additional, size_t n = 1)noexcept
```

Riserva uno spazio per aggiungere almeno `addictional * n` elementi
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t
try_reserve_exact(size_t additional, size_t n = 1)noexcept

constexpr base_str& reserve_exact(size_t additional, size_t n = 1)noexcept
```

Riserva uno spazio per aggiungere esattamente `addictional * n` elementi
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr result<adapter_box_t, error_handler_t>
try_release()noexcept

[[nodiscard]] constexpr adapter_box_t release()noexcept
```

Esporta gli elementi attivi in `adapter_box_t`
Dopo questa operazione `capacity()` ritorna il valore di `fixed_size()`
e `size()` ritorna zero
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr value_t* unsafe_data()noexcept
requires(!is_safe())

[[nodiscard]] constexpr const value_t* unsafe_data()const noexcept
[[nodiscard]] constexpr const value_t* unsafe_cdata()const noexcept
```

Ritorna un puntatore al primo elemento
Il metodo non è sicuro perche ritorna un puntatore
Se il puntatore non è nullo ha come terminatore `\0` per compatibilità con le stringhe `C`

```
[[nodiscard]] constexpr error_handler_t try_shrink_to_fit()noexcept
```

Libera dalla memoria elementi inutilizzati
Non ci devono essere referenti attivi
L'operazione è transazionale

```
constexpr base_str& shrink_to_fit()noexcept
```

Come `try_shrink_to_fit` ma in caso di errore esso viene ignorato

```
[[nodiscard]] constexpr error_handler_t try_raw_resize(
        size_t n
        , value_t v = {}
)noexcept
requires (!is_safe())

constexpr base_str& raw_resize(size_t n, value_t v = {})noexcept
requires (!is_safe())
```

Porta il numero di elementi effettivi a `n` con l'eccedenza a valore `v`
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t try_resize(size_t n)noexcept

constexpr base_str& resize(size_t n)noexcept
```

Porta il numero di caratteri effettivi a `n` con l'eccedenza a valore predefinito
Non ci devono essere referenti attivi
L'operazione è transazionale

```
template <typename U>
[[nodiscard]] constexpr error_handler_t try_resize(size_t n,U v)noexcept
requires (concepts::fixed_charset<U>)
```

```
template <typename U>
constexpr base_str& resize(size_t n,U v)noexcept
requires (concepts::fixed_charset<U>)
```

Porta il numero di caratteri effettivi a `n` con l'eccedenza a valore `v`.
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t try_realloc_on_heap()noexcept
```

```
base_str& realloc_on_heap()noexcept
```

Porta gli elementi effettivi allocati nella memoria dinamica
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr value_t& unchecked_raw_nth(size_t idx)noexcept
requires(!is_safe())
```

```
[[nodiscard]] constexpr value_t unchecked_raw_nth(size_t idx)const noexcept
```

Ritorna l'elemento alla posizione `idx`, il primo elemento ha indice zero
Il metodo non è sicuro perche `idx` è controllato solo in modalità `debug`

```
[[nodiscard]] constexpr option<value_t> raw_nth(size_t idx)const noexcept
```

Ritorna l'elemento alla posizione `idx`, il primo elemento ha indice zero

```
[[nodiscard]] constexpr option<value_t> raw_front()const noexcept
```

Ritorna l'elemento alla prima posizione

```
[[nodiscard]] constexpr option<value_t> raw_back()const noexcept
```

Ritorna l'elemento all' ultima posizione

```
[[nodiscard]] constexpr option<char_t> nth(size_t idx)const noexcept
```

Ritorna il carattere alla posizione `idx`, il primo carattere ha indice zero

```
[[nodiscard]] constexpr option<char_t> front()const noexcept
```

Ritorna il carattere alla prima posizione

```
[[nodiscard]] constexpr option<char_t> back()const noexcept
```

Ritorna il carattere all' ultima posizione

```
[[nodiscard]] constexpr cview_t view()const noexcept
[[nodiscard]] constexpr cview_t cview()const noexcept
[[nodiscard]] constexpr view_t view()noexcept
```

Ritorna una vista sugli elementi

```
[[nodiscard]] constexpr raw_seq_t raw_seq()noexcept
requires (!is_safe())

[[nodiscard]] constexpr const_raw_seq_t raw_seq()const noexcept

[[nodiscard]] constexpr const_raw_seq_t craw_seq()const noexcept
```

Ritorna una sequenza sugli elementi

```
[[nodiscard]] constexpr raw_seq_t raw_rseq()noexcept
requires (!is_safe())

[[nodiscard]] constexpr raw_seq_t rseq()noexcept
requires (!is_safe() && concepts::fixed_charset<value_t>)

[[nodiscard]] constexpr const_raw_seq_t craw_rseq()const noexcept

[[nodiscard]] constexpr const_raw_seq_t rseq()const noexcept
requires (concepts::fixed_charset<value_t>)

[[nodiscard]] constexpr const_raw_seq_t crseq()const noexcept
requires (concepts::fixed_charset<value_t>)
```

Ritorna una sequenza inversa sugli elementi

```
[[nodiscard]] constexpr seq_t seq()noexcept
requires (!is_safe())
```

Ritorna una sequenza sui caratteri
Se `value_t` è a codifica variabile il carattere è immutabile

```
[[nodiscard]] constexpr cseq_t seq()const noexcept
[[nodiscard]] constexpr cseq_t cseq()const noexcept
```

Ritorna una sequenza su caratteri immutabili

```
template <typename U>
[[nodiscard]] constexpr error_handler_t try_push_back(U v,size_t n=1)noexcept
requires (concepts::fixed_charset<U>)

template <typename U>
constexpr base_str& push_back(U v,size_t n=1)noexcept
requires (concepts::fixed_charset<U>)
```

Appende `v` ripetuto `n` volte
Non ci devono essere referenti attivi
L'operazione è transazionale

```
template <typename ITER>
[[nodiscard]] constexpr error_handler_t
try_unchecked_push_back(ITER& b, const ITER& e, size_t n=1)noexcept
requires (
	concepts::charset_forward_iterator<ITER>
	&& !std::is_const_v<ITER>
)

template <typename ITER>
constexpr base_str& unchecked_push_back(ITER& b, const ITER& e, size_t n=1)noexcept
requires (
	concepts::charset_forward_iterator<ITER>
	&& !std::is_const_v<ITER>
)
```

Appende una stringa composta da una coppia di iteratori ripetuta `n` volte
Non ci devono essere referenti attivi
L'operazione è transazionale
Il metodo non è sicuro perché `ITER` non è sicuro

```
template <typename _SEQ>
[[nodiscard]] constexpr error_handler_t
try_push_back(SEQ& seq, size_t n=1)noexcept
requires (
        concepts::charset_sequence<SEQ>
        && !std::is_const_v<SEQ>
)

template <typename SEQ>
[[nodiscard]] constexpr base_str& push_back(SEQ& seq,size_t n=1)noexcept
requires (
        concepts::charset_sequence<SEQ>
        && !std::is_const_v<SEQ>
)
```

Appende una stringa composta da una sequenza ripetuta `n` volte
Se l'operazione termina correttamente `seq` sarà consumata
Non ci devono essere referenti attivi
L'operazione è transazionale

```
template <typename U, size_t N>
[[nodiscard]] constexpr error_handler_t try_push_back(
        const U(&v)[N], size_t n=1
)noexcept
requires(concepts::charset<U> && N < max_size())

template <typename U, size_t N>
constexpr base_str& push_back(const U(&v)[N], size_t n=1)noexcept
requires(concepts::charset<U> && N < max_size())
```

Appende da un `builtin array` ripetuto `n` volte
Se il valore di `v[N - 1]` è `\0` viene escluso dal caricamento
Non ci devono essere referenti attivi
L' operazione è transazionale

```
[[nodiscard]] constexpr result<base_str, error_handler_t>
try_to_upper()const noexcept
requires(concepts::basic_charset<value_t>)

[[nodiscard]] result<base_str, error_handler_t>
try_to_upper()const noexcept
requires(!concepts::basic_charset<value_t>)

[[nodiscard]] constexpr base_str to_upper()const noexcept
requires(concepts::basic_charset<value_t>)

[[nodiscard]] base_str to_upper()const noexcept
requires(!concepts::basic_charset<value_t>)
```

Ritorna una stringa con i caratteri convertiti in lettere maiuscole

```
[[nodiscard]] constexpr result<base_str, error_handler_t>
try_to_lower()const noexcept
requires(concepts::basic_charset<value_t>)

[[nodiscard]] result<base_str, error_handler_t>
try_to_lower()const noexcept
requires(!concepts::basic_charset<value_t>)

[[nodiscard]] constexpr base_str to_lower()const noexcept
requires(concepts::basic_charset<value_t>)

[[nodiscard]] base_str to_lower()const noexcept
requires(!concepts::basic_charset<value_t>)
```

Ritorna una stringa con i caratteri convertiti in lettere minuscole

```
template <typename T1, typename A1, std::size_t FIXED_SIZE1, bool SAFE1>
[[nodiscard]] constexpr bool operator==(
        const base_str<T1,A1,FIXED_SIZE1,SAFE1>&
)const noexcept
```

Ritorna vero per uguaglianza lessicografica

```
template <typename U, std::size_t N>
[[nodiscard]] constexpr bool operator==(const U(&v)[N])const noexcept
requires(concepts::charset<U> && N < max_size())
```

Ritorna vero per uguaglianza lessicografica con un `builtin array`
Se il valore di `v[N - 1]` è `\0` viene escluso dal confronto

```
template <typename T1, typename A1, std::size_t FIXED_SIZE1, bool SAFE1>
[[nodiscard]] constexpr bool operator <(
        const base_str<T1,A1,FIXED_SIZE1,SAFE1>&
)const noexcept
```

Ritorna vero per minoranza lessicografica

```
template <typename U, std::size_t N>
[[nodiscard]] constexpr bool operator<(const U(&v)[N])const noexcept
requires(concepts::charset<U> && N < max_size())
```

Ritorna vero per minoranza lessicografica con un `builtin array`
Se il valore di `v[N - 1]` è `\0` viene escluso dal confronto

```
template <typename T1, typename A1, std::size_t FIXED_SIZE1, bool SAFE1>
[[nodiscard]] constexpr std::strong_ordering operator <=>(
        const base_str<T1, A1, FIXED_SIZE1, SAFE1>& v
)const noexcept
```

Confronto lessicografico usando `tree_way operator`

```
template <typename U, std::size_t N>
[[nodiscard]] constexpr std::strong_ordering operator <=>(const U(&v)[N])const
noexcept
requires(concepts::charset<U> && N < max_size())
```

Confronto lessicografico usando `tree_way operator` con un builtin `array`
Se il valore di `v[N - 1]` è `\0` viene escluso dal confronto

strview

Uso:

vista su una stringa di caratteri

Testata:

```
iron/strview.h
```

Sinossi:

```
template <typename T,bool SAFE> class base_strview final
```

Alias interni:

```
using value_t = T
using value_type = value_t
using stripped_value_t = std::decay_t<value_t>
using char_t = std::conditional_t<concepts::fixed_charset<value_t>, value_t,
std::conditional_t<std::is_const_v<value_t>, const char32_t, char32_t>>
using char_type=char_t
using stripped_char_t = std::decay_t<char_t>
using stripped_char_type = stripped_char_t
using diff_t = std::ptrdiff_t
using difference_type_t = diff_t
using ptr_t = value_t*
using cptr_t = const value_t*
using pointer = ptr_t
using ref_t = ref_wrapper<value_t>
using cref_t = ref_wrapper<const value_t>
using reference = value_t&
using const_reference = const value_t&
using iterator = <private>
using const_iterator = <private>
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using raw_seq_t = <private>
using raw_seq_type = raw_seq_t
using craw_seq_t = <private>
using const_raw_seq_type = craw_seq_t
using rev_raw_seq_t = <private>
using rev_raw_seq_type = rev_raw_seq_t
using crev_raw_seq_t = <private>
using const_rev_raw_seq_type = crev_raw_seq_t
using seq_t = <private>
using seq_type = seq_t
using cseq_t = <private>
using const_seq_type = cseq_t
using size_t = std::size_t
using size_type = size_t
```

Alias esterni:

```
using uvstrb = base_strview<char, false>
using vstrb = base_strview<char,true>
using uvstr8 = base_strview<char8_t, false>
using vstr8 = base_strview<char8_t,true>
using uvstrw = base_strview<wchar_t,false>
using vstrw = base_strview<wchar_t,true>
using uvstr16 = base_strview<wchar_t,false>
using vstr16 = base_strview<char16_t,true>
using uvstr32 = base_strview<char32_t,false>
using vstr32 = base_strview<char32_t,true>
```

Metodi statici:

```
[[nodiscard]] constexpr bool is_safe()noexcept
```

Ritorna vero se la stringa a cui la vista si riferisce deve contenere caratteri validi per la codifica impostata

```
[[nodiscard]] constexpr size_t max_size()noexcept
```

Ritorna il massimo numero di elementi

Vincoli:

```
!std::is_reference_v<value_t>
concepts::charset<value_t>
```

Costruttori:

```
template <typename MASTER>
[[nodiscard]] static constexpr base_strview
unchecked_from(const MASTER& master,ptr_t start, size_t sz)noexcept
#if IRON_SECURITY_CHECKER
        requires(concepts::has_security_checker<MASTER>)
#endif
```

Ritorna una vista che ha come partenza `start` per lunghezza `sz`
Il metodo non è sicuro per l'uso di un puntatore.
`Master` serve unicamente al `security_checker` per tenere traccia della vita dell'istanza della classe principale, la proprietaria degli elementi.
`MASTER` può essere anche una classe referente di una classe principale
Quando la vista è vuota non è piu una istanza referente.

```
constexpr base_strview()noexcept=default
constexpr base_strview(base_strview&&)noexcept
constexpr base_strview(const base_strview&)noexcept
```

Distruttore:

```
constexpr ~base_strview() = default
```

Metodi:

```
constexpr base_strview& operator=(base_strview&&)noexcept
constexpr base_strview& operator=(const base_strview&)noexcept
constexpr base_strview& swap(base_strview&)noexcept
```

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se il numero di elementi effettivi è zero

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se il numero di elementi effettivi non è zero

```
[[nodiscard]] constexpr size_t size()const noexcept
```

Ritorna il numero di elementi effettivi

```
[[nodiscard]] constexpr size_t count()const noexcept
```

Ritorna il numero di caratteri effettivi

```
constexpr base_str& reset()noexcept
```

Riporta il numero di elementi effettivi a zero
Dopo questa operazione `size()` ritorna zero

```
[[nodiscard]] constexpr value_t* unsafe_data()noexcept
requires(!is_safe())

[[nodiscard]] constexpr const value_t* unsafe_data()const noexcept
[[nodiscard]] constexpr const value_t* unsafe_cdata()const noexcept
```

Ritorna un puntatore al primo elemento
Il metodo non è sicuro perche ritorna un puntatore
Se il puntatore non è nullo non è garantito il terminatore `\0`.

```
[[nodiscard]] constexpr value_t& unchecked_raw_nth(size_t idx)noexcept
requires(!is_safe())
```

```
[[nodiscard]] constexpr stripped_value_t unchecked_raw_nth(size_t idx)const noexcept
```

Ritorna l'elemento alla posizione `idx`, il primo elemento ha indice zero
Il metodo non è sicuro perche `idx` è controllato solo in modalità `debug`

```
[[nodiscard]] constexpr option<stripped_value_t> raw_nth(size_t idx)const noexcept
```

Ritorna l'elemento alla posizione `idx`, il primo elemento ha indice zero

```
[[nodiscard]] constexpr option<stripped_value_t> raw_front()const noexcept
```

Ritorna l'elemento alla prima posizione

```
[[nodiscard]] constexpr option<stripped_value_t> raw_back()const noexcept
```

Ritorna l'elemento all' ultima posizione

```
[[nodiscard]] constexpr option<stripped_char_t> nth(size_t idx)const noexcept
```

Ritorna il carattere alla posizione `idx`, il primo elemento ha indice zero

```
[[nodiscard]] constexpr option<stripped_char_t> front()const noexcept
```

Ritorna il carattere alla prima posizione

```
[[nodiscard]] constexpr option<stripped_char_t> back()const noexcept
```

Ritorna il carattere all' ultima posizione

```
[[nodiscard]] constexpr raw_seq_t raw_seq()noexcept
requires (!is_safe())

[[nodiscard]] constexpr craw_seq_t raw_seq()const noexcept
[[nodiscard]] constexpr craw_seq_t craw_seq()const noexcept
```

Ritorna una sequenza sugli elementi

```
[[nodiscard]] constexpr rev_raw_seq_t raw_rseq()noexcept
requires (!is_safe())

[[nodiscard]] constexpr rev_raw_seq_t rseq()noexcept
requires (!is_safe() && concepts::fixed_charset<value_t>)

[[nodiscard]] constexpr crev_raw_seq_t raw_rseq()const noexcept
[[nodiscard]] constexpr crev_raw_seq_t craw_rseq()const noexcept

[[nodiscard]] constexpr crev_raw_seq_t rseq()const noexcept
requires (concepts::fixed_charset<value_t>)

[[nodiscard]] constexpr crev_raw_seq_t crseq()const noexcept
requires (concepts::fixed_charset<value_t>)
```

Ritorna una sequenza inversa sugli elementi

```
[[nodiscard]] constexpr seq_t seq()noexcept
requires (!is_safe())
```

Ritorna una sequenza sui caratteri

```
[[nodiscard]] constexpr cseq_t seq()const noexcept
[[nodiscard]] constexpr cseq_t cseq()const noexcept
```

Ritorna una sequenza su caratteri immutabili

```
constexpr base_strview& make_base_uppercase()noexcept
requires(!std::is_const_v<value_t>)
```

Converte i caratteri il cui valore rientra in un `basic_charset` in maiuscolo

```
constexpr base_strview& make_base_lowercase()noexcept
requires(!std::is_const_v<value_t>)
```

Converte i caratteri il cui valore rientra in un `basic_charset` in minuscolo

```
[[nodiscard]] constexpr base_strview slice(size_t start, size_t len=max_size())const
noexcept
```

Ritorna una vista partendo dal carattere a posizione `start` per lunghezza massima di `len` caratteri
Il primo carattere ha posizione zero.

```
template <typename F>
[[nodiscard]] constexpr base_strview ltrim_if(F f)const noexcept
requires (concepts::bool_predicate<F, stripped_char_t>)
```

Ritorna una vista eliminando i caratteri che soddisfano `f`
fino a trovare un carattere che non lo soddisfa

```
[[nodiscard]] constexpr base_strview ltrim()const noexcept
requires(concepts::basic_charset<stripped_char_t>)

[[nodiscard]] base_strview ltrim()const noexcept
requires(!concepts::basic_charset<stripped_char_t>)
```

Ritorna una vista eliminando i caratteri spazio bianco
fino a trovare un carattere che non sia uno spazio bianco

```
template <typename F>
[[nodiscard]] constexpr base_strview rtrim_if(F f)const noexcept
requires (concepts::bool_predicate<_F, stripped_char_t>)
```

Ritorna una vista,partendo dalla fine ed eliminando i caratteri che soddisfano `f`
fino a trovare un carattere che non lo soddisfa

```
[[nodiscard]] constexpr base_strview rtrim()const noexcept
requires (concepts::basic_charset<stripped_char_t>)

[[nodiscard]] base_strview rtrim()const noexcept
requires (!concepts::basic_charset<stripped_char_t>)
```

Ritorna una vista partendo dalla fine ed eliminando i caratteri spazio bianco
fino a trovare un carattere che non sia uno spazio bianco

```
template <typename U>
[[nodiscard]]constexpr  base_strview find(U ch)const noexcept
requires(concepts::fixed_charset<U>)
```

Ritorna una vista il cui primo carattere è `ch`
Se `ch` non è trovato ritorna una vista vuota

```
[[nodiscard]] constexpr base_strview find(const base_strview& w)const noexcept
```

Ritorna una vista il cui primo carattere è il contenuto della vista `w`
Se il confronto non è soddisfatto ritorna una vista vuota

```
template <typename U>
[[nodiscard]] constexpr  bool starts_with(U ch)const noexcept
requires(concepts::fixed_charset<U>)
```

Ritorna vero se il primo carattere della vista è `ch`

```
[[nodiscard]] constexpr bool starts_with(const base_strview& w)const noexcept
```

Ritorna vero se la vista inizia con i caratteri della vista `w`

```
template <typename U>
[[nodiscard]] constexpr  bool contains(U ch)const noexcept
requires(concepts::fixed_charset<U>)
```

Ritorna vero se la vista contiene il carattere `ch`

```
[[nodiscard]] constexpr bool contains(const base_strview& w)const noexcept
```

Ritorna vero se la vista contiene i caratteri della vista `w`

```
template <typename U>
[[nodiscard]] constexpr  bool ends_with(U ch)const noexcept
requires(concepts::fixed_charset<U>)
```

Ritorna vero se il l'ultimo carattere della vista è `ch`

```
[[nodiscard]] constexpr bool ends_with(const base_strview& w)const noexcept
```

Ritorna vero se la vista finisce con i caratteri di della vista `w`

```
[[nodiscard]] constexpr option<size_t>
raw_in(const base_strview& w)const noexcept
```

Ritorna la posizione della vista `w` nella vista corrente come indice grezzo.
`w` deve avere come istanza principale la stessa della vista corrente

```
[[nodiscard]] constexpr option<size_t> in(const base_strview& w)const noexcept
```

Ritorna la posizione della vista `w` nella vista corrente come indice di carattere.
`w` deve avere come istanza principale la stessa della vista corrente

```
[[nodiscard]] constexpr option<size_t> raw_index(size_t ch_index)const noexcept
```

Ritorna l'indice grezzo della posizione dell'indice carattere `ch_index`

```
[[nodiscard]] constexpr bool is_chr_boundary(size_t idx)const noexcept
```

Ritorna vero se l'indice di elemento `idx` puo essere convertito in un indice di carattere valido

```
template <typename T1, bool SAFE1>
[[nodiscard]] constexpr bool operator ==(const base_strview<T1, SAFE1>& w)const
noexcept
```

Ritorna vero per uguaglianza lessicografica con la vista `w`

```
template <typename U, std::size_t N>
[[nodiscard]] constexpr bool operator ==(const U(&v)[N])const noexcept
requires(concepts::charset<U> && N < max_size())
```

Ritorna vero per uguaglianza lessicografica con un `builtin array`
Se il valore di `v[N - 1]` è `\0` viene escluso dal confronto

```
template <typename T1, bool SAFE1>
[[nodiscard]] constexpr bool operator <(const base_strview<T1, SAFE1>& w)const
noexcept
```

Ritorna vero per minoranza lessicografica con la vista `w`

```
template <typename U, std::size_t N>
[[nodiscard]] constexpr bool operator <(const U(&v)[N])const noexcept
requires(concepts::charset<U> && N < max_size())
```

Ritorna vero per minoranza lessicografica con un `builtin array`
Se il valore di `v[N - 1]` è `\0` viene escluso dal confronto

```
template <typename T1,bool SAFE1>
[[nodiscard]] constexpr std::strong_ordering operator <=>(
        const base_strview<T1,SAFE1>& v
)const noexcept
```

Confronto lessicografico per `tree_way operator`

```
template <typename U, std::size_t N>
[[nodiscard]] constexpr std::strong_ordering operator <=>(const U(&v)[N])const
noexcept
requires(concepts::charset<U> && N < max_size())
```

Confronto lessicografico per `tree_way operator` con un `builtin array`
Se il valore di `v[N - 1]` è `\0` viene escluso dal confronto

vector

Uso:

astrazione di elementi contigui allocati in memoria dinamica

Testata:

```
iron/vector.h
```

Sinossi:

```
template <typename T,typename A> class base_vector final
```

Alias:

```
using value_t = T
using value_type = value_t
using allocator_t = A
using allocator_type = allocator_t
using size_t = std::size_t
using size_type = size_t
using diff_t = std::ptrdiff_t
using difference_type = diff_t
using ref_t = ref_wrapper<value_t>
using cref_t=ref_wrapper<const value_t>
using reference = value_t&
using const_reference = const value_t&
using ptr_t = value_t*
using cptr_t = const value_t*
using pointer = value_t*
using const_pointer = const value_t*
using iterator = <private>
using const_iterator = <private>
using reverse_iterator = <private>
using const_reverse_iterator = <private>
using seq_t = sequence<iterator>
using seq_type = seq_t
using cseq_t = sequence<const_iterator>
using const_seq_type = cseq_t
using rseq_t = sequence<reverse_iterator>
using rev_seq_type = rseq_t
using crseq_t = sequence<const_reverse_iterator>
using const_rev_seq_type = crseq_t
using error_handler_t = error_handler
using error_handler_type = error_handler_t
using adapter_box_t = <private>
using adapter_box_type = adapter_box_t
```

Alias esterni:

```
template <typename T, typename A =unsafe::allocator<T>>
using vector = base_vector<T,A>
```

Costanti statiche:

```
[[nodiscard]] static constexpr size_t max_size()noexcept
```

Ritorna il massimo numero di elementi memorizzabili

Vincoli:

```
!std::is_reference_v<value_t>
!std::is_array_v<value_t>
!std::is_const_v<value_t>
std::is_nothrow_move_constructible_v<value_t>
std::is_nothrow_swappable_v<value_t>
std::is_nothrow_destructible_v<value_t>
concepts::allocator<allocator_t>
```

Costruttori:

```
[[nodiscard]] static constexpr base_vector unchecked_from(allocator_t a)noexcept
```

Costruisce un vettore vuoto dall'allocatore `a`
Il metodo non è sicuro perché `allocator_t` non è sicuro

```
[[nodiscard]] static constexpr base_vector from(adapter_box_t&&)noexcept
```

Costruisce un vettore da adapter_t

```
constexpr base_vector()noexcept
base_vector(const base_vector&)=delete
constexpr base_vector(base_vector&&)noexcept
```

Distruttore:

```
constexpr ~base_vector() = default
```

Metodi:

```
base_vector& operator=(const base_vector&)=delete
constexpr base_vector& operator=(base_vector&&)noexcept
constexpr base_vector& swap(base_vector&)noexcept
```

```
[[nodiscard]] constexpr bool empty()const noexcept
```

Ritorna vero se il numero di elementi effettivi è zero

```
[[nodiscard]] constexpr explicit operator bool()const noexcept
```

Ritorna vero se il numero di elementi effettivi non è zero

```
[[nodiscard]] constexpr size_t size()const noexcept
```

Ritorna il numero di elementi effettivi

```
[[nodiscard]] constexpr size_t capacity()const noexcept
```

Ritorna il numero di elementi preallocati

```
[[nodiscard]] constexpr allocator_t unsafe_allocator()const noexcept
```

Ritorna l'allocatore
Il metodo non è sicuro perche `allocator_t` non è sicuro

```
constexpr base_vector& clear()noexcept
```

Riporta il numero di elementi effettivi a zero ma senza alterarne la capacità
Dopo questa operazione `size()` ritorna zero
Non ci devono essere referenti attivi

```
constexpr base_vector& reset()noexcept
```

Riporta il numero di elementi effettivi a zero liberando lo spazio occupato.
Dopo questa operazione `size()` e `capacity()` ritornano zero.
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr value_t* unsafe_data()noexcept
[[nodiscard]] constexpr const value_t* unsafe_data()const noexcept
[[nodiscard]] constexpr const value_t* unsafe_cdata()const noexcept
```

Ritorna un puntatore al primo elemento oppure `nullptr`
Il metodo non è sicuro perche ritorna un puntatore

```
[[nodiscard]] constexpr error_handler_t
try_reserve_at_least(size_t additional, size_t n = 1)noexcept

constexpr base_vector& reserve_at_least(size_t additional, size_t n = 1)noexcept
```

Riserva uno spazio per aggiungere almeno `addictional * n` elementi
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t
try_reserve_exact(size_t additional, size_t n = 1)noexcept

constexpr base_vector& reserve_exact(size_t additional, size_t n = 1)noexcept
```

Riserva uno spazio per aggiungere esattamente `addictional * n` elementi
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t try_shrink_to_fit()noexcept
```

Libera elementi preallocati inutilizzati
Non ci devono essere referenti attivi
L'operazione è transazionale

```
constexpr base_vector& shrink_to_fit()noexcept
```

Come `try_shrink_to_fit` ma in caso di errore esso viene ignorato

```
[[nodiscard]] constexpr adapter_box_t release()noexcept
```

Esporta gli elementi attivi in una istanza `adapter_box_t`
Non ci devono essere referenti attivi
L'operazione è transazionale
Dopo questa operazione `size()` e `capacity()` ritornano zero

```
[[nodiscard]] constexpr result<base_vector,error_handler_t>
try_clone(size_t n = 1)const noexcept
requires(concepts::copyable_or_cloneable<value_t>)

[[nodiscard]] constexpr base_vector clone(size_t n = 1)const noexcept
requires(concepts::copyable_or_cloneable<value_t>)
```

Ritorna un vettore copiato `n` volte

```
template<typename... ARGS>
[[nodiscard]] constexpr result<seq_t, error_handler_t>
try_push_back(ARGS&&... args)noexcept
requires (concepts::constructible_at<value_t,ARGS&&...>)

template<typename... ARGS>
constexpr seq_t push_back(ARGS&&... args)noexcept
requires (concepts::constructible_at<value_t,ARGS&&...>)
```

Appende un elemento costruito da `args`
Ritorna una sequenza il cui primo elemento si riferisce all'elemento inserito
L'operazione è transazionale
Non ci devono essere referenti attivi

```
template <typename SEQ>
[[maybe_unused]] constexpr result<seq_t, error_handler_t>
try_push_back_range(SEQ& seq)noexcept
requires(
        concepts::sequence<_SEQ>
        && !std::is_const_v<_SEQ>
        && concepts::constructible_at<value_t, std::decay_t<typename
        SEQ::value_t>&&>
)

template <typename SEQ>
constexpr seq_t push_back_range(SEQ& seq)noexcept
requires(
        concepts::sequence<_SEQ>
        && !std::is_const_v<_SEQ>
        && concepts::constructible_at<value_t, std::decay_t<typename
        SEQ::value_t>&&>
)
```

Appende i valori copiati dalla sequenza `seq`
Ritorna una sequenza il cui primo elemento si riferisce al primo elemento inserito
L'operazione è transazionale ed in caso di operazione riuscita `seq` sarà consumata
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr error_handler_t try_resize(size_t n)noexcept
requires(std::is_default_constructible_v<value_t>)

constexpr base_vector& resize(size_t n)noexcept
requires(std::is_default_constructible_v<value_t>)
```

Porta il numero di elementi effettivi a `n` con l'eccedenza a valore predefinito
Non ci devono essere referenti attivi
L'operazione è transazionale

```
[[nodiscard]] constexpr error_handler_t try_resize(size_t n,const value_t& v)noexcept
requires(concepts::copyable_or_cloneable<value_t>)

constexpr base_vector& resize(size_t n,const value_t& v)noexcept
requires(concepts::copyable_or_cloneable<value_t>)
```

Porta il numero di elementi effettivi a `n` con l'eccedenza a valore `v`
Non ci devono essere referenti attivi
L'operazione è transazionale

```
constexpr value_t unchecked_pop_back()noexcept
```

Ritorna, rimuovendolo, l'elemento di coda
L'operazione non è sicura perché il vettore potrebbe essere vuoto

```
constexpr option<value_t> pop_back()noexcept
```

Ritorna, rimuovendolo, l'elemento di coda
Non ci devono essere referenti attivi

```
[[nodiscard]] constexpr seq_t seq()noexcept
[[nodiscard]] constexpr cseq_t seq()const noexcept
[[nodiscard]] constexpr cseq_t cseq()const noexcept
```

Ritorna una sequenza sugli elementi

```
[[nodiscard]] constexpr rseq_t rseq()noexcept
[[nodiscard]] constexpr crseq_t rseq()const noexcept
[[nodiscard]] constexpr crseq_t crseq()const noexcept
```

Ritorna una sequenza inversa sugli elementi

```
[[nodiscard]] constexpr value_t& unchecked_nth(size_t idx) noexcept
[[nodiscard]] constexpr const value_t& unchecked_nth(size_t idx)const noexcept
[[nodiscard]] constexpr const value_t& unchecked_cnth(size_t idx)const noexcept
```

Ritorna l'elemento alla posizione `idx`, il primo elemento ha posizione zero
L'operazione non è sicura perché il valore di `idx` potrebbe essere non valido e ritornare una referenza non è sicuro

```
[[nodiscard]] constexpr value_t& unchecked_front()noexcept
[[nodiscard]] constexpr const value_t& unchecked_front()const noexcept
[[nodiscard]] constexpr const value_t& unchecked_cfront()const noexcept
```

Ritorna il primo elemento
L'operazione non è sicura perché il vettore potrebbe essere vuoto e ritornare una referenza non è sicuro

```
[[nodiscard]] constexpr value_t& unchecked_back()noexcept
[[nodiscard]] constexpr const value_t& unchecked_back()const noexcept
[[nodiscard]] constexpr const value_t& unchecked_cback()const noexcept
```

Ritorna l' ultimo elemento
L'operazione non è sicura perché il vettore potrebbe essere vuoto e ritornare una referenza non è sicuro

```
[[nodiscard]] constexpr option<ref_t> nth(size_t idx)noexcept
[[nodiscard]] constexpr option<cref_t> nth(size_t idx)const noexcept
[[nodiscard]] constexpr option<cref_t> cnth(size_t idx)const noexcept
```

Ritorna l'elemento alla posizione `idx`, il primo elemento ha posizione zero

```
[[nodiscard]] constexpr ref_t operator[](size_t idx)noexcept
[[nodiscard]] constexpr ref_t at(size_t idx)noexcept
[[nodiscard]] constexpr cref_t operator[](size_t idx)const noexcept
[[nodiscard]] constexpr cref_t at(size_t idx)const noexcept
```

Ritorna l'elemento alla posizione `idx`, il primo elemento ha posizione zero
In caso di `idx` a valore non valido viene chiamata `PANIC()`

```
[[nodiscard]] constexpr option<ref_t> front()noexcept
[[nodiscard]] constexpr option<cref_t> front()const noexcept
[[nodiscard]] constexpr option<cref_t> cfront()const noexcept
```

Ritorna il primo elemento

```
[[nodiscard]] constexpr option<ref_t> back()noexcept
[[nodiscard]] constexpr option<cref_t> back()const noexcept
[[nodiscard]] constexpr option<cref_t> cback()const noexcept
```

Ritorna l' ultimo elemento

```
template <typename A1>
[[nodiscard]] constexpr bool operator==(const base_vector<value_t,A1>& v)const
noexcept
requires (concepts::nothrow_equality_comparable<value_t>)
```

Ritorna il confronto per uguaglianza degli elementi

Version

Uso:

memorizzare la versione della libreria

Testata:

`iron/version.h`

Macro:

```
IRON_VERSION <major>,<minor>,<patch_release>
```